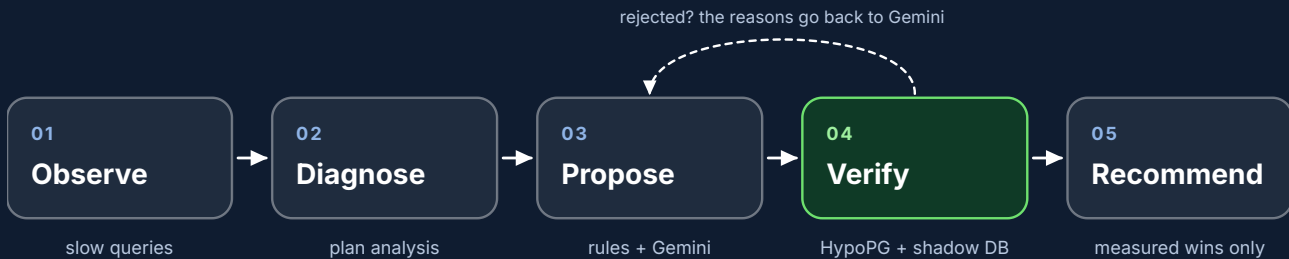


A PostgreSQL query advisor that checks its own work

The complete project guide: from what an index is, all the way to how every part of the verification engine works.



Project

qopt 0.1.0 · a TypeScript monorepo

Built with

PostgreSQL 17 · HypoPG · pgvector ·
Gemini

Edition

30 September 2026

About this guide

This guide explains **qopt** completely, in one place. It starts with what a database index is, builds up the ideas the project depends on, walks through the architecture, lists every technology used, and ends with a line-by-line tour of the most important code and the measured results.

Who it is for

It is written for someone who is new to database performance and to AI tooling, but who can read a little code. Every term is explained the first time it appears (there is also a glossary in [Appendix A](#)). If you already know PostgreSQL well, you can skip Part I.

Suggested reading paths

If you want to...	Read
understand what the project is and why it matters	Chapter 1 , then Chapter 30
learn the database and AI concepts from scratch	Part I (Chapters 1–6)
understand how the system is designed	Part II (Chapters 7–13)
know which technologies are used, and why	Part III (Chapters 14–15)
read and change the code	Part IV (Chapters 16–27)
run it, and see what it achieved	Part V (Chapters 28–31)

Conventions

BEGINNER NOTE Background for readers meeting an idea for the first time.	KEY IDEA The one thing to remember from a section.
WATCH OUT A trap, a limitation, or something that surprised us.	IN THE CODE Where to find the idea in the repository.

Code excerpts are copied from the repository when this guide is built. The caption shows the file path and the exact line numbers, so you can open the same lines in your editor. Numbers quoted in the text (timings, sizes, hit rates) come from real runs on the demo database described in [Chapter 2](#).

The project at a glance

What it is	A PostgreSQL slow-query advisor. It finds slow queries, explains why they are slow, and recommends fixes that it has verified.
Main idea	Suggestions from rules and from an LLM (Gemini) are tested against ground truth: hypothetical indexes (HypoPG) and real experiments on a shadow copy of the database, inside transactions that are always rolled back.
Built as	A TypeScript monorepo: a core engine, a command-line tool, an HTTP API and a web UI.
Databases	Three PostgreSQL 17 instances: <i>target</i> (being tuned), <i>shadow</i> (the lab), <i>meta</i> (qopt's own storage).
Headline result	On a 15-query demo workload, validated suggestions made the workload 5.2× faster. The same LLM suggestions applied without validation gave only 1.3×.

Contents

PART I · FOUNDATIONS	6
1 What is qopt?	7
1.1 The problem it solves	7
1.2 What qopt does	7
1.3 What makes it different	8
1.4 What you get	8
1.5 A one-minute example	8
2 Databases and SQL in ten minutes	9
2.1 Tables, rows and columns	9
2.2 Asking questions with SQL	9
2.3 PostgreSQL in one page	10
2.4 Transactions: the idea qopt's safety is built on	10
3 Why queries get slow, and how indexes help	11
3.1 The full table scan	11
3.2 B-tree indexes	11
3.3 Multicolumn indexes and column order	12
3.4 Indexes that avoid sorting and table visits	12
3.5 Expression and partial indexes	12
3.6 GIN and trigram indexes	13
3.7 BRIN and hash indexes	13
3.8 Choosing an index: a cheat sheet	14
3.9 Slowness that indexes do not fix	14
4 How PostgreSQL decides: the planner, EXPLAIN and statistics	15
4.1 The life of a query	15
4.2 Plans are trees	15
4.3 EXPLAIN and EXPLAIN ANALYZE	16
4.4 Cost: the planner's currency	17
4.5 Statistics: what the planner knows about your data	17
4.6 When estimates go wrong	18
4.7 Self time, loops and parallel workers	18
4.8 Queries with parameters	18
5 LLMs, embeddings and RAG: just enough	19
5.1 What a large language model is	19
5.2 Why every answer must be checked	19
5.3 Structured output	19
5.4 Embeddings and vector search	19
5.5 Keyword search and hybrid ranking	20
5.6 Retrieval-augmented generation (RAG)	20
6 The tools that make verification possible	21
6.1 pg_stat_statements: what is slow, in aggregate	21
6.2 auto_explain: the real plans	21
6.3 HypoPG: hypothetical indexes	21
6.4 The shadow database	21
6.5 How the four fit together	22
PART II · ARCHITECTURE	23
7 The big picture	24
7.1 Components	24
7.2 The three databases	25
7.3 One engine, three front ends	25
7.4 Where everything lives	25
8 The analysis pipeline, stage by stage	27
8.1 What each stage does	27
8.2 Which plan is "the baseline"?	28
8.3 The findings catalogue	28
8.4 The result object	29
9 Capturing slow queries	30
9.1 Finding the most expensive statements	30
9.2 Reading plans from the server log	30
9.3 The 64-bit query id trap	30
9.4 When there is no sample	31

10	The validation engine	32
10.1	Gate 1: static checks with the real parser	32
10.2	Gate 2: HypoPG on the target	33
10.3	Gate 3: a real experiment on the shadow	33
10.4	Thresholds and ranking	34
10.5	Statistics and settings	34
10.6	Rewrites: same rows, then faster	34
10.7	When PostgreSQL says no: error classes	35
11	The LLM loop	36
11.1	What Gemini is told	36
11.2	What Gemini must answer	38
11.3	Feedback and revision	38
11.4	Surviving overload and quotas	39
11.5	Case studies from real runs	39
12	Documentation retrieval (RAG)	40
12.1	The corpus	40
12.2	Chunking	40
12.3	Embeddings and storage	40
12.4	Hybrid search	41
12.5	What RAG did, and did not, do	41
13	The safety model	42
PART III · TECHNOLOGIES		43
14	The technology stack	44
14.1	Language and runtime	44
14.2	Database	44
14.3	Core libraries	45
14.4	Applications	45
14.5	Testing, tooling and infrastructure	45
14.6	AI models	46
14.7	Design choices worth knowing	46
15	Project structure and configuration	47
15.1	The repository	47
15.2	Settings (.env)	47
15.3	npm scripts	48
15.4	Processes and ports	48
PART IV · CODE WALKTHROUGH		49
16	Configuration and database access	50
16.1	Settings with safe defaults	50
16.2	Getting numbers back as numbers	50
16.3	The Db class: pools, tags and read-only sessions	51
16.4	sandbox(): the lab pattern	51
16.5	Four roles, created once	52
17	Understanding SQL: parser, query shape, safety	53
17.1	Loading the real PostgreSQL parser	53
17.2	From a parse tree to a “query shape”	54
17.3	Deciding whether SQL may run	55
18	Reading plans and finding problems	56
18.1	Self time and q-error	56
18.2	Reporting a misestimate once, where it starts	56
18.3	Measuring a query fairly	57
19	Capture and schema context	58
19.1	The statements query	58
19.2	Parsing auto_explain’s log lines	58
19.3	Schema context: the statistics query	59
20	The heuristic advisor	60
20.1	The rules	60
20.2	Rules 1–4 in code	61
20.3	Sizing work_mem from the evidence	61
21	The validator	62
21.1	HypoPG in one session	62
21.2	The index path, end to end	62
21.3	Rewrites: build, lock down, compare, time	64

21.4	Errors and scores	65
22	The LLM layer	66
22.1	A two-method interface	66
22.2	The Gemini client: cache, fallback, parse	66
22.3	One schema, used twice	68
22.4	The conversation	68
23	The RAG code	69
23.1	Chunking a manual page	69
23.2	Embedding and storing	70
23.3	Hybrid search with RRF	70
24	The pipeline orchestrator	72
24.1	Timed, observable stages	72
24.2	Validating candidates one by one	72
24.3	The generate → validate → revise loop	73
25	Demo workload and evaluation harness	74
25.1	Deterministic data	74
25.2	Queries with known answers	74
25.3	Scoring a configuration	75
26	The applications: CLI, API, web UI	76
26.1	The CLI	76
26.2	The API: one experiment at a time	76
26.3	The web UI	77
27	Tests	79
27.1	Testing the LLM loop without an LLM	79
27.2	Safety rules as tests	80
PART V · USING IT, AND WHAT IT ACHIEVED		81
28	Setup and daily use	82
28.1	Requirements	82
28.2	Option A: Homebrew (no Docker)	82
28.3	Option B: Docker	82
28.4	First run	82
28.5	Everyday commands	83
28.6	Troubleshooting	83
28.7	Using it on your own database	83
29	Worked examples	84
29.1	In the terminal: from capture to recommendation	84
29.2	In the browser: the Analyze page	85
29.3	The feedback loop at work (W13)	86
29.4	When validation rejects everything (W10, lite model)	87
30	Evaluation results	88
30.1	Method	88
30.2	Results	88
30.3	What the numbers say	89
30.4	How much to trust these numbers	90
31	Limitations and future work	91
31.1	Known limitations	91
31.2	Ideas for future work	91
A	Glossary	92
B	CLI reference	94
C	HTTP API reference	95
D	The 15 demo queries	96

PART I

Foundations

Everything you need to know before reading the design and the code: what the project does, how databases store and find data, why queries become slow, how PostgreSQL chooses a plan, and the small amount of AI theory the project relies on.

- Chapter 1 · What is qopt?
- Chapter 2 · Databases and SQL in ten minutes
- Chapter 3 · Why queries get slow, and how indexes help
- Chapter 4 · How PostgreSQL decides: the planner, EXPLAIN and statistics
- Chapter 5 · LLMs, embeddings and RAG: just enough
- Chapter 6 · The tools that make verification possible

What is qopt?

qopt looks at a PostgreSQL database, finds the queries that are slow, works out why, and suggests fixes: new indexes, rewritten queries, better statistics, or a memory setting. What makes it different is that it never recommends a fix on faith. Every suggestion is tested against the real query planner and real data first, and only proven improvements reach you.

1.1 The problem it solves

Applications talk to databases through **queries**: small programs written in SQL, such as “give me the 20 most recent orders of customer 42”. When a table is small, every query is fast. As data grows, some queries slow down dramatically. On the demo database used throughout this guide, one query takes about 700 milliseconds; with the right index it takes 150. Another drops from 95 ms to under 1 ms.

Finding the right fix is expert work. You have to read the database’s **execution plan**, know which kind of index suits which kind of filter, and understand how the database estimates costs. It is tempting to ask an AI model instead, and models are good at producing plausible advice. But plausible is not the same as correct. During this project the Gemini model made exactly the kinds of mistakes an expert would catch:

- It proposed an index on `date(created_at)`. PostgreSQL refuses that index, because `date()` of a timestamp-with-time-zone depends on the session’s time zone.
- It proposed a rewritten query that looked equivalent but returned different rows, because it hard-coded a time-zone offset.
- It proposed raising a memory setting for the slowest query. That made the query *slower* (704 → 735 ms).
- It bundled a `CREATE EXTENSION` statement into what should have been a single index suggestion.

Applied blindly, advice like this wastes time at best and returns wrong answers at worst.

1.2 What qopt does

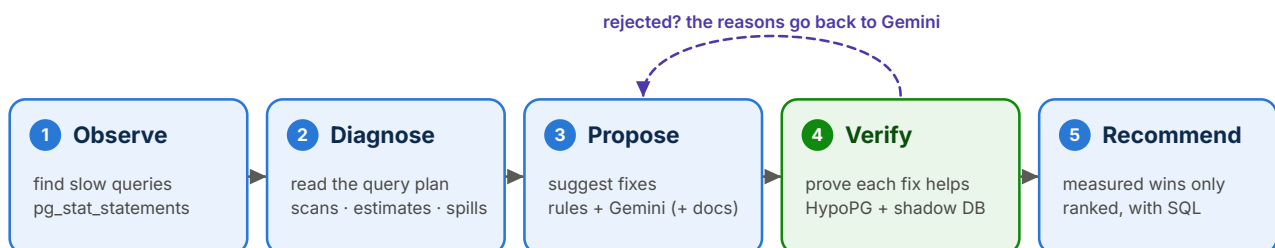


Figure 1. The five steps. Step 4 is what makes qopt different: nothing is recommended until it has been checked, and rejections go back to the model with the reason.

- 1 Observe.** Two PostgreSQL extensions record which queries use the most time, and capture the real execution plan of slow runs together with the actual parameter values.
- 2 Diagnose.** qopt reads the plan and runs fifteen deterministic checks, such as “a full scan that throws away 99.9% of rows” or “a sort that spilled to disk”.
- 3 Propose.** Candidate fixes come from two sources: a rule-based advisor, and the Gemini model, which gets the plan, the findings, the schema and relevant pages of the PostgreSQL manual.

- 4 **Verify.** Each candidate is checked. Indexes are first simulated with *hypothetical indexes* (HypoPG), then built for real on a *shadow* copy of the database and timed. Rewritten queries must return exactly the same rows. Everything happens inside a transaction that is rolled back, so nothing changes.
- 5 **Recommend.** Only candidates that measurably helped are recommended, ranked by the measured gain and by cost (such as index size).

1.3 What makes it different

KEY IDEA

An AI suggestion is a *hypothesis*. qopt treats it like one: it runs the experiment and keeps the suggestion only if the evidence supports it. This “generate, then verify” loop turns a text generator into an engineering tool.

Typical “AI for SQL” tool	qopt
Returns text that sounds right	Returns changes that were measured to work, with the numbers
Cannot tell a wrong rewrite from a right one	Runs both queries and compares the results row by row
May invent columns that don’t exist	PostgreSQL itself rejects them; the error is shown and classified as a hallucination
One-shot answer	Rejections go back to the model with the reason, and it tries again
No way to know if it helps	An evaluation harness measures hit rate, precision and speedup against known answers

1.4 What you get

- **A command-line tool** (`qopt`) to set up databases, generate a workload, capture slow queries, analyse them, ingest documentation and run evaluations.
- **A web UI** showing the slow queries, the analysis progress, every candidate with its verdict, the plan before and after, and the evaluation results.
- **An evaluation harness** that compares four configurations on 15 queries with known correct fixes. Validated suggestions made the workload 5.2× faster (5.7× with documentation retrieval). The same LLM suggestions applied without validation managed 1.3×, because one bad suggestion slowed the heaviest query.

1.5 A one-minute example

Workload query W13 counts, for each product, how many distinct orders it appears in. It takes about **704 ms**. Here is what happened when qopt analysed it with Gemini:

- 1 The plan analysis found that a sort of 1.8 million rows spilled 45 MB to disk, taking 73% of the time.
- 2 Gemini’s first suggestion was the obvious one: give the sort more memory (`SET work_mem = '64MB'`).
- 3 qopt measured it on the shadow database: 704 → 735 ms, **4.5% slower**. Rejected, and the reason was sent back.
- 4 Gemini revised: an index on `order_items (product_id, order_id)` , which hands the rows over already sorted. HypoPG estimated the cost 75% lower. qopt then built the index for real on the shadow copy: **704 → 149 ms (–79%)**. Accepted, and recommended.

Nobody had to trust the model: the database told us which idea worked. [Chapter 29](#) shows this run in the web UI.

Databases and SQL in ten minutes

This chapter introduces the handful of database ideas the rest of the guide depends on, using the demo shop database that qopt creates for testing.

2.1 Tables, rows and columns

A relational database stores data in **tables**. Each table has named **columns** with a type (number, text, timestamp, JSON...), and holds any number of **rows**. qopt's demo database models a small online shop with six tables and about 4.2 million rows in total:

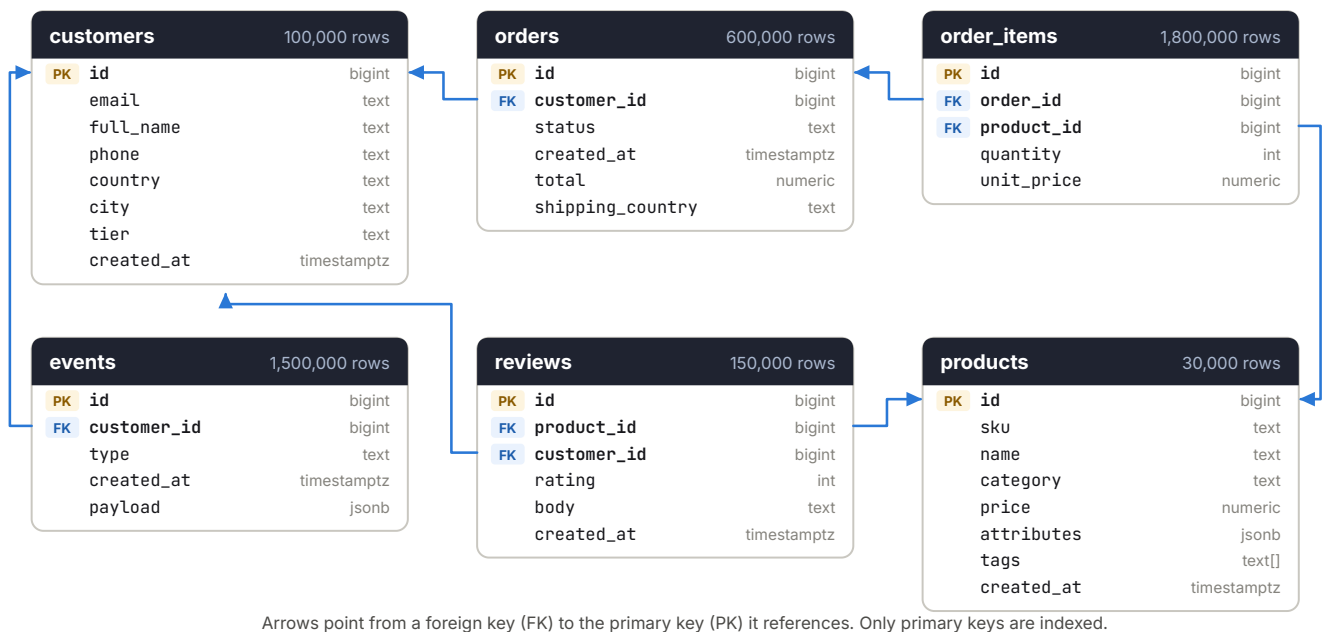


Figure 2. The demo shop schema created by `qopt db setup`. Each table has only a primary-key index: every other index is missing on purpose, so there is something to find.

Every table has a **primary key** (PK), a column whose value uniquely identifies a row: here always `id`. A **foreign key** (FK) is a column that points at another table's primary key: `orders.customer_id` says which customer placed the order. PostgreSQL automatically creates an index for each primary key, but *not* for foreign keys. Missing indexes on foreign keys are one of the most common causes of slow queries in real applications, and the demo reproduces that on purpose.

2.2 Asking questions with SQL

SQL describes *what* you want, not *how* to get it. Here are three of the demo workload's queries, read clause by clause:

W01 · a customer's latest orders

```
SELECT id, status, total, created_at  -- the columns to return
FROM orders                          -- the table to read
WHERE customer_id = 42                -- keep only this customer's rows
ORDER BY created_at DESC              -- newest first
LIMIT 20;                             -- only the first 20
```

W11 · combining two tables with a JOIN, then grouping

```
SELECT c.id, count(o.id) AS orders
FROM customers c
JOIN orders o ON o.customer_id = c.id      -- match each order to its customer
WHERE c.country = 'NZ' AND c.city = 'Auckland'
GROUP BY c.id;                          -- one output row per customer
```

W13 · aggregating 1.8 million rows

```
SELECT product_id, count(DISTINCT order_id) AS orders
FROM order_items
GROUP BY product_id
ORDER BY orders DESC
LIMIT 10;
```

Because SQL only says *what*, the database must decide *how*: which table to read first, whether to use an index, how to join, and how to sort. That decision is called the **query plan**, and it is the subject of [Chapter 4](#).

2.3 PostgreSQL in one page

- **Server and databases.** A PostgreSQL server (also called a *cluster*) listens on a network port and holds one or more databases. qopt uses three servers on ports 5433, 5434 and 5435, each with a database called `qopt`.
- **Connections and sessions.** A program opens a connection and sends SQL. Settings changed with `SET` apply only to that session.
- **Extensions.** Plugins that add features, installed with `CREATE EXTENSION`. qopt relies on five: `pg_stat_statements`, `auto_explain`, `hypopg`, `pg_trgm` and `vector` (pgvector).
- **System catalogs.** PostgreSQL describes itself in ordinary tables such as `pg_class` (tables and indexes), `pg_index`, `pg_attribute` (columns) and `pg_stats` (data statistics). qopt reads these to understand your schema.
- **Pages.** Table and index data is stored in 8 kB *pages*. Reading a page from disk is the basic unit of work; recently used pages stay in memory (the buffer cache). Performance tuning is largely about reading fewer pages.

2.4 Transactions: the idea qopt's safety is built on

A **transaction** groups statements so that they succeed or fail as a unit. You start one with `BEGIN` and end it with `COMMIT` (keep the changes) or `ROLLBACK` (undo everything). Unusually, PostgreSQL can also roll back *schema changes*, such as creating an index. That is called *transactional DDL* (DDL means statements that change structure, such as `CREATE INDEX`; DML means statements that change rows, such as `UPDATE`), and qopt depends on it:

An experiment that leaves no trace

```
BEGIN;
CREATE INDEX ON orders (customer_id);      -- really built, on the shadow copy
EXPLAIN ANALYZE SELECT ... WHERE customer_id = 42; -- measure with the index
ROLLBACK;                                  -- the index never existed
```

Two refinements appear in the code. A **savepoint** (`SAVEPOINT name ... ROLLBACK TO SAVEPOINT name`) undoes part of a transaction; qopt uses one around each timed run, so that even an `UPDATE` sees identical data every time. A **read-only transaction** (`BEGIN READ ONLY`, or `SET transaction_read_only = on`) makes PostgreSQL refuse any write, which is how qopt runs model-written SQL safely.

Why queries get slow, and how indexes help

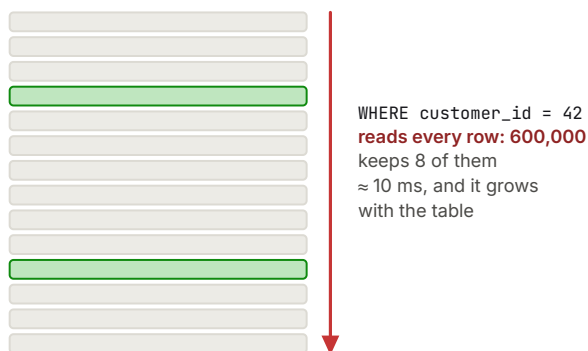
Most slow queries read far more data than they need. An index lets the database jump straight to the rows it wants. This chapter explains the index types qopt can recommend, and the problems indexes cannot fix.

3.1 The full table scan

Without a suitable index, the only way to answer `WHERE customer_id = 42` is a **sequential scan**: read every row of `orders` and keep the ones that match. It is like finding a name in a phone book whose pages are in random order: you must read the whole book. The cost grows with the table, whatever the size of the answer.

Sequential scan

no usable index



Index scan

B-tree on `customer_id`

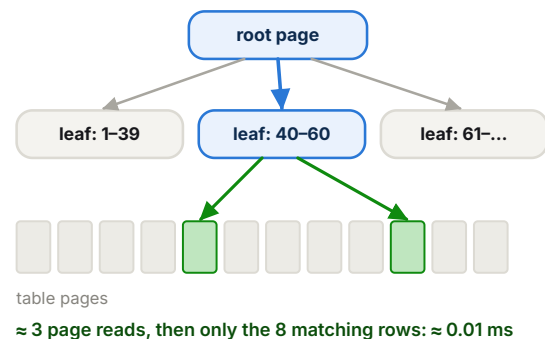


Figure 3. W01 on the demo data: the scan reads 600,000 rows to return 8 (about 10 ms). With a B-tree on `customer_id`, the same query reads a few pages (about 0.01 ms).

3.2 B-tree indexes

An **index** is a separate, sorted structure that maps column values to row locations. PostgreSQL's default index type is the **B-tree**: a balanced tree of pages whose leaves hold the keys in sorted order, each pointing to a row in the table (the *heap*).

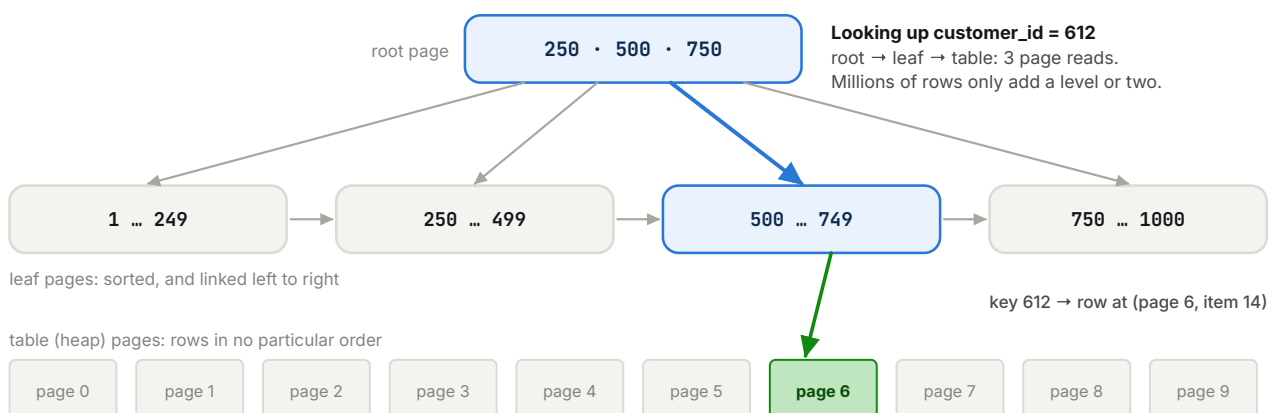


Figure 4. A lookup walks from the root to one leaf, then to the row. Because leaves are sorted and linked, a range (`BETWEEN`, `>`) is one walk plus a sideways read, and the keys come out already in order.

B-trees serve equality (= , IN), ranges (< , > , BETWEEN), sorting (ORDER BY) and prefix matches (LIKE 'abc%'). They are not free, though. An index takes disk space (the recommended W13 index is 54 MB), and every INSERT , UPDATE or DELETE must also update every index on the table. That is why qopt penalizes large indexes and indexes on write-heavy tables when it ranks recommendations.

3.3 Multicolumn indexes and column order

An index can cover several columns, and the **order of the columns matters**. The index is sorted by the first column, then by the second within equal first values, and so on, like a phone book sorted by surname, then first name.

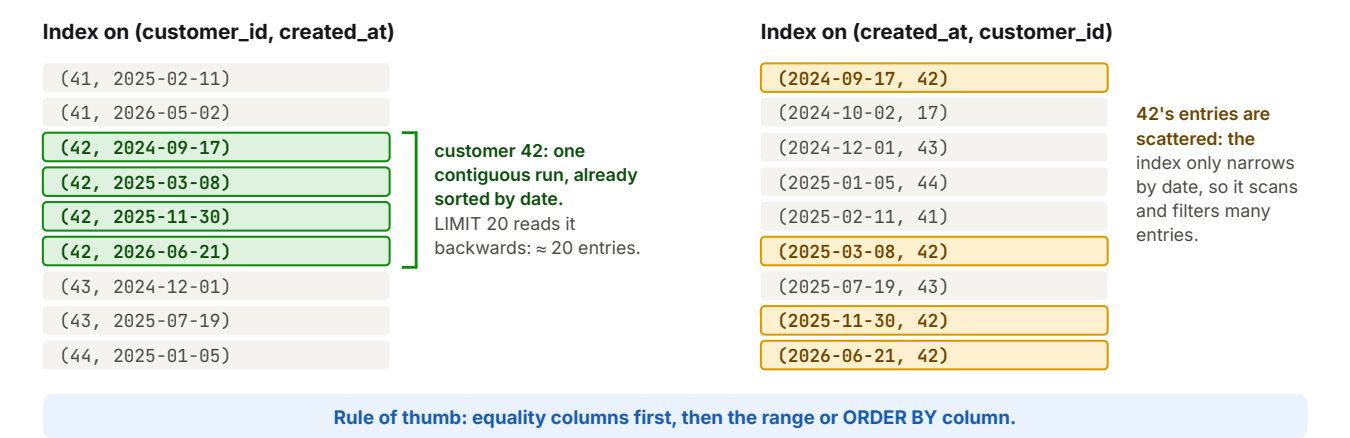


Figure 5. For “customer 42’s latest orders”, (customer_id, created_at) puts all of customer 42’s rows together, already in date order. The reverse order scatters them.

The rule qopt’s heuristic advisor follows is the classic one: put the columns tested with **equality first**, then **one range** or **ORDER BY** column. An index on (a, b) can also serve queries on a alone (the “leftmost prefix”), but not queries on b alone.

3.4 Indexes that avoid sorting and table visits

- **Avoiding a sort.** If the index order matches ORDER BY , the database reads entries in order and stops after LIMIT rows. W14 (deep pagination) drops from 53 ms to 4.6 ms with an index on created_at .
- **Index-only scans.** If every column the query needs is in the index, PostgreSQL can skip the table entirely. INCLUDE (col) adds extra columns to the leaf entries without making them sort keys; that is a **covering index**. This works fully only when the table’s visibility map says pages are “all visible”, which VACUUM maintains. qopt flags index-only scans that still visit the heap (finding HEAP_FETCHES).

3.5 Expression and partial indexes

An **expression index** indexes the result of a function. W03 searches WHERE lower(email) = lower('...') ; a plain index on email is useless because the query compares lower(email) . An index on (lower(email)) matches exactly (17 ms → 0.01 ms).

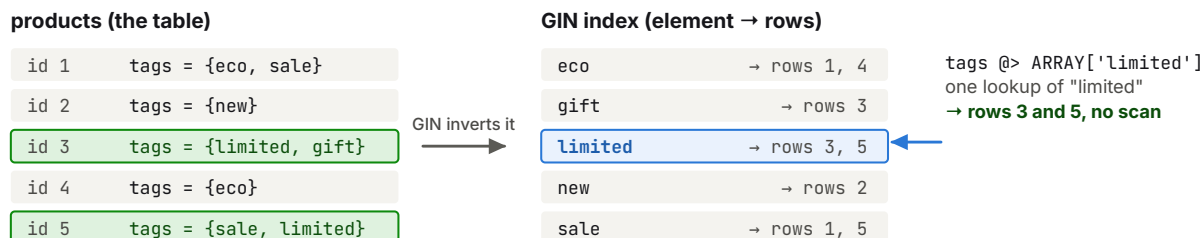
WATCH OUT

Index expressions must be **IMMUTABLE**: always the same output for the same input. date(created_at) on a timestamptz column is *not*, because the calendar date depends on the session time zone. PostgreSQL refuses such an index. Both qopt’s rule-based advisor and Gemini suggested it for W10, and validation caught it both times. The correct fix is to rewrite the filter as a range (created_at >= '2026-03-30' AND created_at < '2026-03-31').

A **partial index** indexes only the rows matching a condition. W04 looks for recent *pending* orders, a tiny fraction of the table. `CREATE INDEX ... (created_at) WHERE status = 'pending'` is small, cheap to maintain, and took the query from 16.4 ms to 0.08 ms.

3.6 GIN and trigram indexes

Some operators look *inside* a value: “does this array contain ‘limited’?” (`tags @> ARRAY['limited']`) or “does this JSON contain `{"color": "teal"}`?”. A B-tree over whole values cannot answer these. A **GIN** (Generalized Inverted Index) stores, for every element or key, the list of rows containing it, like the index at the back of a book.



Trigrams (pg_trgm) use the same idea for text:

'overheats' → ove ver erh rhe hea eat ats so `ILIKE '%overheat%'` can use GIN

Figure 6. GIN inverts the data: from “row → elements” to “element → rows”. `pg_trgm` splits text into three-letter *trigrams*, so even `ILIKE '%overheat%'` can use an index.

For `jsonb`, the operator class `jsonb_path_ops` builds a smaller, faster GIN index that supports only `@>`. For text search with a leading wildcard, the `pg_trgm` extension provides `gin_trgm_ops`. W07 (`body ILIKE '%overheat%'`) goes from 95 ms to 0.9 ms with it.

3.7 BRIN and hash indexes

A **BRIN** (Block Range INdex) stores only a minimum and a maximum for each run of 128 table pages. It works when a column follows the physical order of the table, as timestamps do in an append-only log. PostgreSQL reports this as a *correlation* close to 1.0 in `pg_stats`.

events: 1.5 M rows appended in time order (created_at correlation ≈ 1.0)



BRIN stores one (min, max) summary per block range, 128 pages by default.

`WHERE created_at BETWEEN 'May 10' AND 'May 11'`

Only the green range can contain matches, so only its pages are read and rechecked. The rest are skipped.

The whole index is a handful of summaries: typically hundreds to thousands of times smaller than a B-tree.

Figure 7. BRIN skips whole block ranges whose [min, max] cannot match. It is tiny, but useful only for naturally ordered data.

A **hash** index supports only equality and is rarely better than a B-tree. `qopt` can validate hash indexes, but its rules never propose them.

3.8 Choosing an index: a cheat sheet

Query pattern	Index	Demo query
<code>col = value</code>	B-tree on <code>(col)</code>	W02
<code>a = x AND b > y</code> , or <code>a = x ORDER BY b LIMIT n</code>	B-tree on <code>(a, b)</code>	W01, W04
filter on a rare constant plus a range	partial B-tree ... <code>WHERE status = 'pending'</code>	W04
<code>lower(email) = ...</code>	expression index <code>((lower(email)))</code>	W03
<code>array @> ...</code> , <code>jsonb @> ...</code>	GIN (<code>jsonb_path_ops</code> for jsonb)	W05, W06
<code>LIKE '%text%'</code> , <code>ILIKE</code>	GIN with <code>gin_trgm_ops</code>	W07
time ranges on huge append-only tables	BRIN, or a B-tree	W08
<code>a = x OR b = y</code>	one index per branch (combined with BitmapOr)	W15
join column on a big table	B-tree on the foreign key	W11, W12
<code>GROUP BY a</code> with <code>count(DISTINCT b)</code>	B-tree on <code>(a, b)</code> : pre-sorted input	W13

3.9 Slowness that indexes do not fix

- **Sorts and hashes that spill to disk.** Each sort or hash may use `work_mem` of memory (4 MB by default). Beyond that it writes temporary files, shown in plans as `Sort Method: external merge Disk: 45808kB`. Raising `work_mem` for one session can help, but not always: for W13 it made no real difference.
- **Non-sargable predicates.** A filter that wraps a column in a function (`date(created_at) = ...`) cannot use a plain index on the column. ("Sargable" means "can use a search argument", i.e. an index.) The fix is a rewrite or an expression index.
- **NOT IN (subquery)**. Besides being hard to optimize, it has a surprising meaning: if the subquery returns a single `NULL`, `NOT IN` returns no rows at all. `NOT EXISTS` is usually what was meant, and the planner can execute it as an efficient *anti-join*. The two are equivalent only when the column can never be `NULL`, which is why qopt compares results on real data before trusting such a rewrite.
- **Deep OFFSET**. `OFFSET 50000 LIMIT 50` still produces and throws away 50,000 rows. An index helps; "keyset pagination" (`WHERE created_at < last_seen`) avoids the waste, but it changes the application, so qopt only mentions it.
- **OR across different columns.** `email = ... OR phone = ...` cannot use a single index. With one index per column, PostgreSQL combines them with a *BitmapOr*. An index on only one of the two columns does not help at all.

How PostgreSQL decides: the planner, EXPLAIN and statistics

Every SQL query can be executed in many ways. PostgreSQL's planner estimates the cost of each option and picks the cheapest. qopt's diagnosis is mostly about reading those decisions and spotting where the planner was misled or had no good option.

4.1 The life of a query

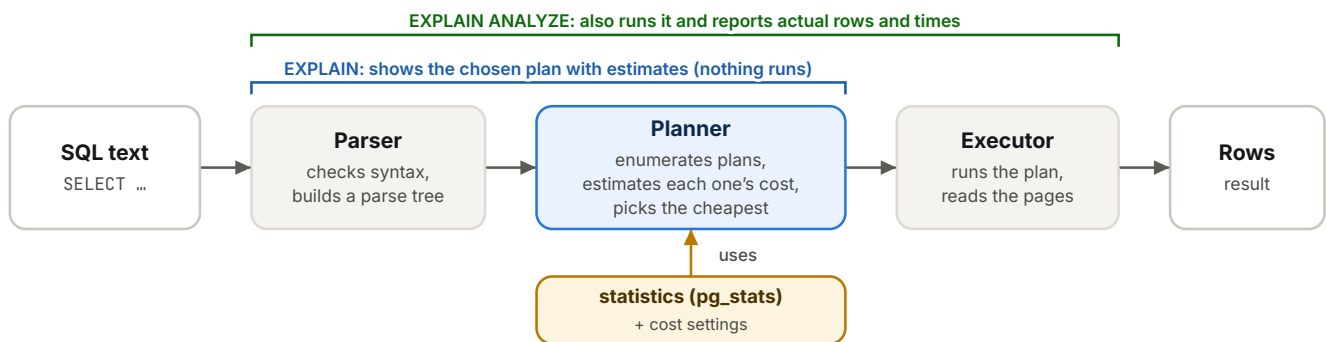


Figure 8. The parser checks the SQL and turns it into a tree. The planner chooses a plan using statistics about the data. The executor runs it. EXPLAIN stops after planning; EXPLAIN ANALYZE also executes.

4.2 Plans are trees

A plan is a tree of **nodes**. Leaves read tables; inner nodes join, sort, aggregate or limit the rows their children produce. Here is the real plan of W11 ("orders from Auckland customers"), which qopt drew from the

EXPLAIN ANALYZE output:

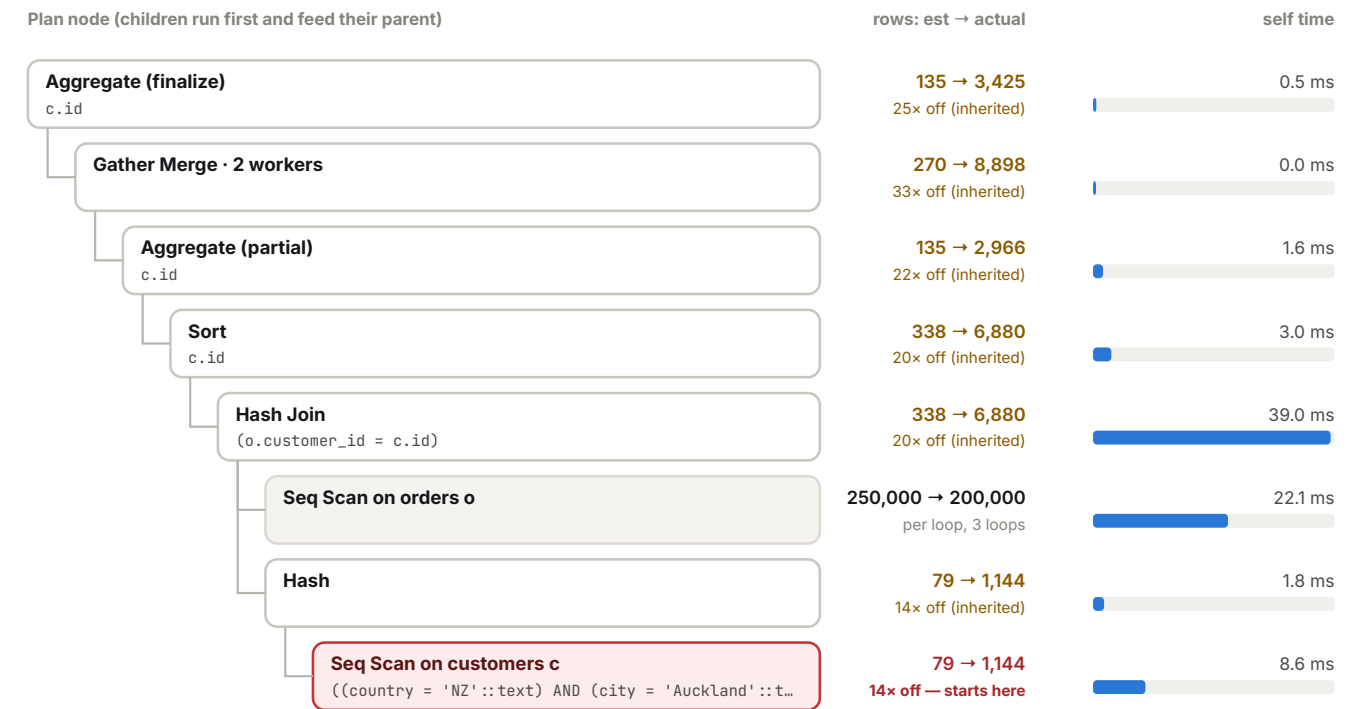


Figure 9. W11's real plan (PostgreSQL 17, demo data). Read it bottom-up: scan customers, build a hash table, scan orders and join, then sort, aggregate and merge the results from 2 parallel workers. The misestimate starts at the customers scan (79 estimated rows, 1,144 actual per worker, 14× off) and every parent inherits it. Self times are summed over the 3 parallel processes, so they add up to more than the 29 ms wall-clock time.

Node type	What it does
Seq Scan	Reads every row of a table, applying the filter.
Index Scan	Walks an index, then fetches each matching row from the table.
Index Only Scan	Answers from the index alone (covering index).
Bitmap Index Scan + Bitmap Heap Scan	Collects matching row locations from one or more indexes into a bitmap (<code>BitmapOr</code> / <code>BitmapAnd</code>), then reads the table pages in physical order.
Nested Loop , Hash Join , Merge Join	The three ways to join. A nested loop is fast when the outer side is small and the inner side has an index; a hash join builds an in-memory hash table from one side.
Sort , Aggregate , Limit	Order rows; compute <code>count</code> and <code>sum</code> or group; stop early.
Gather / Gather Merge	Combine rows from parallel worker processes.

4.3 EXPLAIN and EXPLAIN ANALYZE

`EXPLAIN query` shows the chosen plan with the planner's *estimates*, without running anything.

`EXPLAIN (ANALYZE)` actually executes the query and adds the *actual* rows, loops and time for every node.

`BUFFERS` adds how many pages were read, and `FORMAT JSON` makes the output machine-readable. `qopt` always uses JSON. This is one node of what it receives. Note that rows and time are *per loop*: 3 loops of 2.868 ms is 8.6 ms of work.

The customers scan from W11's EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), abridged

```
{
  "Node Type": "Seq Scan",
  "Relation Name": "customers",
  "Alias": "c",
  "Startup Cost": 0.00,
  "Parallel Aware": true,
  "Total Cost": 2238.35,
  "Plan Rows": 79,
  "Actual Rows": 1144,
  "Actual Loops": 3,
  "Actual Total Time": 2.868,
  "Filter": "((country = 'NZ'::text) AND (city = 'Auckland'::text))",
  "Rows Removed by Filter": 32189,
  "Shared Hit Blocks": 1356
}
```

WATCH OUT

EXPLAIN ANALYZE **really runs the statement**. For a DELETE or UPDATE it really changes data, and a slow query really puts load on the server. This is why qopt never runs EXPLAIN ANALYZE on the target database. It uses the shadow copy, inside transactions that are rolled back.

4.4 Cost: the planner's currency

The planner compares plans using an abstract **cost**. Roughly, reading a page sequentially costs

`seq_page_cost = 1`, reading a page at a random position costs `random_page_cost = 4`, processing a row costs `cpu_tuple_cost = 0.01`, and evaluating an operator costs `cpu_operator_cost = 0.0025`. A worked example with the real demo numbers: `orders` has 600,000 rows in 6,184 pages, so a sequential scan with one filter costs

$$6,184 \times 1 + 600,000 \times 0.01 + 600,000 \times 0.0025 = 13,684$$

That is exactly the cost PostgreSQL printed for the scan of `orders` in W09. An index scan instead costs a few random page reads per matching row, so the planner prefers it when few rows match. It can only make that call correctly if it knows how many rows *will* match. That knowledge comes from statistics.

4.5 Statistics: what the planner knows about your data

The `ANALYZE` command samples each table and stores per-column statistics, visible in the `pg_stats` view. The real values for two columns of the demo `orders` table:

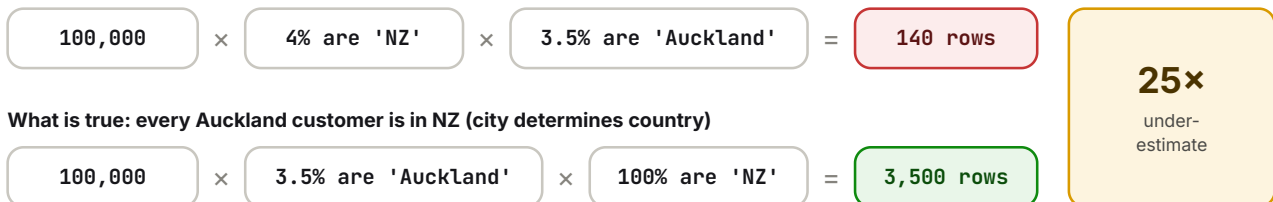
Statistic	orders.status	orders.created_at	Meaning
<code>null_frac</code>	0	0	fraction of NULLs
<code>n_distinct</code>	5	-1	number of distinct values; negative means "this fraction of the rows", so -1 means all unique
<code>most_common_vals</code>	delivered, cancelled, returned, pending, shipped	—	the most frequent values...
<code>most_common_freqs</code>	0.927, 0.050, 0.019, 0.0027, 0.0017	—	...and how often each occurs
<code>histogram_bounds</code>	—	101 bounds	equal-population buckets for the other values, used for ranges
<code>correlation</code>	0.870	1.000	how closely physical row order follows the column's sort order (1.0 is ideal for BRIN)

From these, the planner estimates that `status = 'pending'` matches 0.27% of the rows, and uses the histogram to estimate ranges on `created_at`. qopt passes these same statistics to Gemini for the columns a query touches, because they explain the planner's choices.

4.6 When estimates go wrong

Statistics are per column. When a query filters on two columns, the planner by default assumes they are **independent** and multiplies the fractions. Usually that is fine; when the columns are correlated, it is badly wrong. In the demo, every customer in Auckland is in New Zealand:

What the planner assumes: the two filters are independent



`CREATE STATISTICS ... (dependencies, mcv) ON country, city FROM customers`
 teaches the planner this dependency, so the estimate moves close to the truth.

Figure 10. Multiplying “4% NZ” by “3.5% Auckland” predicts 140 of 100,000 customers; the truth is 3,500. In W11’s real plan the estimate is 79 rows per worker against 1,144 actual.

qopt measures how wrong an estimate is with the **q-error**: the larger of *estimated* ÷ *actual* and *actual* ÷ *estimated*. 1 means perfect, and 10 or more is flagged. Wrong estimates matter because they cascade: the planner may pick a nested loop expecting 79 rows and then execute it 1,144 times. **Extended statistics** (`CREATE STATISTICS ... (dependencies, mcv) ON country, city`) teach the planner about correlated columns. qopt validated exactly this fix for W11: the estimate error dropped from 14x to 1.8x.

4.7 Self time, loops and parallel workers

`Actual Total Time` for a node *includes* its children, and it is an average per *loop* (a node inside a nested loop may run thousands of times). To find where time really goes, qopt computes each node’s **self time**: total time × loops, minus the same for its children. In parallel plans a node runs once in each worker process, so self times are summed CPU time and can add up to more than the wall-clock time.

4.8 Queries with parameters

Applications usually send `WHERE customer_id = $1` and pass the value separately. `pg_stat_statements` stores this normalized form. Without a concrete value, `EXPLAIN ANALYZE` cannot run the query, but since PostgreSQL 16, `EXPLAIN (GENERIC_PLAN)` can still plan it with average estimates. qopt uses that as a fallback: validation then compares estimated costs only, and says so.

LLMs, embeddings and RAG: just enough

qopt uses a large language model to propose fixes, and a small search engine over the PostgreSQL manual to ground those proposals. This chapter explains the handful of ideas involved, and why every model answer is treated as a hypothesis.

5.1 What a large language model is

A **large language model** (LLM) such as Google’s Gemini is a neural network trained on huge amounts of text to predict the next piece of text. You give it a **prompt** (instructions plus data) and it continues. Text is processed in **tokens**, pieces of words; the W13 analysis used about 3,000 input and 530 output tokens across two calls. **Temperature** controls randomness: qopt uses 0.2, low but not zero, so answers are focused yet can differ between runs.

Because the model has read a great deal about PostgreSQL, its advice is often right. But it has no way to *check* itself: it has never seen your data, cannot run your query, and can be confidently wrong.

5.2 Why every answer must be checked

Real Gemini suggestion in this project	What was wrong	Caught by
Index on <code>date(created_at)</code> (W10)	PostgreSQL rejects it: <code>date()</code> of a <code>timestamptz</code> is not IMMUTABLE	HypoPG / PostgreSQL error
Rewrite with <code>'2026-03-30 00:00:00+00'</code> (W10)	Hard-coded UTC; the original uses the session time zone. 1 row missing, 1 extra	result comparison on the shadow
Index on <code>created_at</code> for the same query	Useless while the filter still wraps the column in <code>date()</code> : the planner ignored it	HypoPG
<code>SET work_mem = '64MB'</code> (W13)	Measured 4.5% slower	timing on the shadow
<code>CREATE EXTENSION pg_trgm; CREATE INDEX ...</code> (W07)	Two statements where one index was expected	parser safety check

5.3 Structured output

Free-form text is hard for a program to use. qopt asks Gemini to answer in **JSON** that follows a **JSON Schema**: a diagnosis, plus 1–4 candidates, each with a kind, SQL statements, a rationale and citations. The schema is written once with the `zod` library, sent to Gemini as `responseJsonSchema`, and used again to re-check the answer. “Matches the schema” still does not mean “valid SQL”, which is why the answer then goes through PostgreSQL’s parser and, eventually, PostgreSQL itself.

5.4 Embeddings and vector search

An **embedding** turns a piece of text into a list of numbers (a *vector*), such that texts with similar meaning get similar vectors. qopt embeds each chunk of the PostgreSQL manual with Gemini’s `gemini-embedding-001` model into 768 numbers. To search, it embeds the question the same way and looks for the chunks whose vectors are closest, measured by **cosine distance**: the angle between two vectors.

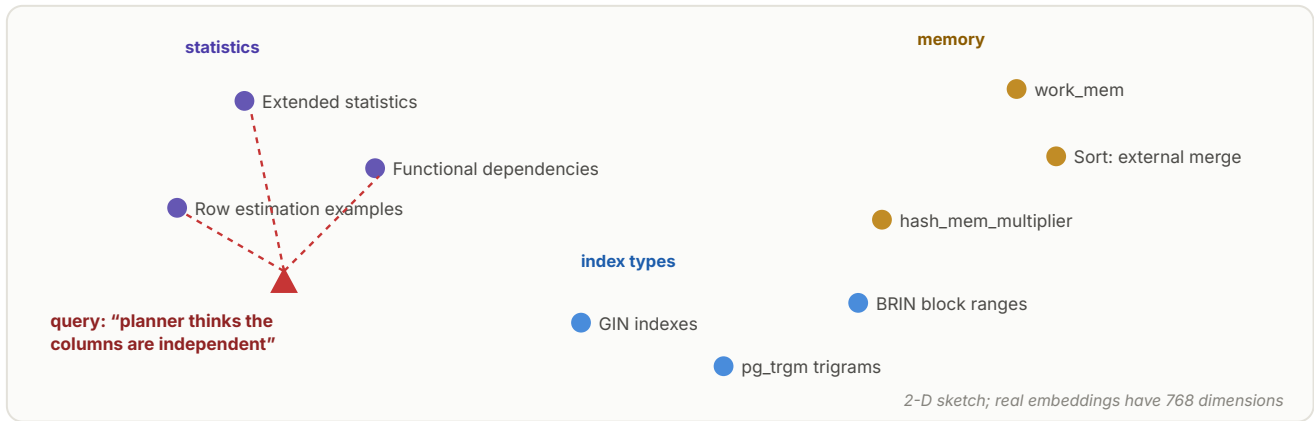


Figure 11. A 2-D sketch of the idea. A question about “columns treated as independent” lands near the chunks on extended statistics, although it shares few words with them.

The vectors are stored in the meta database using the **pgvector** extension, which adds a `vector(768)` column type, the `<=>` cosine-distance operator, and an **HNSW** index for fast nearest-neighbour search.

5.5 Keyword search and hybrid ranking

Vector search is good at meaning; classic **keyword search** is good at exact terms like `work_mem`. PostgreSQL has keyword search built in: text is converted to a `tsvector` (normalized words) and matched against a `tsquery`. `qopt` runs both searches and merges the two ranked lists with **reciprocal rank fusion** (RRF): each chunk scores $1 / (60 + \text{rank})$ in every list it appears in, and the scores are added. An example with invented ranks:

Chunk	Keyword rank	Vector rank	RRF score
“Functional dependencies”	1	2	$1/61 + 1/62 = 0.0325$
“Extended statistics”	3	1	$1/63 + 1/61 = 0.0323$
“CREATE STATISTICS: examples”	2	—	$1/62 = 0.0161$

A chunk near the top of both lists wins, and a chunk found by only one method still has a chance. RRF needs no tuning, which is why it is a common default.

5.6 Retrieval-augmented generation (RAG)

RAG means retrieving relevant documents and putting them into the prompt, so the model answers with the right facts in front of it. `qopt` retrieves six excerpts of the manual for the PostgreSQL version being tuned, labels them D1...D6, and asks Gemini to cite the ones it used. The citations appear in the web UI as links to [postgresql.org](https://www.postgresql.org). [Chapter 12](#) explains how the documents are prepared, and [Chapter 30](#) discusses what RAG did, and did not, change.

The tools that make verification possible

qopt's value depends on four pieces of PostgreSQL machinery. Two of them observe the workload; two provide ground truth.

6.1 pg_stat_statements: what is slow, in aggregate

This extension (it ships with PostgreSQL) keeps running totals for every distinct statement: how often it ran (`calls`), total and mean execution time, rows returned, and page reads. Statements that differ only in their constant values are grouped: `WHERE customer_id = 42` and `= 43` both become `WHERE customer_id = $1`, identified by a 64-bit hash called the `queryid`. Sorting by total time answers "what should I fix first?": a 5 ms query called a million times matters more than one 2-second report.

6.2 auto_explain: the real plans

`pg_stat_statements` does not keep plans or parameter values. `auto_explain` (also bundled) fills the gap: when a statement runs longer than a threshold, it writes the *actual* execution plan to the server log, with the full query text including the real values. qopt configures it to log in JSON, with the same `query_id`, so the two sources can be joined. The threshold is 5 ms for the demo; on a production system you would use 100 ms or more.

6.3 HypoPG: hypothetical indexes

Building a real index on a large table takes time, locks the table against writes (unless built `CONCURRENTLY`) and uses disk. **HypoPG** is a third-party extension that creates *hypothetical* indexes: entries that exist only in the current session's planner. Plain `EXPLAIN` then shows whether the planner *would* use the index, and how the estimated cost changes, in milliseconds and without touching disk.

Session A: the qopt validator

```
SELECT * FROM hypopg_create_index('CREATE INDEX ON orders (customer_id)');
EXPLAIN SELECT ... WHERE customer_id = 42;
→ Index Scan using <13543>btree_orders_customer_id cost 10934 → 8
SELECT hypopg_reset();
```

Session B: your application, at the same moment

```
EXPLAIN SELECT ... WHERE customer_id = 42; → Seq Scan (sees nothing)
```

A hypothetical index...

- exists only in session A's planner memory
- no disk writes, no locks, no build time
- affects plain EXPLAIN only (estimates, not timings)
- btree, brin, hash, bloom; not GIN or GiST

Figure 12. A hypothetical index is invisible to every other session and disappears with `hypopg_reset()`, so it is safe to use on the production database itself.

HypoPG's answer is an *estimate*: it cannot tell you the real timing, and it supports B-tree, BRIN, hash and bloom indexes but not GIN or GiST. That is why qopt uses it as a fast first filter, not as the final word.

6.4 The shadow database

The **shadow** is a second PostgreSQL server holding a copy of the target's schema and data. It is qopt's laboratory. On the shadow, qopt can build indexes for real, run `EXPLAIN ANALYZE`, compare the results of two queries, or try a setting, all inside transactions that are always rolled back. In the demo, target and shadow are filled by the same

deterministic script, and a checksum confirms they match. For a real database you would restore a backup, or use `qopt shadow sync ( pg_dump | psql )` for small databases.

6.5 How the four fit together

Tool	Question it answers	Cost	Limitation
pg_stat_statements	Which statements use the most time?	tiny, always on	no plans, no parameter values
auto_explain	What did a slow run actually do, with which values?	small (timing overhead)	only runs over the threshold
HypoPG	Would the planner use this index, and would it look cheaper?	milliseconds, no writes	estimates only; B-tree/BRIN/hash/bloom
Shadow database	Is it really faster? Are the results identical?	real execution time	needs a realistic copy of the data

PART II

Architecture

How the pieces fit together: the components and databases, the path a single analysis takes, and the four subsystems that do the real work: capture, validation, the LLM loop and documentation retrieval. It ends with the safety model that keeps all of it harmless.

- Chapter 7 · The big picture
- Chapter 8 · The analysis pipeline, stage by stage
- Chapter 9 · Capturing slow queries
- Chapter 10 · The validation engine
- Chapter 11 · The LLM loop
- Chapter 12 · Documentation retrieval (RAG)
- Chapter 13 · The safety model

The big picture

qopt is a TypeScript engine with three front ends, three PostgreSQL servers, and two outside services. All the logic lives in one package; the front ends are thin.

7.1 Components

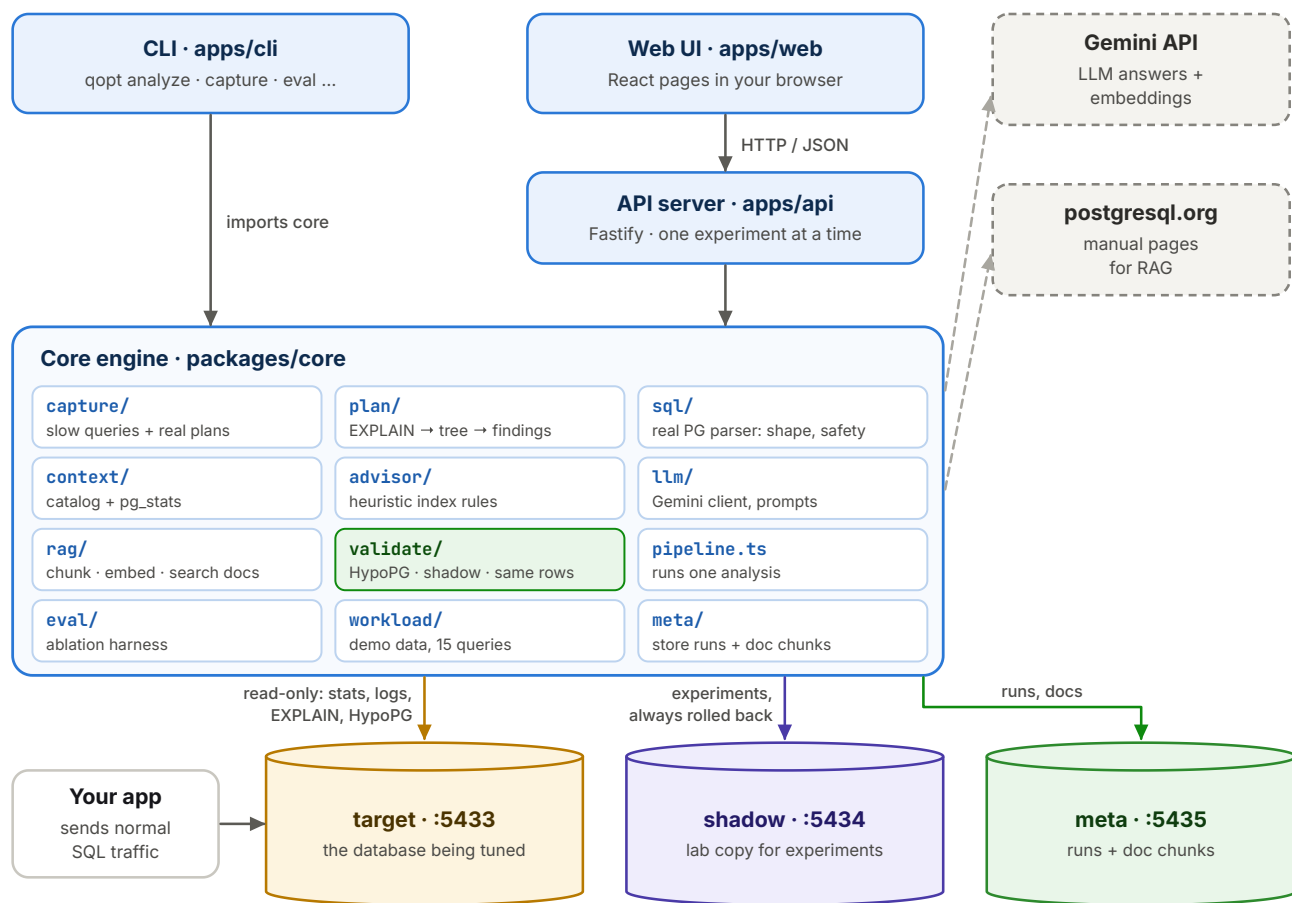


Figure 13. System architecture. The core engine reads from the target (never writes), runs experiments on the shadow (always rolled back), and stores its own data in meta. The web UI reaches the engine through the API; the CLI imports it directly.

Component	Package	Role
Core engine	packages/core	Everything that matters: capture, plan analysis, SQL parsing, schema introspection, the heuristic advisor, the Gemini client, retrieval, validation, the pipeline, the demo workload and the evaluation harness.
CLI	apps/cli	The <code>qopt</code> command. Parses arguments and prints results; each command calls one core function.
API server	apps/api	A small HTTP server (Fastify) for the web UI. Starts analyses in the background, serves progress snapshots, and runs one experiment at a time.
Web UI	apps/web	A React single-page app: captured queries, analysis runs with plans and verdicts, and evaluation results.
Gemini API	external	Proposes fixes (<code>gemini-3.8-flash</code> and fallbacks) and embeds text (<code>gemini-embedding-001</code>).

Component	Package	Role
postgresql.org	external	Source of the manual pages used for retrieval (fetched once by <code>qopt docs ingest</code>).

7.2 The three databases

	target	shadow	meta
Port (local)	5433	5434	5435
Purpose	the database being tuned (your production database in real use)	a lab copy of the target, for experiments	qopt's own storage
Extensions	pg_stat_statements, auto_explain, hypopg, pg_trgm	hypopg, pg_trgm	vector (pgvector)
How qopt uses it	read-only sessions: statistics, logs, catalog, plain EXPLAIN, session-only HypoPG	builds indexes, runs EXPLAIN ANALYZE, compares results, always in rolled-back transactions	reads and writes <code>runs</code> and <code>doc_chunks</code>
Contents in the demo	the six shop tables, seeded identically (checked with a checksum)		459 manual chunks with embeddings; every analysis run

BEGINNER NOTE

Why not experiment on the target directly? Because building an index on a busy production table takes locks and I/O, and timing a slow query adds load. The shadow absorbs all of that. HypoPG is the one exception: it writes nothing, so it runs on the target, where the statistics are the real ones.

7.3 One engine, three front ends

The repository is an **npm workspaces monorepo**: four packages in one repository, sharing one `node_modules` and one lockfile.

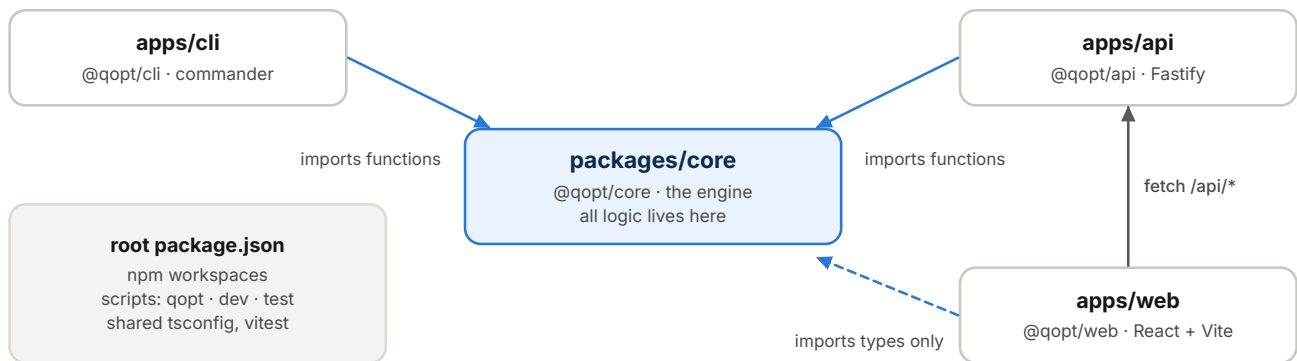


Figure 14. The CLI and the API import functions from `@qopt/core`. The web app imports only its TypeScript types (so it never bundles database code) and talks to the API over HTTP.

7.4 Where everything lives

Data	Location
Per-statement counters	<code>pg_stat_statements</code> on the target (in shared memory, saved at shutdown)
Captured plans	the target's log directory, e.g. <code>.pgdata/target/log/postgresql-2026-09-29.json</code>

Data	Location
Analysis runs	meta database, table <code>runs</code> (the whole result as JSON)
Documentation chunks and embeddings	meta database, table <code>doc_chunks</code>
Gemini answers and embeddings (cache)	<code>.cache/llm/*.json</code> and <code>.cache/embeddings/*.json</code>
Evaluation reports	<code>eval/results/<timestamp>.json</code> and <code>latest.json</code>
Settings and the API key	<code>.env</code> (not committed)
Local database files	<code>.pgdata/target</code> , <code>.pgdata/shadow</code> , <code>.pgdata/meta</code>

The analysis pipeline, stage by stage

One call, `analyze()`, takes a query from “slow” to “ranked, verified recommendations”. The CLI, the API and the evaluation harness all use this same function.

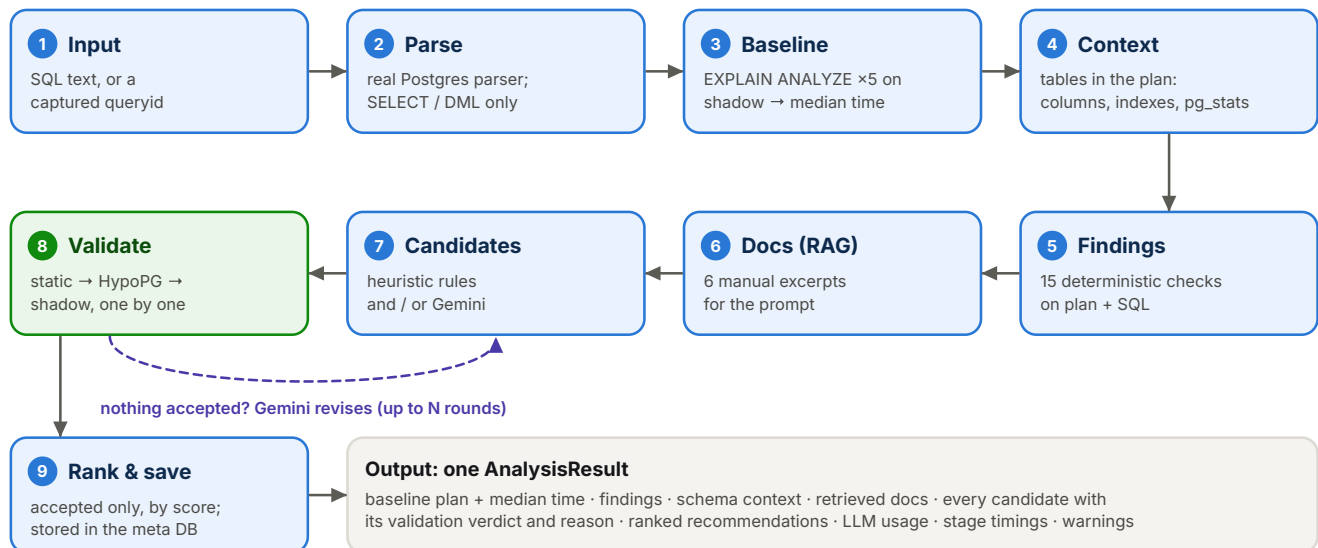


Figure 15. The nine stages of one analysis. Candidates are validated one at a time, because experiments on the shadow must not run in parallel or they would distort each other’s timings.

8.1 What each stage does

#	Stage	What happens	Code
1	Input	Either SQL text, or a <code>queryid</code> from <code>pg_stat_statements</code> . For a <code>queryid</code> , the captured <code>auto_explain</code> sample supplies the literal query and its real plan.	<code>capture/</code>
2	Parse	The real PostgreSQL parser checks the SQL. Only <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> and <code>DELETE</code> are analysed. <code>\$1</code> placeholders switch to the generic-plan path.	<code>sql/</code>
3	Baseline	Plain <code>EXPLAIN</code> on the target, then <code>EXPLAIN ANALYZE</code> on the shadow: one warm-up run and five timed runs. The median time and the plan of the median run are the baseline.	<code>plan/explain.ts</code>
4	Context	For each table in the plan or the SQL: columns, indexes (with size and use count), constraints, extended statistics, write activity, and <code>pg_stats</code> for the columns the query uses.	<code>context/schema.ts</code>
5	Findings	Fifteen deterministic checks on the plan and on the SQL itself.	<code>plan/findings.ts</code>
6	Docs	With RAG on: six manual excerpts, retrieved using the findings.	<code>rag/</code>
7	Candidates	The heuristic advisor’s proposals and/or Gemini’s. Duplicates are removed by comparing SQL <i>fingerprints</i> .	<code>advisor/</code> , <code>llm/</code>
8	Validate	Each candidate: static checks, then HypoPG, then the shadow. If no Gemini candidate was accepted, Gemini gets the verdicts and revises (default: one extra round).	<code>validate/</code>
9	Rank & save	Accepted candidates sorted by score; the whole result is stored in the meta database.	<code>pipeline.ts</code>

8.2 Which plan is “the baseline”?

The pipeline uses the best evidence available, in this order:

1. **shadow-analyze**: a real `EXPLAIN ANALYZE` on the shadow (median of 5). This is the normal case, and the only one that gives timings for validation to compare against.
2. **auto-explain**: the real plan captured on the target, if there is no shadow.
3. **target-explain** or **target-generic**: estimates only, when the query has `$1` placeholders and no captured values.

The source is shown next to the baseline in the CLI and the UI, so you always know whether a number was measured or estimated.

8.3 The findings catalogue

Findings are plain facts about the plan or the SQL, computed without any AI. They feed the heuristic advisor, choose which documentation to retrieve, and are given to Gemini as a structured list.

Code	Raised when...	Source
<code>SEQ_SCAN_SELECTIVE</code>	a sequential scan reads $\geq 10,000$ rows and keeps $\leq 10\%$ of them	plan
<code>SEQ_SCAN_IN_LOOP</code>	a sequential scan runs repeatedly on the inner side of a nested loop	plan
<code>ROW_MISESTIMATE</code>	q-error ≥ 10 on ≥ 100 rows, reported only at the node where it starts	plan
<code>SORT_SPILL</code> / <code>HASH_SPILL</code>	a sort wrote to disk / a hash split into batches	plan
<code>LOSSY_BITMAP</code>	a bitmap ran out of memory and became page-level	plan
<code>HEAP_FETCHES</code>	an index-only scan still visited the table often	plan
<code>TEMP_IO</code> , <code>JIT_OVERHEAD</code>	temporary files were written; JIT compilation took $> 20\%$ of the time	plan
<code>HOT_NODE</code>	one node takes $\geq 50\%$ of the execution time	plan
<code>NOT_IN_SUBQUERY</code>	<code>NOT IN (SELECT ...)</code>	SQL
<code>NON_SARGABLE_PREDICATE</code>	a filter wraps a column in a function	SQL
<code>LEADING_WILDCARD</code>	<code>LIKE</code> / <code>ILIKE</code> pattern starting with <code>%</code>	SQL
<code>LARGE_OFFSET</code>	<code>OFFSET</code> $\geq 1,000$	SQL
<code>OR_ACROSS_COLUMNS</code>	<code>OR</code> between filters on different columns	SQL

8.4 The result object

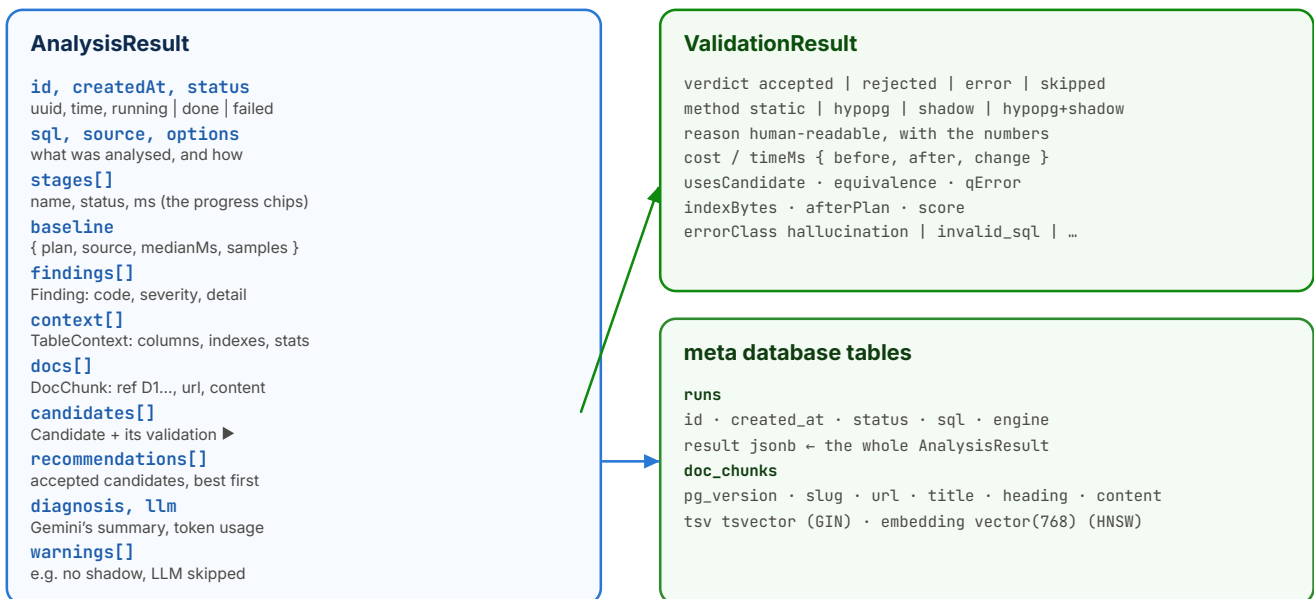


Figure 16. Everything an analysis produces is one `AnalysisResult`, saved whole as JSON in the meta database. Each candidate carries its `ValidationResult`: the verdict, how it was reached, and the measured numbers.

While an analysis runs, the pipeline calls an `onUpdate` callback after every stage and every validated candidate. The API keeps the latest snapshot in memory, which is how the web UI shows progress chips filling in live.

Capturing slow queries

Capture answers two questions: *which statements* cost the most, and *what exactly happened* when they ran. The first comes from `pg_stat_statements`, the second from `auto_explain`'s log, joined on the query id.

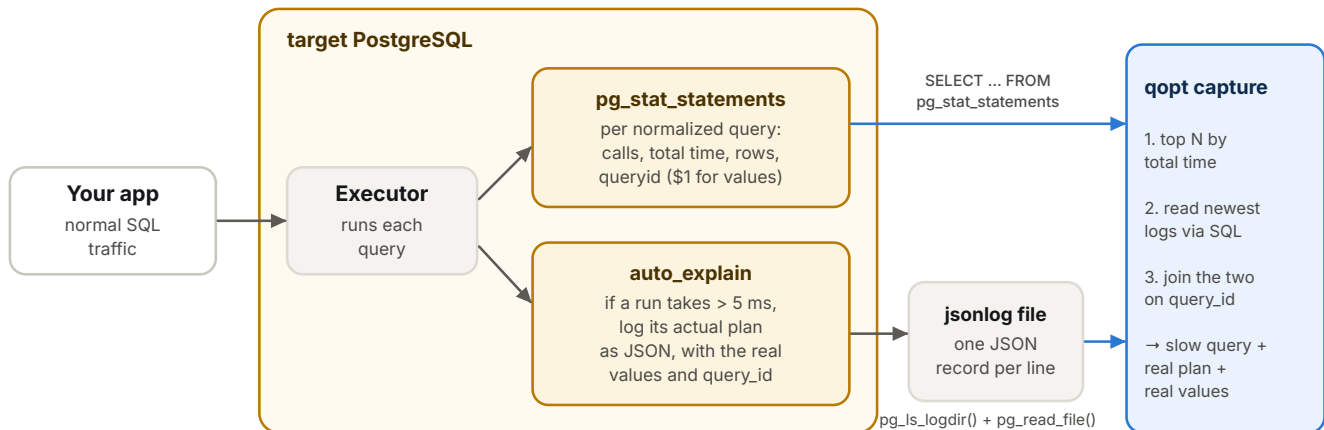


Figure 17. The application's queries are counted by `pg_stat_statements` and, when slow, logged with their real plan by `auto_explain`. `qopt` reads both through ordinary SQL and joins them.

9.1 Finding the most expensive statements

`qopt` asks `pg_stat_statements` for the top statements by **total** execution time in the current database. It filters out noise:

- its own statements: every query `qopt` sends to the target starts with the comment `/*qopt*/`, and those are excluded;
- catalog and monitoring queries (anything touching `pg_catalog`, `pg_stat_*`, `hypopg` ...);
- anything that is not a top-level `SELECT`, `WITH`, `INSERT`, `UPDATE` or `DELETE`.

9.2 Reading plans from the server log

On the target, `auto_explain` writes each slow execution as one JSON line in the server's log file (`log_destination = jsonlog`). Rather than reading files from disk, which would not work for a remote or Docker server, `qopt` reads them through SQL:

- `pg_ls_logdir()` lists the newest three `.json` log files;
- `pg_read_file(path, offset, length)` reads at most the last 32 MB of them;
- each line is parsed. Lines from `qopt`'s own sessions (application name `qopt-...`) are skipped, and the `message` field (`"duration: 10.227 ms plan: {...}"`) yields the duration, the literal query text and the plan.

For each query id, the **slowest** sample is kept: it is the most representative of the problem.

9.3 The 64-bit query id trap

The two sources are joined on the query id, a signed 64-bit integer such as `669985620531413121`. JavaScript numbers are 64-bit floating point and represent integers exactly only up to 2^{53} (about 9×10^{15}). `JSON.parse` would silently turn that id into a different number (it prints as `669985620531413100`), and the join would fail. `qopt`

therefore:

- selects the id as text from PostgreSQL (`queryid::text`), and
- extracts it from each log line with a regular expression *before* parsing the JSON.

[Chapter 19](#) shows the code, and a unit test checks the exact 18-digit id.

9.4 When there is no sample

A statement can be expensive in total and yet never cross the `auto_explain` threshold in a single run. Then `qopt` has only the normalized text with `$1` placeholders. It analyses it anyway with `EXPLAIN (GENERIC_PLAN)`, and validation compares estimated costs only: HypoPG on the target for B-tree/BRIN/hash, and a real build on the shadow with a generic `EXPLAIN` for GIN. The run carries a warning that no timings were possible.

The validation engine

Validation is the heart of qopt. It turns “the model says this index helps” into “PostgreSQL used this index, and the query went from 704 ms to 149 ms”, or into a precise reason why not.

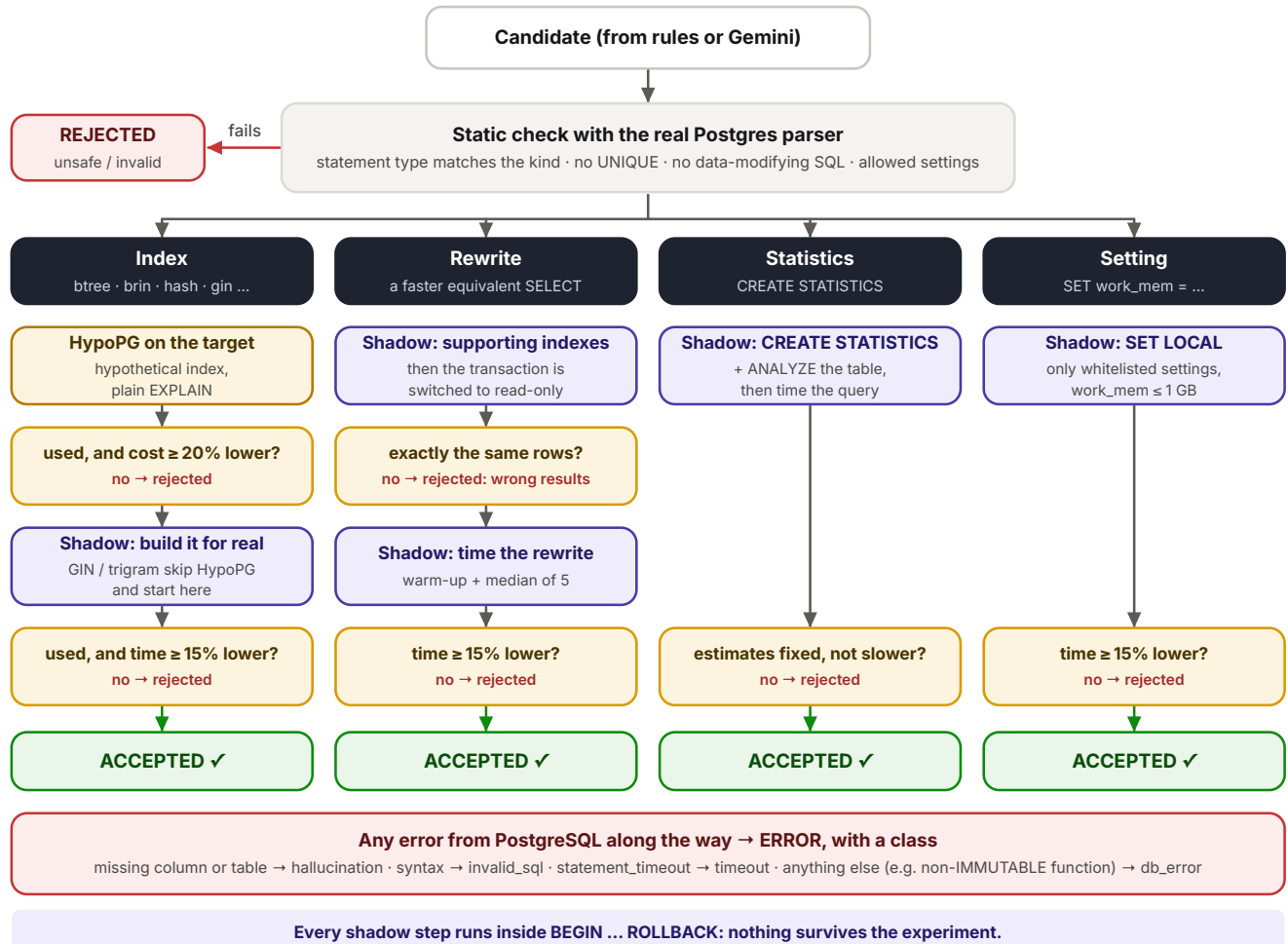


Figure 18. The validation flow for each kind of candidate. A candidate must pass every gate on its path; any PostgreSQL error along the way ends it with a classified ERROR.

10.1 Gate 1: static checks with the real parser

Before any database sees a candidate, its SQL is parsed with `libpg-query`, PostgreSQL’s own parser compiled to WebAssembly. The parser reports what each statement *is*, independent of what the model *claims* it is.

Kind	Must be	Also rejected
index	only <code>CREATE INDEX</code> statements	<code>UNIQUE</code> indexes (they add a constraint and can make writes fail)
statistics	one <code>CREATE STATISTICS</code> on one table	anything else
config	one <code>SET name = value</code>	settings outside the allow-list; <code>work_mem</code> above 1 GB
rewrite	one <code>SELECT</code> (+ optional supporting <code>CREATE INDEX</code>)	data-modifying CTEs, <code>SELECT INTO</code> , <code>FOR UPDATE</code> , functions like <code>pg_sleep</code> , <code>pg_terminate_backend</code> , <code>dblink</code> , <code>set_config</code> , <code>nextval</code>

The allowed settings are `work_mem`, `hash_mem_multiplier`, `random_page_cost`, `seq_page_cost`, `effective_cache_size`, `effective_io_concurrency`, `jit`, `jit_above_cost` and `max_parallel_workers_per_gather`. A statement that does not parse is classed as `invalid_sql`; the wrong type of statement as `unsafe`. The parser also lets qopt remove `CONCURRENTLY` for experiments (it is not allowed inside a transaction) and add it back when displaying recommendations.

10.2 Gate 2: HypoPG on the target

For B-tree, BRIN and hash indexes, qopt opens a session on the target and, in that one session:

1. clears old hypothetical indexes (`hypopg_reset()`) and runs plain `EXPLAIN` for the baseline cost;
2. creates each candidate index with `hypopg_create_index('CREATE INDEX ...')`, which returns a name like `<13543>btree_orders_customer_id`;
3. runs `EXPLAIN` again and looks for that name in the plan;
4. records the estimated size (`hypopg_relation_size`) and resets.

The candidate passes only if the planner **uses** the index **and** the estimated cost drops by at least **20%** (`QOPT_MIN_COST_GAIN`). This gate is cheap and runs where the statistics are real. It filters most bad index ideas in milliseconds.

10.3 Gate 3: a real experiment on the shadow

One validation experiment on the shadow database

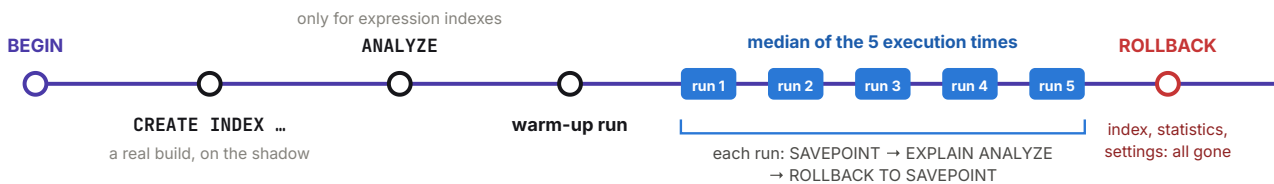


Figure 19. Every experiment is one transaction on the shadow that ends in ROLLBACK. Each timed run is wrapped in a savepoint, so even an UPDATE sees the same data every time.

The candidate's index is built for real inside a transaction. qopt finds the new index by comparing the table's index list before and after, and records its real size. Expression indexes are followed by `ANALYZE`, because PostgreSQL only gathers statistics on the indexed expression then. The query is then measured exactly like the baseline: one warm-up run to load pages into memory, then five timed `EXPLAIN ANALYZE` runs. The **median** execution time is compared with the baseline median, and the candidate passes if the planner used the new index and the time dropped by at least **15%** (`QOPT_MIN_TIME_GAIN`).

BEGINNER NOTE

Why the median, and why a warm-up? The first run of a query may read pages from disk that later runs find in memory. And any single run can be slowed by something else happening on the machine. The median of five is robust to one or two outliers.

GIN and trigram indexes skip Gate 2, because HypoPG cannot simulate them, and go straight to the shadow.

10.4 Thresholds and ranking

An accepted candidate gets a **score**: its percentage improvement (measured time, or estimated cost if no timing was possible), minus penalties for index size relative to the table (up to 20 points) and for tables with many more writes than reads (5 points). Recommendations are sorted by score. That is why, for W04, a partial index measured at -99.5% ranks above a composite index at -99.4%: both are dramatic, but the partial index is much smaller.

Real CLI output for W04 (rule-based advisor only)

```
Candidates (3)
ACCEPTED [heuristic] index      B-tree on orders (status, created_at) -99.4%
CREATE INDEX CONCURRENTLY qopt_orders_status_created_at_idx ON public.orders (status, created_at)
hypopg+shadow: HypoPG cost -99.8%; shadow: measured 16.40 → 0.10 ms (-99.4%, median of 5)
ACCEPTED [heuristic] index      B-tree on orders (created_at) WHERE status = 'pending' -99.5%
CREATE INDEX CONCURRENTLY qopt_orders_created_at_partial_idx ON public.orders (created_at) WHERE status = 'pending'
hypopg+shadow: HypoPG cost -99.9%; shadow: measured 16.40 → 0.08 ms (-99.5%, median of 5)
ACCEPTED [heuristic] index      BRIN on orders (created_at) -97.8%
CREATE INDEX CONCURRENTLY qopt_orders_created_at_brin_idx ON public.orders USING brin (created_at)
hypopg+shadow: HypoPG cost -41.5%; shadow: measured 16.40 → 0.35 ms (-97.8%, median of 5)
Recommendations (3)
1. B-tree on orders (created_at) WHERE status = 'pending' -99.5%
2. BRIN on orders (created_at) -97.8%
3. B-tree on orders (status, created_at) -99.4%
```

10.5 Statistics and settings

- **Extended statistics:** on the shadow, `CREATE STATISTICS` + `ANALYZE`, then five timed runs. Accepted if the worst row-estimate error on that table at least **halves** (or the query gets 15% faster), unless it made the query more than 25% slower. For W11 the error went from 14× to 1.8×, although the time barely moved: better estimates are valuable even when this particular plan does not change.
- **Settings:** `SET LOCAL` (which lasts only until the end of the transaction), then timing. Accepted only on a 15% gain; the result also records whether a disk spill disappeared.

10.6 Rewrites: same rows, then faster

A rewritten query is useless if it returns different results. On the shadow, qopt first builds any supporting indexes, then **switches the transaction to read-only**, so that nothing the model wrote can modify data, and compares the two queries:

Runs on the shadow in a read-only transaction. Testing on data catches real mistakes, but it is not a proof.

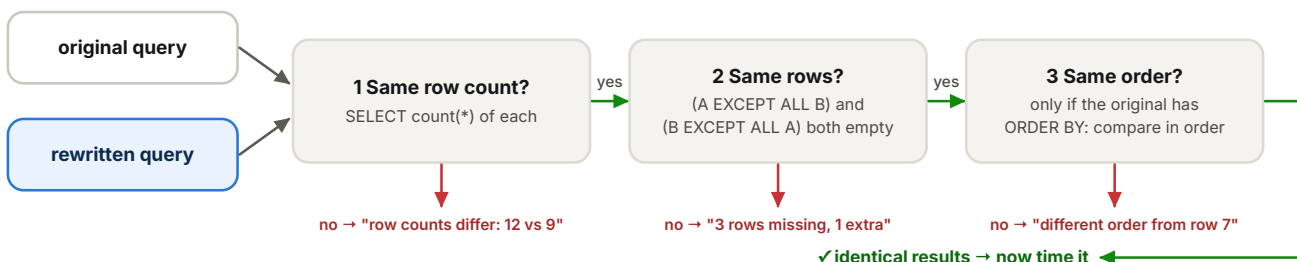


Figure 20. Three checks in order: row counts, then the rows themselves as multisets (`EXCEPT ALL` in both directions), then the order if the original has `ORDER BY`.

`A EXCEPT ALL B` returns the rows of A that are not in B, respecting duplicates; if both directions are empty, the multisets are equal. When a column type has no equality operator (plain `json`), qopt fetches both results (up to 50,000 rows) and compares them in memory. Only an equivalent rewrite is timed, and it must also be 15% faster.

WATCH OUT

Comparing results on data is strong evidence, not a proof. Two queries can agree on today's data and differ tomorrow; for example, `NOT IN` and `NOT EXISTS` differ only once a `NULL` appears. The shadow's data is what gives this check its power, so it should resemble production.

10.7 When PostgreSQL says no: error classes

Error	Class	Example from this project
column / table / function / operator class "does not exist" (SQLSTATE 42703, 42P01, 42883, 42704)	hallucination	an index on <code>orders (customer_uuid)</code> in the integration tests
syntax error (42601)	invalid_sql	—
statement timeout (57014)	timeout	—
anything else	db_error	"functions in index expression must be marked IMMUTABLE" (W10)

HypoPG reports some of these with a generic internal error code, so qopt also matches the message text ("... does not exist"). The evaluation counts hallucinations and invalid SQL separately, as a measure of model quality.

The LLM loop

Gemini is used as a proposal engine inside a feedback loop: it sees what the deterministic analysis found, proposes candidates in a strict JSON format, and when all of them fail, it gets the verdicts and tries again.

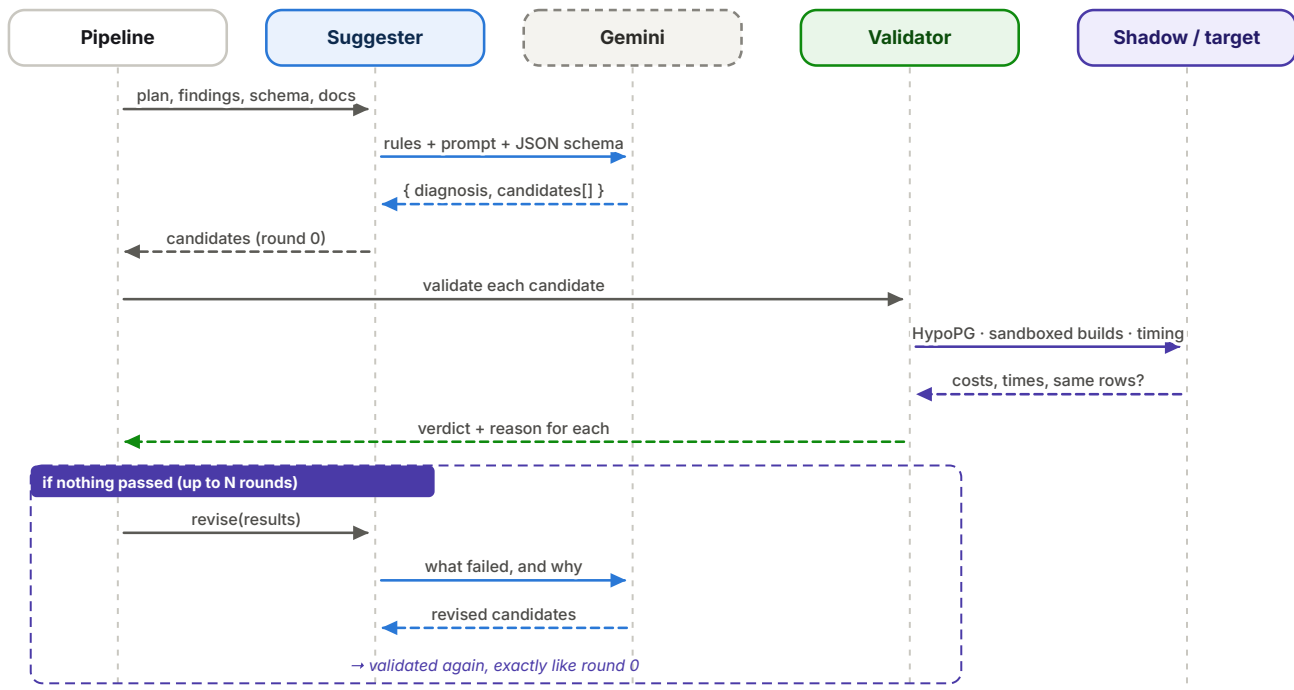


Figure 21. One conversation per analysis. Round 0 proposes; if nothing passed validation, the verdicts go back to Gemini and it revises (by default, once).

11.1 What Gemini is told

The **system instruction** sets the rules. Here is the real text as generated for the demo (PostgreSQL 17.11, 15% time threshold):

System instruction (generated by llm/prompt.ts)

You are a senior PostgreSQL performance engineer. You propose changes that make ONE query faster.

Every candidate you propose is machine-verified before a human sees it:

- index: simulated with HypoPG (btree, brin, hash) and then built for real on a shadow copy of the database inside a rolled-back transaction. Kept only if the planner actually uses it AND measured execution time drops by at least 15%.
 - rewrite: executed on the shadow copy. Kept only if it returns exactly the same rows (same multiset, and the same order when the original has ORDER BY) AND is at least 15% faster.
 - statistics: CREATE STATISTICS + ANALYZE on the shadow. Kept if it fixes the row misestimate or reduces time.
 - config: SET for one of work_mem, hash_mem_multiplier, random_page_cost, seq_page_cost, effective_cache_size, effective_io_concurrency, jit, jit_above_cost, max_parallel_workers_per_gather, applied with SET LOCAL on the shadow. Kept if measured time drops by 15%.
- Rejected candidates are thrown away, so propose only what you expect to pass.

Rules:

1. Use only tables and columns present in the schema section. Never invent names.
2. kind=index: statements holds complete "CREATE INDEX CONCURRENTLY qopt_<name> ON <table> ..." statements (never UNIQUE; several statements only if they must work together, e.g. one per OR branch). Do not duplicate an existing index. Prefer the smallest index that works: partial (WHERE), covering (INCLUDE), expression, or the access method the operator needs (GIN for @> on jsonb/arrays, GIN with gin_trgm_ops for LIKE '%x%', BRIN for large naturally ordered columns).
3. Functions in index expressions must be IMMUTABLE. For example date(timestamptz) and to_char are not; rewrite the predicate as a range instead.
4. kind=rewrite: rewritten_sql is the complete rewritten query with identical semantics, including NULL handling, duplicates and ORDER BY/LIMIT. statements may hold CREATE INDEX statements the rewrite relies on.
5. kind=statistics: statements holds one "CREATE STATISTICS qopt_<name> (dependencies, mcv) ON col1, col2 FROM table".
6. kind=config: statements holds one "SET name = 'value'" meant for this query's session only. Keep work_mem at or below 1GB.
7. Cite the documentation excerpts you relied on by id (e.g. "D2"); use [] if none apply.
8. Return 1 to 4 candidates, best first, and a one-paragraph root-cause diagnosis.

Target server: PostgreSQL 17.11 (Homebrew).

The **user message** carries the evidence: the query, the findings, a compact text rendering of the plan, the schema and statistics of the tables involved, and, with RAG, the documentation excerpts. This is the real message for W13:

```

## Query
```sql
SELECT product_id, count(DISTINCT order_id) AS orders FROM order_items GROUP BY product_id ORDER BY orders DESC
LIMIT 10
```

## Findings (deterministic analysis of the plan)
- [SORT_SPILL/high] node 4 Sort spilled 45808 kB to disk. Sort Method "external merge" means the sort did not fit in work_mem and used temporary files. (method=external merge, spaceKb=45808)
- [HOT_NODE/low] node 4 73% of time in Sort. This node dominates execution time; improvements elsewhere will not matter much. (selfMs=516.62, totalMs=703.86)

## Execution plan (EXPLAIN ANALYZE on the shadow copy)
[1] Limit cost=295,823 est_rows=10 actual_rows=10 loops=1 self=0ms total=691.36ms
[2] Sort cost=295,884 est_rows=24,562 actual_rows=10 loops=1 self=1.28ms total=691.36ms misestimate=2,456x
sort_key=(count(DISTINCT order_id)) DESC sort=top-N heapsort (Memory 25kB)
[3] Aggregate cost=295,292 est_rows=24,562 actual_rows=30,000 loops=1 self=90.56ms total=690.08ms
[4] Sort cost=286,046 est_rows=1,800,000 actual_rows=1,800,000 loops=1 self=516.62ms total=599.51ms
sort_key=product_id, order_id sort=external merge (Disk 45808kB)
[5] Seq Scan on order_items cost=33,000 est_rows=1,800,000 actual_rows=1,800,000 loops=1 self=82.90ms
total=82.90ms
planning=0.05ms execution=703.86ms

## Schema and statistics
TABLE public.order_items: ~1,800,000 rows, 117.2 MB heap, 155.8 MB total
columns: id bigint not null, order_id bigint not null, product_id bigint not null, quantity integer not null,
unit_price numeric(10,2) not null
indexes: UNIQUE INDEX order_items_pkey ON public.order_items USING btree (id) [38.6 MB, 0 scans]
constraints: order_items_order_fk: FOREIGN KEY (order_id) REFERENCES orders(id); order_items_pkey: PRIMARY
KEY (id); order_items_product_fk: FOREIGN KEY (product_id) REFERENCES products(id)
activity: 95 seq scans, 0 index scans, 1800000 inserts, 0 updates, 0 deletes since stats reset

```

11.2 What Gemini must answer

| Field | Content |
|--|---|
| diagnosis | one paragraph on the root cause (shown in the UI) |
| candidates[] | 1 to 4 items, best first, each with: |
| kind | index · rewrite · statistics · config |
| title, rationale, expected_plan_change | human-readable explanation |
| statements | the SQL (DDL, SET, or supporting indexes for a rewrite) |
| rewritten_sql | the full rewritten query (empty for other kinds) |
| citations | documentation labels such as ["D2"] |

11.3 Feedback and revision

If none of Gemini's candidates passed, qopt adds the verdicts to the same conversation and asks for new or revised candidates. This is the real feedback message for W13, after the `work_mem` suggestion failed:

Feedback message for W13 (generated by llm/prompt.ts)

Validation results for your candidates:

1. (config) Increase work_mem for the session

SQL: SET work_mem = '64MB'

Result: REJECTED: shadow with work_mem=64MB: measured 703.86 → 735.19 ms (+4.5%), below the 15% threshold

Propose up to 3 new or revised candidates that address these results. Do not repeat a rejected statement unchanged. If a candidate was rejected because the planner did not use an index, reconsider column order, selectivity, or whether a different kind of change is needed.

Gemini's second answer was the `(product_id, order_id)` index, which passed at -79%. Revision rounds are limited by `retries` (default 1). They are skipped when a round already produced an accepted candidate, or when validation is switched off.

11.4 Surviving overload and quotas

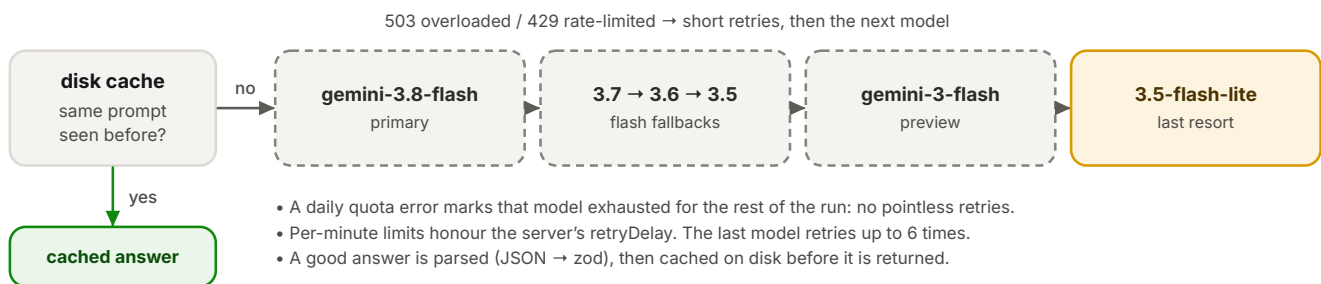


Figure 22. The Gemini client tries models in order. Overloaded (503) or rate-limited (429) models get a couple of quick retries before the next model is tried; a spent daily quota skips that model for the rest of the run.

This machinery exists because of real problems during development. The primary model was overloaded for long stretches, and the free tier's **per-model daily request cap** ran out partway through an evaluation. The client now records which model actually answered each call. The evaluation can pin a single model with `--model`, so that configurations are compared on equal terms.

Every successful answer is cached on disk, keyed by a hash of the model, the instructions, the conversation and the schema. The same prompt never costs a second call. Answers are parsed and validated *before* being cached, so a malformed answer is never replayed.

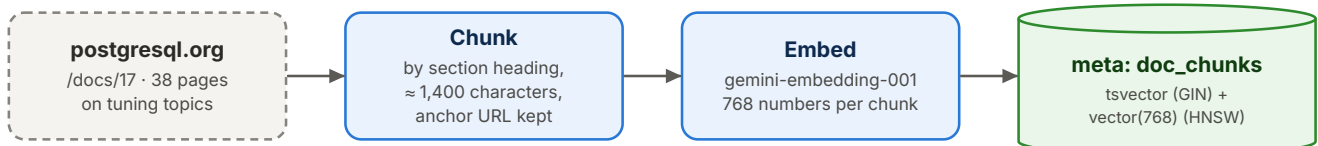
11.5 Case studies from real runs

| Query | Round 0 | Verdict | Round 1 | Verdict |
|------------------------------|---|---|-------------------------------------|--|
| W13 (704 ms) | SET work_mem = '64MB' | ✗ 704 → 735 ms (+4.5%) | B-tree
(product_id,
order_id) | ✓ 704 → 149 ms
(-79%) |
| W07 (94 ms) | CREATE EXTENSION pg_trgm; +
trigram GIN | ✗ unsafe: "expected a single
CREATE INDEX, got
CreateExtensionStmt" | trigram GIN alone | ✓ 93.8 → 0.94
ms (-99%) |
| W10 (20 ms), lite
model | index on date(created_at);
UTC-offset range rewrite | ✗ not IMMUTABLE; ✗ "1 row
missing, 1 extra" | B-tree on
created_at | ✗ planner
ignored it (the
filter still used
date()) |
| W10,
gemini-
3.5-flash | range rewrite with '2025-06-
17':::date bounds + covering
index (created_at, total) | ✓ same rows; 20.98 → 0.08 ms
(-99.6%) | — | — |

Documentation retrieval (RAG)

qopt keeps a small, version-specific slice of the PostgreSQL manual in its meta database, and retrieves the most relevant excerpts for each analysis.

OFFLINE · qopt docs ingest (once)



ONLINE · every analysis with RAG on

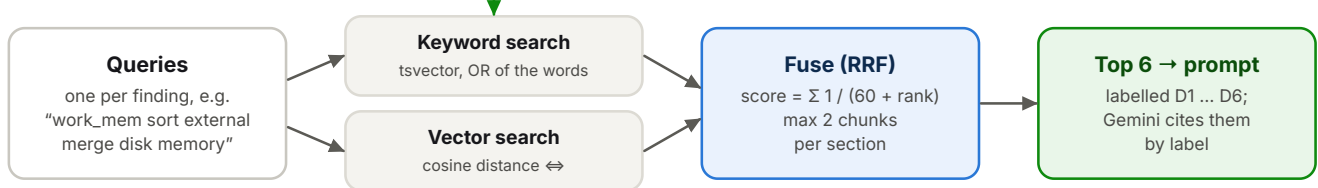


Figure 23. Offline, the manual is fetched, chunked, embedded and stored once. Online, each analysis turns its findings into search queries, runs keyword and vector search, fuses the rankings and passes the top six excerpts to Gemini.

12.1 The corpus

38 pages of the PostgreSQL 17 manual about query tuning: index types and their operator classes, multicolumn/expression/partial indexes, index-only scans, `EXPLAIN`, planner statistics and extended statistics, planner and resource settings (`work_mem`), `CREATE INDEX`, `CREATE STATISTICS`, subquery expressions (`NOT IN / EXISTS`), `LIMIT / OFFSET`, `pg_trgm`, JSON indexing, vacuum and the visibility map, `pg_stat_statements` and `auto_explain`. The version matches the target server, because behaviour and settings change between releases.

12.2 Chunking

Each page is split along its section headings into chunks of about 1,400 characters (never more than 2,000). Each chunk keeps the page title, its section heading, and a link to the section's anchor, so a citation opens the exact section. Navigation bars are dropped, as are long reference tables (more than 15 rows, such as operator-class listings), which drowned out prose in early tests. The result is 459 chunks.

12.3 Embeddings and storage

Chunks are embedded with `gemini-embedding-001` using the task type `RETRIEVAL_DOCUMENT`, and questions with `RETRIEVAL_QUERY`. The model returns 3,072 numbers by default; qopt asks for **768**, because pgvector's HNSW index supports at most 2,000 dimensions for the `vector` type. Texts are sent in batches of 50. The free tier allows 100 texts per minute, so ingestion waits when the server says so. Each chunk is stored in `doc_chunks` with a generated `tsvector` (GIN index) and its `vector(768)` (HNSW index).

12.4 Hybrid search

- Each **finding** has a built-in search phrase (`SORT_SPILL` → “work_mem sort external merge disk memory”). Up to five distinct phrases are used, plus one built from SQL keywords in the query (`ORDER BY` , `ILIKE` , `@>` ...).
- For each phrase, the 20 best keyword matches and the 20 nearest vectors are fetched. The keyword query ORs the words together, because requiring every word of a long phrase would match nothing.
- All lists are fused with RRF (`k = 60`). At most two chunks per section are kept, for variety, and the top six become D1...D6.

It works without a Gemini key too: then only keyword search runs. Two real retrievals: “extended statistics functional dependencies row estimation” returns sections 14.2.2.1 and 68.2.1 on functional dependencies and the `CREATE STATISTICS` parameters; “work_mem sort external merge disk memory” returns 19.4.1 (Resource Consumption: Memory) and the `EXPLAIN ANALYZE` section.

12.5 What RAG did, and did not, do

In the evaluation, the RAG configuration had the best precision (70%) and speedup (5.7×). But the difference is small enough to be noise, and when we checked which excerpts were retrieved for the one query it uniquely solved (W10), they were about covering and multicolumn indexes, not time zones. The honest conclusion is that retrieval *works* (the excerpts are relevant, and they are cited), but its effect on outcomes is not demonstrated by one run of 15 queries. [Chapter 30](#) discusses this further.

The safety model

qopt runs model-written SQL against databases. It is safe because of several independent layers, each of which would block most problems on its own.

- 1 **Nothing is applied for you**
Recommendations are printed as `CREATE INDEX CONCURRENTLY ...`. You decide what reaches production.
- 2 **The target is read-only**
`default_transaction_read_only` + `statement_timeout`. Only catalog reads, logs, plain `EXPLAIN` and session-local HypoPG.
- 3 **Every statement is parsed first**
The real Postgres parser checks the statement type: no `UNIQUE`, no `DML`, no `SELECT INTO` or row locks, no blocked functions.
- 4 **Experiments are sandboxed**
Every shadow experiment runs inside `BEGIN ... ROLLBACK`. DML baselines run in savepoints.
- 5 **Rewrites run read-only**
`transaction_read_only` is switched on before any model-written SQL runs; `statement_timeout` applies.
- 6 **Sample values can be redacted**
`QOPT_REDACT_VALUES` keeps `most_common_vals` (real data) out of LLM prompts.

Figure 24. Defence in depth. The outer layers limit what can reach a database at all; the inner layers limit what a statement can do once it runs.

| Layer | What it prevents |
|---------------------------|--|
| 1 · No automatic changes | A good-looking but harmful change reaching production. You apply recommendations yourself, with <code>CONCURRENTLY</code> so tables stay writable while the index builds. |
| 2 · Read-only target | Any write to the tuned database. Its sessions are opened with <code>default_transaction_read_only = on</code> and a 30-second <code>statement_timeout</code> . <code>EXPLAIN ANALYZE</code> is never run there. |
| 3 · Parser checks | A model disguising a <code>DROP</code> or <code>DELETE</code> as an “index”, a rewrite that modifies data through a CTE, or calls to dangerous functions. A test feeds in <code>DROP INDEX orders_pkey</code> labelled as an index and checks that it is rejected. |
| 4 · Sandboxed experiments | Leftovers on the shadow: every experiment ends in <code>ROLLBACK</code> , and an integration test checks that the number of indexes is unchanged afterwards. |
| 5 · Read-only rewrites | A rewrite writing data even if it slipped past the parser: the transaction is switched to read-only before model SQL runs. |
| 6 · Redaction | Real values leaking into prompts: <code>QOPT_REDACT_VALUES=true</code> omits the most-common-value lists from what is sent to Gemini. This matters on the free tier, where prompts may be used by Google. |

PART III

Technologies

Every technology the project uses: what it is, which version was used, why it was chosen, and where it appears. Then a map of the repository and all its settings.

- Chapter 14 · The technology stack
- Chapter 15 · Project structure and configuration

The technology stack

One language end to end (TypeScript), one database technology in three roles (PostgreSQL), and one AI provider (Google Gemini). The versions below are the ones installed when this guide was built.

14.1 Language and runtime

| Technology | Version | What it is · why it is used | Where |
|-----------------------|----------------------|--|--------------------------------|
| Node.js | 25.1 (≥ 22 required) | JavaScript runtime for the engine, CLI and API. Built-in <code>process.loadEnvFile</code> reads <code>.env</code> without a library; built-in <code>fetch</code> downloads the manual. | everything server-side |
| TypeScript | 7.0.2 | JavaScript with types. The shared types (<code>AnalysisResult</code> , <code>ValidationResult</code> ...) are what keep the engine, API and UI in agreement. Used only for type-checking (<code>npm run typecheck</code>); nothing is compiled ahead of time. | all packages |
| tsx | 4.23 | Runs <code>.ts</code> files directly, with no build step. <code>npm run qopt</code> is simply <code>tsx apps/cli/src/index.ts</code> ; <code>tsx watch</code> restarts the API on changes. | scripts |
| npm workspaces | npm 11 | Several packages in one repository with one install. <code>@qopt/core</code> is linked into the apps, so they import the engine's TypeScript source directly. | root <code>package.json</code> |

14.2 Database

| Technology | Version | What it is · why it is used | Where |
|---------------------------|---------|--|--|
| PostgreSQL | 17.11 | The database being tuned, the lab, and qopt's own store. Chosen because its planner is transparent (<code>EXPLAIN</code> in JSON), its DDL is transactional (experiments can be rolled back), and its extension ecosystem provides everything else below. | target · shadow · meta |
| pg_stat_statements | 1.11 | Bundled extension: cumulative time, calls and rows per normalized statement. | target · <code>capture/</code> |
| auto_explain | bundled | Bundled module: logs the actual plan of slow executions, as JSON, with the real query text. | target · <code>capture/</code> |
| HypoPG | 1.4.3 | Third-party extension for hypothetical indexes: “would the planner use this index?” without building it. | target (and shadow) · <code>validate/</code> |
| pgvector | 0.8.6 | Third-party extension adding the <code>vector</code> type, cosine distance (<code><=></code>) and HNSW indexes. Keeps the RAG store inside PostgreSQL, so there is no separate vector database. | meta · <code>rag/</code> |
| pg_trgm | 1.6 | Bundled extension for trigram matching; enables GIN indexes for <code>LIKE '%...%'</code> . Needed so qopt can recommend and validate them. | target · shadow |

14.3 Core libraries

| Library | Version | What it is · why it is used | Where |
|---------------------------|---------|--|----------------------------|
| pg (node-postgres) | 8.23 | The standard PostgreSQL driver for Node: connection pools, queries, parameters. <code>qopt</code> adds its own thin wrapper (<code>Db</code>) for read-only pools, sandbox transactions and query tagging. No ORM is used: <code>qopt</code> needs raw SQL for <code>EXPLAIN</code> , catalogs and HypoPG. | <code>db.ts</code> |
| libpg-query | 18.1.5 | PostgreSQL's own parser (from the PostgreSQL 18 source) compiled to WebAssembly. Produces the same parse tree PostgreSQL uses internally. <code>qopt</code> uses it to understand queries (tables, filters, joins) and to verify model output before running it. A regex could not do either safely. | <code>sql/</code> |
| @google/genai | 2.24 | Google's official Gemini SDK: <code>generateContent</code> with structured JSON output, and <code>embedContent</code> . | <code>llm/gemini.ts</code> |
| zod | 4.6.5 | Schema validation. One schema defines the LLM's answer format: <code>z.toJSONSchema()</code> turns it into the JSON Schema sent to Gemini, and <code>parse()</code> re-checks the reply. | <code>llm/schema.ts</code> |
| cheerio | 1.2 | jQuery-like HTML parsing on the server; used to split manual pages into sections. | <code>rag/chunk.ts</code> |

14.4 Applications

| Technology | Version | What it is · why it is used | Where |
|---------------------------------|---------|---|-----------------------|
| commander | 15.0 | Command-line argument parsing: sub-commands, options, help text. | <code>apps/cli</code> |
| Fastify (+ cors, static) | 5.12 | A fast, small HTTP framework with typed routes and built-in JSON logging. <code>@fastify/static</code> serves the built UI in production. | <code>apps/api</code> |
| React | 19.3 | UI library: the three pages and their components. | <code>apps/web</code> |
| Vite (+ plugin-react) | 8.3 | Dev server with instant reload, and the production bundler. Its proxy forwards <code>/api</code> to the API during development. | <code>apps/web</code> |

14.5 Testing, tooling and infrastructure

| Technology | Version | What it is · why it is used | Where |
|------------------------|---------|---|----------------------------------|
| Vitest | 5.0.2 | Test runner with native TypeScript support. Two projects: <i>unit</i> (no database) and <i>integration</i> (against the local databases, run one file at a time). | <code>vitest.config.ts</code> |
| concurrently | 10.0 | Runs the API and the web dev server together (<code>npm run dev</code>). | root scripts |
| Homebrew | — | macOS package manager: installs <code>postgresql@17</code> . HypoPG and pgvector are compiled from their official sources against it. | <code>scripts/local-db.sh</code> |
| Docker Compose | — | Alternative setup: the same three servers as containers built from <code>postgres:17</code> plus <code>postgresql-17-hypopg</code> and <code>postgresql-17-pgvector</code> . Not tested in this environment (Docker was not installed). | <code>docker-compose.yml</code> |
| Headless Chrome | — | Used to build this PDF guide from HTML; not part of the application. | — |

14.6 AI models

| Model | Used for |
|---|---|
| gemini-3.8-flash | Primary model for proposing fixes (GEMINI_MODEL). |
| gemini-3.7-flash , 3.6-flash , 3.5-flash , 3-flash-preview , 3.5-flash-lite | Fallbacks, tried in this order when the previous model is overloaded or out of quota (GEMINI_FALLBACK_MODELS). The published evaluation used gemini-3.5-flash-lite for every LLM configuration. |
| gemini-embedding-001 | Embeddings for documentation retrieval, reduced to 768 dimensions. |

14.7 Design choices worth knowing

- **TypeScript everywhere.** The engine, CLI, API and UI share one set of types. When a field is added to ValidationResult , the compiler points to every place that must change, including the React components.
- **The real parser, not regular expressions.** Safety checks have to be exact: a regular expression cannot reliably tell a harmless SELECT from one that deletes rows inside a CTE. The PostgreSQL parser can.
- **No ORM.** The project's job is to reason about SQL, so it writes SQL directly, always with parameters (\$1) for values.
- **Vectors inside PostgreSQL.** pgvector means no extra service to run, and documentation search uses the same tools as everything else.
- **A provider interface for the LLM.** The pipeline depends on a two-method interface (LlmProvider), not on Gemini. The tests plug in a scripted fake model; another provider could be added the same way.

Project structure and configuration

15.1 The repository

Repository layout (build output and caches omitted)

```

DB Project/
├─ package.json                workspaces + scripts (qopt, dev, test, typecheck ...)
├─ tsconfig.base.json          shared TypeScript settings
├─ vitest.config.ts             two test projects: unit and integration
├─ .env.example                 configuration template (copy to .env)
├─ docker-compose.yml          target / shadow / meta as containers
├─ docker/Dockerfile           postgres:17 + hypopg + pgvector
├─ scripts/local-db.sh         the same three servers via Homebrew
├─ packages/core/              @qopt/core: the engine
│   └─ src/
│       ├── config.ts           settings from .env, with defaults
│       ├── db.ts               pools, sessions, sandbox() and tx()
│       ├── types.ts            shared types (the web UI imports these too)
│       ├── setup.ts            extensions, seeding, checksums, status
│       ├── services.ts         wires up Gemini, retrieval and the run store
│       ├── pipeline.ts         analyze(): one analysis, start to finish
│       ├── capture/            pg_stat_statements + auto_explain samples
│       ├── sql/                ast.ts · shape.ts · safety.ts
│       ├── plan/               tree.ts · findings.ts · explain.ts
│       ├── context/            schema.ts (catalog + pg_stats)
│       ├── advisor/            heuristic.ts (rule-based advisor)
│       ├── llm/                provider · gemini · cache · schema · prompt · suggest
│       ├── rag/                chunk · ingest · retrieve
│       ├── validate/           validator.ts · equivalence.ts
│       ├── meta/               store.ts (tables in the meta database)
│       ├── workload/           seed.ts · queries.ts · runner.ts
│       ├── eval/               runner.ts (evaluation harness)
│       └─ shadow/              sync.ts (pg_dump | psql)
├─ test/                        *.test.ts (unit) · *.int.test.ts · fixtures/
├─ apps/cli/src/index.ts        the qopt command
├─ apps/api/src/server.ts       HTTP API (also serves the built UI)
├─ apps/web/src/                App.tsx · pages/ · components/ · api.ts · styles.css
├─ eval/results/                evaluation reports; latest.json feeds the UI
└─ docs/                        this guide

```

15.2 Settings (.env)

| Variable | Default | Meaning |
|------------------------|--|--|
| TARGET_URL | postgres://postgres:postgres@localhost:5433/qopt | the database being tuned |
| SHADOW_URL | (port 5434 in .env.example) | the lab copy; without it, validation is cost-only |
| META_URL | (port 5435 in .env.example) | qopt's storage; without it, runs are kept in memory and RAG is off |
| GEMINI_API_KEY | empty | enables Gemini; without it the heuristic advisor and keyword search still work |
| GEMINI_MODEL | gemini-3.8-flash | primary model |
| GEMINI_FALLBACK_MODELS | 3.7-flash, 3.6-flash, 3.5-flash, 3-flash-preview, 3.5-flash-lite | tried in order on overload or quota errors |

| Variable | Default | Meaning |
|---------------------------|----------------------|---|
| GEMINI_EMBED_MODEL | gemini-embedding-001 | embedding model (768 dimensions) |
| QOPT_MIN_COST_GAIN | 0.20 | HypoPG gate: estimated cost must drop by 20% |
| QOPT_MIN_TIME_GAIN | 0.15 | shadow gate: median time must drop by 15% |
| QOPT_MEASURE_RUNS | 5 | timed runs per measurement (after one warm-up) |
| QOPT_STATEMENT_TIMEOUT_MS | 30000 | per-statement limit on target and shadow sessions |
| QOPT_REDACT_VALUES | false | keep sample values out of prompts |
| QOPT_API_PORT | 8787 | API port (the Vite proxy uses it too) |
| QOPT_SLOW_MS, PGBIN | 5, Homebrew path | for scripts/local-db.sh :
auto_explain threshold,
PostgreSQL binaries |

15.3 npm scripts

| Command | What it does |
|--|--|
| npm run db:local -- init start stop status destroy | manage the three local Homebrew servers |
| npm run db:up / db:down | start / stop the Docker version |
| npm run qopt -- <command> | the CLI (see Appendix B) |
| npm run dev | API (port 8787, auto-restart) + web UI (port 5173, hot reload) |
| npm run build:web then npm start | production: the API serves the built UI on port 8787 |
| npm test / npm run test:unit | all 43 tests / the 34 unit tests only |
| npm run typecheck | type-check all four packages |

15.4 Processes and ports

| Port | Process |
|--------------------|---|
| 5433 · 5434 · 5435 | PostgreSQL: target · shadow · meta |
| 8787 | API server (and the built UI with npm start) |
| 5173 | Vite dev server for the UI (development only) |

PART IV

Code walkthrough

A guided tour of the most important code, in the order data flows through the system. Every excerpt is copied from the repository with its real line numbers; the text explains what each part does and why it is written that way.

- Chapter 16 · Configuration and database access
- Chapter 17 · Understanding SQL: parser, query shape, safety
- Chapter 18 · Reading plans and finding problems
- Chapter 19 · Capture and schema context
- Chapter 20 · The heuristic advisor
- Chapter 21 · The validator
- Chapter 22 · The LLM layer
- Chapter 23 · The RAG code
- Chapter 24 · The pipeline orchestrator
- Chapter 25 · Demo workload and evaluation harness
- Chapter 26 · The applications: CLI, API, web UI
- Chapter 27 · Tests

Configuration and database access

Two small files underpin everything: `config.ts` turns environment variables into typed settings, and `db.ts` wraps the PostgreSQL driver with the three behaviours qopt relies on: read-only pools, tagged queries and rolled-back sandboxes.

16.1 Settings with safe defaults

loadConfig(): every setting in one typed object

packages/core/src/config.ts · lines 53–75

```
53 export function loadConfig(): Config {
54   return {
55     targetUrl: str('TARGET_URL', 'postgres://postgres:postgres@localhost:5433/qopt'),
56     shadowUrl: str('SHADOW_URL'),
57     metaUrl: str('META_URL'),
58     geminiApiKey: str('GEMINI_API_KEY'),
59     geminiModel: str('GEMINI_MODEL', 'gemini-3.8-flash'),
60     geminiFallbackModels: str('GEMINI_FALLBACK_MODELS', 'gemini-3.7-flash,gemini-3.6-flash,gemini-3.5-
flash,gemini-3-flash-preview,gemini-3.5-flash-lite')
61       .split(',')
62       .map((m) => m.trim())
63       .filter(Boolean),
64     geminiEmbedModel: str('GEMINI_EMBED_MODEL', 'gemini-embedding-001'),
65     // pgvector's HNSW index supports at most 2000 dims for `vector`; Gemini defaults to 3072.
66     embedDimensions: 768,
67     minCostGain: num('QOPT_MIN_COST_GAIN', 0.2),
68     minTimeGain: num('QOPT_MIN_TIME_GAIN', 0.15),
69     measureRuns: Math.max(1, Math.round(num('QOPT_MEASURE_RUNS', 5))),
70     statementTimeoutMs: num('QOPT_STATEMENT_TIMEOUT_MS', 30000),
71     redactValues: bool('QOPT_REDACT_VALUES', false),
72     apiPort: num('QOPT_API_PORT', 8787),
73     cacheDir: resolve(REPO_ROOT, '.cache'),
74   };
75 }
```

- At import time, `config.ts` loads the `.env` file at the repository root with Node's built-in `process.loadEnvFile`. The small helpers `str`, `num` and `bool` fall back to the default when a value is missing or malformed, so a typo never crashes the CLI.
- `SHADOW_URL` and `META_URL` default to *empty*: each feature degrades gracefully without them (cost-only validation; in-memory runs).
- The embedding size is fixed at 768 with a comment explaining why: this is the kind of constraint that is invisible until it breaks.

16.2 Getting numbers back as numbers

Type parsers for 64-bit integers and numerics

packages/core/src/db.ts · lines 4–7

```
4 // int8 and numeric arrive as strings by default. Row counts, sizes and costs fit in a double;
5 // the one int8 that doesn't (pg_stat_statements.queryid) is always selected as ::text.
6 pg.types.setTypeParser(20, (v) => Number(v));
7 pg.types.setTypeParser(1700, (v) => Number(v));
```

By default, the `pg` driver returns PostgreSQL `bigint` and `numeric` values as *strings*, because they may not fit in a JavaScript number. Left alone, calls from `pg_stat_statements` would be `"10"`, and sorting would compare text. qopt converts them to numbers, which is safe for counts, sizes and costs. The one exception, the 64-bit `queryid`,

is always selected as text in SQL (`queryid::text`).

16.3 The Db class: pools, tags and read-only sessions

The tag that marks qopt's own statements

packages/core/src/db.ts · lines 11-12

```
11 /** Comment prefixed to every statement qopt itself sends to the target, so capture can ignore them. */
12 export const QOPT_TAG = '/*qopt*/';
```

Db: one connection pool per role

packages/core/src/db.ts · lines 32-51

```
32 export class Db {
33   readonly pool: pg.Pool;
34
35   constructor(
36     readonly role: DbRole,
37     readonly url: string,
38     private readonly opts: DbOptions = {},
39   ) {
40     const settings: string[] = [];
41     if (opts.statementTimeoutMs) settings.push(`-c statement_timeout=${opts.statementTimeoutMs}`);
42     if (opts.readOnly) settings.push(`-c default_transaction_read_only=on`);
43     this.pool = new pg.Pool({
44       connectionString: url,
45       application_name: `qopt-${role}`,
46       max: opts.max ?? 4,
47       options: settings.length ? settings.join(' ') : undefined,
48     });
49     // An idle client erroring (e.g. server restart) must not crash the process.
50     this.pool.on('error', () => {});
51   }
```

- Each `Db` owns a pool of connections. `statement_timeout` and `default_transaction_read_only` are set with the `options` connection parameter, so they apply to every session in the pool before any SQL runs.
- `application_name` is `qopt-target`, `qopt-shadow` and so on. Capture uses it to skip qopt's own `auto_explain` entries.
- The empty `pool.on('error')` handler matters: without it, a database restart would crash the whole Node process through an unhandled event.

16.4 sandbox(): the lab pattern

Db.sandbox(): every experiment ends in ROLLBACK

packages/core/src/db.ts · lines 66-76

```
66 /** Runs fn inside a transaction that is always rolled back: the lab pattern used for every experiment. */
67 sandbox<T>(fn: (s: Session) => Promise<T>, opts: { readOnly?: boolean } = {}): Promise<T> {
68   return this.withSession(async (s) => {
69     await s.query(opts.readOnly ? 'BEGIN READ ONLY' : 'BEGIN');
70     try {
71       return await fn(s);
72     } finally {
73       await s.query('ROLLBACK').catch(() => {});
74     }
75   });
76 }
```

This small function is the foundation of qopt's safety. It checks out one connection, starts a transaction (optionally read-only), runs the experiment, and rolls back in a `finally` block. The rollback happens whether the experiment returns, throws, or times out. Everything the experiment did, including `CREATE INDEX`, `CREATE STATISTICS`, `ANALYZE` results and `SET LOCAL`, disappears.

16.5 Four roles, created once

getDb(): the four database handles

packages/core/src/db.ts · lines 118-128

```
118 export function getDb(): Db {
119   if (db) return db;
120   const timeout = config.statementTimeoutMs;
121   db = {
122     target: new Db('target', config.targetUrl, { readOnly: true, statementTimeoutMs: timeout, tag: true }),
123     targetAdmin: new Db('target-admin', config.targetUrl, { max: 2 }),
124     shadow: config.shadowUrl ? new Db('shadow', config.shadowUrl, { statementTimeoutMs: timeout, max: 4 })
125     : undefined,
126     meta: config.metaUrl ? new Db('meta', config.metaUrl, { max: 4 }) : undefined,
127   };
128   return db;
129 }
```

Note the two handles for the target: `target` is read-only and tagged, and is used for all analysis. `targetAdmin` is writable, but only setup and the workload generator use it. The workload deliberately looks like an application, untagged, so capture sees it.

Understanding SQL: parser, query shape, safety

The `sql/` folder gives qopt a real understanding of SQL. `ast.ts` wraps the parser, `shape.ts` extracts what an index advisor needs from a query, and `safety.ts` decides whether model-written SQL may run at all.

17.1 Loading the real PostgreSQL parser

`initParser()`: load the WebAssembly parser once

packages/core/src/sql/ast.ts · lines 3–12

```
3 // libpg-query is the real Postgres parser compiled to WASM. It must be loaded once before use.
4 let loading: Promise<void> | undefined;
5 let ready = false;
6
7 export async function initParser(): Promise<void> {
8   loading ??= loadModule().then(() => {
9     ready = true;
10   });
11   await loading;
12 }
```

The parser is WebAssembly and loads asynchronously, but parsing itself is synchronous and fast. Every entry point (CLI, API, tests, pipeline) awaits `initParser()` once; `??=` makes repeated calls share the same promise.

The parser returns a JSON tree in which every node looks like `{"ColumnRef": {...}}`: one key naming the node type. Two helpers make that tree easy to work with:

`unwrap()`: recognise a node

packages/core/src/sql/ast.ts · lines 57–66

```
57 /** A wrapped AST node looks like {"ColumnRef": {...}}: one key, capitalised, object value. */
58 export function unwrap(node: unknown): { type: string; body: AstNode } | undefined {
59   if (!node || typeof node !== 'object' || Array.isArray(node)) return undefined;
60   const keys = Object.keys(node);
61   if (keys.length !== 1) return undefined;
62   const type = keys[0];
63   const body = (node as AstNode)[type];
64   if (!/^[A-Z]/.test(type) || typeof body !== 'object' || body === null) return undefined;
65   return { type, body };
66 }
```

`walk()`: visit every node in the tree

packages/core/src/sql/ast.ts · lines 68–80

```
68 /** Depth-first visit of every wrapped node below `node` (including itself). */
69 export function walk(node: unknown, visit: (type: string, body: AstNode) => void): void {
70   if (Array.isArray(node)) {
71     for (const item of node) walk(item, visit);
72     return;
73   }
74   if (!node || typeof node !== 'object') return;
75   const w = unwrap(node);
76   if (w) visit(w.type, w.body);
77   for (const value of Object.values(w ? w.body : (node as AstNode))) {
78     if (value && typeof value === 'object') walk(value, visit);
79   }
80 }
```

Here is what the tree looks like for a simple filter, `WHERE o.customer_id = 42` (abridged):

Parse tree fragment produced by libpg-query

```
{ "A_Expr": {
  "kind": "AEXPR_OP",
  "name": [ { "String": { "sval": "=" } } ],
  "lexpr": { "ColumnRef": { "fields": [ { "String": { "sval": "o" } },
    { "String": { "sval": "customer_id" } } ] } },
  "rexpr": { "A_Const": { "ival": { "ival": 42 } } } }
```

17.2 From a parse tree to a “query shape”

`analyzeQuery()` turns the tree into a `QueryShape`: the tables, the filter predicates (each classified), the join pairs, `ORDER BY`, `GROUP BY`, `LIMIT / OFFSET` and a list of anti-patterns. It keeps a **scope** per `SELECT` (including subqueries and CTEs) that maps aliases to tables, so `o.customer_id` resolves to `orders.customer_id`. Unqualified columns are resolved using the column lists from the catalog. The core is the classification of each comparison:

Two columns compared with = across tables: a join

packages/core/src/sql/shape.ts · lines 317–321

```
317 // column = column across two tables is a join key.
318 if (op === '=' && body.kind === 'AEXPR_OP' && lcol && rcol) {
319   if (lcol.table !== rcol.table || !lcol.table) this.shape.joins.push({ left: lcol, right: rcol });
320   return;
321 }
```

Classifying a predicate: eq, range, contains, like...

packages/core/src/sql/shape.ts · lines 336–367

```
336 let kind: PredKind = 'other';
337 const value = exprText(rhs);
338 switch (body.kind) {
339   case 'AEXPR_OP':
340     if (op === '=') kind = 'eq';
341     else if (op === '<>' || op === '!=') kind = 'neq';
342     else if (RANGE_OPS.has(op)) kind = 'range';
343     else if (CONTAINS_OPS.has(op)) kind = 'contains';
344     break;
345   case 'AEXPR_IN':
346     kind = op === '=' ? 'in' : 'other';
347     break;
348   case 'AEXPR_BETWEEN':
349   case 'AEXPR_BETWEEN_SYM':
350     kind = 'range';
351     break;
352   case 'AEXPR_LIKE':
353   case 'AEXPR_ILIKE': {
354     if (op.startsWith('!')) break;
355     const pattern = unwrap(rhs)?.type === 'A_Const' ? String(constValue(unwrap(rhs)!.body)) : '';
356     const leading = /^[%_]/.test(pattern);
357     kind = leading || body.kind === 'AEXPR_ILIKE' ? 'like_infix' : 'like_prefix';
358     if (leading) {
359       this.shape.antiPatterns.push({
360         code: 'LEADING_WILDCARD',
361         relation: target.table,
362         detail: `${body.kind === 'AEXPR_ILIKE' ? 'ILIKE' : 'LIKE'} '${pattern}' starts with a wildcard, so
no B-tree can serve it; a trigram (pg_trgm) GIN index can`,
363       });
364     }
365     break;
366   }
367 }
```

- The classification (`eq`, `in`, `range`, `contains`, `like_prefix`, `like_infix` ...) is exactly what the index rules need: equality columns first, one range column, GIN for `@>`, trigram for a leading `%`.
- Anti-patterns are recorded while walking: a leading wildcard, a function around a column (`lower(email)`, `date(created_at)`), `NOT IN (subquery)`, `OR` across different columns, a large `OFFSET`. They become SQL *findings* alongside the plan findings.
- Predicates inside an `OR` are marked `inOr`, so the advisor knows an index on just one branch cannot help.

17.3 Deciding whether SQL may run

`checkCandidate()`: an "index" must be CREATE INDEX, and nothing else

packages/core/src/sql/safety.ts · lines 112–119

```
112 case 'index': {
113   if (!c.statements.length) return fail('invalid_sql', 'index candidate has no CREATE INDEX statement');
114   for (const s of c.statements) {
115     const r = checkIndex(s);
116     if (typeof r === 'string') return fail('unsafe', r);
117     checked.indexes.push(r);
118   }
119   break;
```

`checkRewriteAst()`: walk every node of a rewrite looking for danger

packages/core/src/sql/safety.ts · lines 89–105

```
89 function checkRewriteAst(body: AstNode): string | undefined {
90   let problem: string | undefined;
91   walk({ SelectStmt: body }, (type, node) => {
92     if (problem) return;
93     if (type === 'InsertStmt' || type === 'UpdateStmt' || type === 'DeleteStmt' || type === 'MergeStmt') {
94       problem = `rewrite contains a data-modifying ${type.replace('Stmt', '')}.toUpperCase()`;
95     } else if (type === 'SelectStmt' && node.intoClause) {
96       problem = 'SELECT INTO creates a table';
97     } else if (type === 'SelectStmt' && node.lockingClause) {
98       problem = 'row locking clauses (FOR UPDATE/SHARE) are not allowed in a rewrite';
99     } else if (type === 'FuncCall') {
100       const name = strings(node.funcname).pop() ?? '';
101       if (FORBIDDEN_FUNCTIONS.some((re) => re.test(name))) problem = `function ${name}() is not allowed`;
102     }
103   });
104   return problem;
105 }
```

A rewrite may contain subqueries and CTEs, so the check walks the *entire* tree: any `InsertStmt`, `UpdateStmt`, `DeleteStmt` or `MergeStmt` hidden in a CTE, any `SELECT ... INTO` (which creates a table), any row lock, and any call to a blocked function makes the candidate `unsafe`. Because this runs on the parse tree, spacing, comments, quoting and letter case cannot fool it.

Reading plans and finding problems

The `plan/` folder turns PostgreSQL's JSON plan into a typed tree, derives the numbers that matter, and runs the deterministic checks.

18.1 Self time and q-error

`deriveTimes()`: inclusive time, self time, estimate error

`packages/core/src/plan/tree.ts` · lines 67–80

```
67 function deriveTimes(node: PlanNode): void {
68   for (const c of node.children) deriveTimes(c);
69   if (node.actualTotalTime === undefined) return;
70   // "Actual Total Time" is an average per loop; multiply by loops for the node's total.
71   node.inclusiveMs = node.actualTotalTime * (node.actualLoops ?? 1);
72   // InitPlans/SubPlans run inside their parent, so subtracting every child is correct.
73   const childMs = node.children.reduce((sum, c) => sum + (c.inclusiveMs ?? 0), 0);
74   node.exclusiveMs = Math.max(0, node.inclusiveMs - childMs);
75   if (node.actualRows !== undefined && node.actualLoops) {
76     const est = Math.max(node.planRows, 1);
77     const act = Math.max(node.actualRows, 1);
78     node.qError = Math.max(est, act) / Math.min(est, act);
79   }
80 }
```

- `Actual Total Time` is an average *per loop*, so it is multiplied by `Actual Loops` to get the node's total.
- Self (exclusive) time is total minus the children's totals, clamped at zero. Clamping handles parallel plans, where children's summed worker time can exceed the parent's wall-clock time.
- The q-error uses `max(..., 1)` on both sides so that zero rows do not divide by zero.

18.2 Reporting a misestimate once, where it starts

An early version flagged a misestimate on every node above the real culprit, and on nodes under a `LIMIT` that simply stopped early. Both were noise, and both were fixed by these two rules:

Two rules that remove false alarms

`packages/core/src/plan/findings.ts` · lines 47–59

```
47 // Nodes below a Limit stop early, so "fewer rows than estimated" there is expected, not a misestimate.
48 const underLimit = new Set<number>();
49 const markLimit = (node: PlanNode, inside: boolean) => {
50   if (inside) underLimit.add(node.id);
51   for (const c of node.children) markLimit(c, inside || node.nodeType === 'Limit');
52 };
53 markLimit(plan.root, false);
54 const misestimated = (node: PlanNode): boolean => {
55   if (node.qError === undefined || node.qError < 10) return false;
56   if (Math.max(node.planRows, node.actualRows ?? 0) < 100) return false;
57   const over = (node.actualRows ?? 0) < node.planRows;
58   return !(over && underLimit.has(node.id));
59 };
```

Only the node where the error originates is reported

`packages/core/src/plan/findings.ts` · lines 96–97

```
96 // Report a misestimate only where it originates; parents inherit their children's errors.
97 if (misestimated(node) && !node.children.some(misestimated) && node.qError !== undefined) {
```

The unit tests use the real W11 plan to check that exactly the customers scan is reported, and the real W09 plan to check that a `LIMIT` does not trigger a false alarm.

18.3 Measuring a query fairly

measure(): one warm-up, N timed runs, median

packages/core/src/plan/explain.ts · lines 43–60

```
43 export async function measure(s: Session, sql: string, runs: number): Promise<Measurement> {
44   const results: ExplainResult[] = [];
45   for (let i = 0; i <= runs; i++) {
46     await s.query('SAVEPOINT qopt_measure');
47     try {
48       const plan = await explainAnalyze(s, sql);
49       if (i > 0) results.push(plan);
50     } finally {
51       await s.query('ROLLBACK TO SAVEPOINT qopt_measure');
52     }
53   }
54   const samples = results.map((r) => r.executionMs ?? 0);
55   const med = median(samples);
56   const plan = results.reduce((best, r) =>
57     Math.abs((r.executionMs ?? 0) - med) < Math.abs((best.executionMs ?? 0) - med) ? r : best,
58   );
59   return { medianMs: med, samples, plan };
60 }
```

- The loop runs `runs + 1` times and discards the first result: the warm-up that loads pages into memory.
- Each run sits in its own savepoint that is rolled back, so a measured `UPDATE` sees identical data every time.
- The function returns the *plan of the median run*, not just the median number, so the UI shows a plan that matches the reported time.

Capture and schema context

19.1 The statements query

STATEMENTS_SQL: the top statements, without qopt's own noise

packages/core/src/capture/capture.ts · Lines 8-19

```

8  const STATEMENTS_SQL = `
9    SELECT queryid::text AS queryid, query, calls, total_exec_time, mean_exec_time, max_exec_time, rows,
10           shared_blks_hit, shared_blks_read, temp_blks_written
11  FROM pg_stat_statements
12  WHERE dbid = (SELECT oid FROM pg_database WHERE datname = current_database())
13         AND toplevel
14         AND query NOT LIKE '/*qopt*/%'
15         AND query ~* '^\\s*(select|with|insert|update|delete)\\M'
16         AND query !~* '\\m(pg_catalog|information_schema|pg_stat_statements|pg_stat_[a-z_]+|hypopg[a-
17           z_]*|pg_class|pg_namespace|pg_index|pg_attribute|pg_extension|pg_settings|pg_database|pg_ls_logdir|pg_read_
18           file)\\M'
19         %FILTER%
20  ORDER BY total_exec_time DESC
21  LIMIT $1`;

```

The `%FILTER%` placeholder lets the same SQL serve two purposes: the top-N list, and fetching a single statement by `queryid`. The `toplevel` column excludes statements executed inside functions.

19.2 Parsing auto_explain's log lines

parseAutoExplainLog(): exact ids, plans and literal query text

packages/core/src/capture/capture.ts · Lines 44-72

```

44 // query_id is a signed 64-bit hash: JSON.parse would round it, so read it from the raw text.
45 const QUERY_ID_RE = /"query_id":(-?\d+)/;
46 const DURATION_RE = /^duration: ([\d.]+) ms\s+plan:\n/;
47
48 /** Parse jsonlog lines written by auto_explain (log_format = json, log_destination = jsonlog). */
49 export function parseAutoExplainLog(text: string): PlanSample[] {
50   const out: PlanSample[] = [];
51   for (const line of text.split('\n')) {
52     if (!line.includes("message": "duration: ")) continue;
53     let rec: any;
54     try {
55       rec = JSON.parse(line);
56     } catch {
57       continue;
58     }
59     if (typeof rec.application_name !== 'string' || !rec.application_name.startsWith('qopt-')) continue;
60     const qid = QUERY_ID_RE.exec(line)?.[1];
61     const m = DURATION_RE.exec(rec.message ?? '');
62     if (!qid || qid === '0' || !m) continue;
63     try {
64       const doc = JSON.parse(rec.message.slice(m[0].length));
65       const plan = normalizeExplain(doc);
66       out.push({ queryid: qid, queryText: doc['Query Text'], durationMs: Number(m[1]), loggedAt:
67         rec.timestamp, plan });
68     } catch {
69       // Truncated or non-JSON plan (e.g. log_format = text); skip.
70     }
71   }
72   return out;
73 }

```

Each log line is a JSON record; its `message` field contains `duration: 10.227 ms plan:` followed by the plan as a JSON document. The `query_id` is read with a regular expression from the *raw line* (see [Section 9.3](#)) before the line is parsed. Anything unexpected, such as truncated lines, text-format plans or other log messages, is skipped instead of failing the whole capture.

19.3 Schema context: the statistics query

introspectTables(): pg_stats for the columns in use

packages/core/src/context/schema.ts · lines 71-78

```
71 const stats = await db.query(  
72   `SELECT attname, null_frac, n_distinct, correlation, avg_width,  
73         most_common_vals::text AS mcv, most_common_freqs AS mcf,  
74         coalesce(array_length(histogram_bounds, 1), 0) AS hist  
75   FROM pg_stats WHERE schemaname = $1 AND tablename = $2 ${wanted?.size ? 'AND attname = ANY($3)' : ''}  
76   ORDER BY attname`,  
77   wanted?.size ? [t.nspname, t.relname, [...wanted]] : [t.nspname, t.relname],  
78 );
```

`introspectTables()` resolves each table name with `to_regclass()` (which follows the search path, like PostgreSQL itself), then gathers columns, indexes (definition, size, scan count), constraints, extended statistics, write activity and these column statistics. Negative `n_distinct` values are converted to absolute counts. `most_common_vals` is truncated, and omitted entirely when `QOPT_REDACT_VALUES` is on.

The heuristic advisor

A deterministic, rule-based advisor in the spirit of the open-source tool Dexter. It is the baseline the LLM is measured against, and it runs alongside Gemini in the default “both” mode.

20.1 The rules

| # | Rule | Produces |
|----|---|---|
| 1 | equality columns (most distinct values first), then one range column | B-tree (eq..., range) |
| 2 | equality + ORDER BY + LIMIT | B-tree (eq..., order by...) |
| 3 | ORDER BY + LIMIT / OFFSET without filters | B-tree on the sort columns |
| 4 | low-cardinality equality constant (≤ 10 distinct values) + a range or sort column | partial B-tree ... WHERE col = 'value' |
| 5 | function around a column | expression index |
| 6 | @> and other containment operators | GIN (jsonb_path_ops for jsonb) |
| 7 | LIKE '%x%' / ILIKE (if pg_trgm is installed); LIKE 'x%' | trigram GIN; B-tree text_pattern_ops |
| 8 | OR across columns | one B-tree per branch, as one candidate |
| 9 | range on a table $\geq 500k$ rows with correlation ≥ 0.9 | BRIN |
| 10 | single-table GROUP BY (+ aggregate DISTINCT) fed by a sort | B-tree in group order |
| 11 | join column on a large, sequentially scanned table | B-tree on the join key |
| 12 | misestimate on a table with ≥ 2 equality filters | CREATE STATISTICS (dependencies, mcv) |
| 13 | any spill (sort, hash, lossy bitmap) | SET work_mem sized to the spill |

Candidates that duplicate an existing index (same leading columns) are skipped, and at most eight are returned. The advisor proposes freely because validation is the filter: in the evaluation, 77% of its proposals passed.

20.2 Rules 1–4 in code

The multicolumn, ORDER BY and partial-index rules

packages/core/src/advisor/heuristic.ts · lines 81–102

```
81 // 1. Equality then range (the classic multicolumn rule).
82 if (eq.length || range) {
83   const keys = [...eq, ...(range ? [range] : [])].slice(0, 3);
84   btree(t, keys, `Filter on ${keys.join(', ')}: equality columns lead, the range column goes last so one
    index range scan covers the predicate.`);
85 }
86 // 2. Equality then ORDER BY, so a LIMIT can stop early without sorting.
87 if (eq.length && sortCols.length && shape.limit !== undefined) {
88   const keys = [...eq, ...sortCols.filter((c) => !eq.includes(c))].slice(0, 3);
89   btree(t, keys, `Equality on ${eq.join(', ')} plus ORDER BY ${sortCols.join(', ')}: the index returns rows
    already sorted, so LIMIT ${shape.limit} reads only ${shape.limit} entries.`);
90 }
91 // 3. ORDER BY alone (pagination / top-N).
92 if (!eq.length && !range && sortCols.length && (shape.limit !== undefined || shape.offset !== undefined)) {
93   btree(t, sortCols, `ORDER BY ${sortCols.join(', ')} with LIMIT: an index in sort order avoids sorting the
    whole table.`);
94 }
95 // 4. Partial index: a low-cardinality equality constant filters a range/sort column.
96 const lowCard = and.find((p) => p.kind === 'eq' && p.column && p.value && nDistinct(t, p.column) > 0 &&
    nDistinct(t, p.column) <= 10);
97 const partialKey = range ?? sortCols[0];
98 if (lowCard && partialKey && partialKey !== lowCard.column) {
99   btree(t, [partialKey], `Only rows with ${lowCard.column} = ${lowCard.value} are queried; a partial index
    stores just those, so it is small and cheap to maintain.`, {
100     where: `${lowCard.column} = ${quoteLiteral(lowCard.value!)}`,
101   });
102 }
```

20.3 Sizing work_mem from the evidence

Rule 13: work_mem sized to the observed spill

packages/core/src/advisor/heuristic.ts · lines 200–212

```
200 // 13. Spills: size work_mem to the observed spill.
201 const spill = planNodes.reduce((m, n) => Math.max(m, n.sortSpaceType === 'Disk' ? n.sortSpaceUsedKb ?? 0 :
    0, n.diskUsageKb ?? 0), 0);
202 if (spill > 0 || findings.some((x) => x.code === 'LOSSY_BITMAP' || x.code === 'HASH_SPILL')) {
203   // In-memory sorts need roughly 2-3x the on-disk run size.
204   const neededMb = Math.max(16, 2 ** Math.ceil(Math.log2(Math.max(1, (spill * 3) / 1024))));
205   const mb = Math.min(1024, neededMb);
206   add({
207     kind: 'config',
208     title: `SET work_mem = '${mb}MB' for this query`,
209     statements: [`SET work_mem = '${mb}MB`],
210     rationale: `The plan spilled ${kb(spill)} to disk. Raising work_mem for this session/statement (not
    globally: it is per sort/hash node, per connection) lets it run in memory.`,
211   });
212 }
```

An in-memory sort needs roughly two to three times the size of its on-disk runs, so the rule multiplies the spill by three, rounds up to a power of two, and clamps between 16 MB and 1 GB. For W13's 45 MB spill, that gives 256 MB. The rule is reasonable, and validation still rejected it: in two separate runs the query was only 7.7% and 13.2% faster, both below the 15% threshold. The index from rule 10 was the real fix.

The validator

`validate/validator.ts` implements [Chapter 10](#). Its public function, `validateCandidate()`, runs the static check, dispatches by kind, catches every error, classifies it, and scores accepted candidates.

21.1 HypoPG in one session

`hypoCheck(): before and after, in the same session`

`packages/core/src/validate/validator.ts · lines 73–93`

```
73 async function hypoCheck(db: Db, ctx: ValidationContext, indexes: ExecIndex[]) {
74   return db.withSession(async (s) => {
75     await s.query('SELECT hypopg_reset()');
76     try {
77       const before = await explainCost(s, ctx.sql, { generic: ctx.generic });
78       const names: string[] = [];
79       const oids: number[] = [];
80       for (const idx of indexes) {
81         const res = await s.query('SELECT indexrelid::int AS oid, indexname FROM hypopg_create_index($1)',
[ idx.execSql ]);
82         names.push(res.rows[0].indexname);
83         oids.push(res.rows[0].oid);
84       }
85       const after = await explainCost(s, ctx.sql, { generic: ctx.generic });
86       const size = await s.query('SELECT coalesce(sum(hypopg_relation_size(o)), 0)::bigint AS b FROM
unnest($1::oid[]) o', [ oids ]);
87       const used = indexesUsed(after).filter((n) => names.includes(n));
88       return { before, after, used, bytes: Number(size.rows[0].b) };
89     } finally {
90       await s.query('SELECT hypopg_reset()').catch(() => {});
91     }
92   });
93 }
```

Hypothetical indexes live in one backend's memory, so everything must happen on **one** connection (`withSession`), not across a pool. The `finally` block resets them even on error, so a pooled connection never carries a leftover hypothetical index into the next query.

21.2 The index path, end to end

`validateIndex(): HypoPG gate, then a real build on the shadow`

`packages/core/src/validate/validator.ts · lines 137–194`

```
137 async function validateIndex(c: CheckedCandidate, ctx: ValidationContext): Promise<ValidationResult> {
138   const hypoable = c.indexes.every((i) => HYPOPG_METHODS.has(i.shape.method));
139   const hypoDb = ctx.hypopgOnTarget ? ctx.target : ctx.hypopgOnShadow ? ctx.shadow : undefined;
140   const v: ValidationResult = { verdict: 'rejected', method: 'hypopg', reason: '', score: 0 };
141
142   if (hypoable && hypoDb) {
143     const h = await hypoCheck(hypoDb, ctx, c.indexes);
144     v.cost = delta(h.before.totalCost, h.after.totalCost);
145     v.usesCandidate = h.used.length > 0;
146     v.indexBytes = h.bytes;
147     v.afterPlan = h.after;
148     if (!v.usesCandidate) {
149       v.reason = `HypoPG: the planner ignored the hypothetical index (cost ${h.before.totalCost.toFixed(0)}
→ ${h.after.totalCost.toFixed(0)})`;
150       return v;
151     }
152   }
```

```

152     if (v.cost.change > -ctx.minCostGain) {
153         v.reason = `HypoPG: the index is used but estimated cost only changes ${pct(v.cost.change)}
(threshold ${ctx.minCostGain * 100}.toFixed(0)}%)`;
154         return v;
155     }
156     if (!ctx.shadow || ctx.generic) {
157         v.verdict = 'accepted';
158         v.reason = `HypoPG: planner uses it; estimated cost ${h.before.totalCost.toFixed(0)} →
${h.after.totalCost.toFixed(0)} (${pct(v.cost.change)})${ctx.generic ? ' ; not timed (no captured parameter
values)' : ' ; not timed (no shadow configured)'}`;
159         return v;
160     }
161     v.method = 'hypopg+shadow';
162 } else if (!ctx.shadow) {
163     return { verdict: 'skipped', method: 'static', reason: `${c.indexes.map((i) =>
i.shape.method).join('/')} indexes cannot be simulated by HypoPG and no shadow database is configured`,
score: 0 };
164 } else {
165     v.method = 'shadow';
166 }
167
168 // Real build on the shadow, rolled back afterwards.
169 const shadow = ctx.shadow!;
170 const r = await shadow.sandbox(async (s) => {
171     // Without a timed baseline (generic plans), compare estimated costs within this transaction.
172     const before = ctx.baseline ? undefined : await explainCost(s, ctx.sql, { generic: ctx.generic });
173     const built = await buildIndexes(s, c.indexes);
174     const after = await measureOrCost(s, ctx);
175     return { built, after, before };
176 });
177 v.indexBytes = r.built.bytes;
178 v.afterPlan = r.after.plan;
179 const used = indexesUsed(r.after.plan).filter((n) => r.built.names.includes(n));
180 v.usesCandidate = used.length > 0;
181 if (!v.cost) v.cost = delta(r.before?.totalCost ?? ctx.baseline?.plan.totalCost ?? 0,
r.after.plan.totalCost);
182 if (r.after.medianMs === undefined || !ctx.baseline) {
183     v.verdict = v.usesCandidate && v.cost.change <= -ctx.minCostGain ? 'accepted' : 'rejected';
184     v.reason = `shadow (cost only): ${v.usesCandidate ? 'used' : 'not used'} by the planner, cost
${pct(v.cost.change)}`;
185     return v;
186 }
187 v.timeMs = { ...delta(ctx.baseline.medianMs, r.after.medianMs), samples: r.after.samples ?? 0 };
188 if (!v.usesCandidate) {
189     v.reason = `shadow: index built but the planner did not use it (${v.timeMs.before.toFixed(2)} →
${v.timeMs.after.toFixed(2)} ms)`;
190     return v;
191 }
192 timeVerdict(v, ctx, v.method === 'hypopg+shadow' ? `HypoPG cost ${pct(v.cost.change)}; shadow` :
'shadow');
193 return v;
194 }

```

- **Lines 142–166:** the HypoPG gate, for index methods it supports. Without a shadow, or for a generic plan, an index that passes this gate is accepted on cost alone, and the reason says so.
- **Lines 169–176:** the real build and measurement inside `sandbox()`. With no timed baseline (generic plans), the cost before the build is taken inside the same transaction.
- **Lines 179–193:** the decision. The index must appear in the new plan, and the median time must beat the threshold.

21.3 Rewrites: build, lock down, compare, time

validateRewrite(): the sandbox body

packages/core/src/validate/validator.ts · lines 259-267

```
259 const rewrite = c.rewrite!;
260 const r = await ctx.shadow.sandbox(async (s) => {
261   const built = c.indexes.length ? await buildIndexes(s, c.indexes) : { names: [], bytes: 0, analyzed:
false };
262   // From here on nothing may write, whatever the rewrite contains.
263   await s.query('SET LOCAL transaction_read_only = on');
264   const equivalence = await checkEquivalence(s, ctx.sql, rewrite, ctx.ordered);
265   const m = equivalence.equal ? await measure(s, rewrite, ctx.measureRuns) : undefined;
266   return { built, equivalence, m };
267 });
```

The order matters. Supporting indexes are built first (writes are still allowed), then

SET LOCAL transaction_read_only = on locks the transaction. From that moment, even if a data-modifying statement had slipped past the parser, PostgreSQL itself would refuse it.

checkEquivalence(): counts, then EXCEPT ALL in both directions

packages/core/src/validate/equivalence.ts · lines 33-51

```
33 const counts = await s.query(`SELECT (SELECT count(*) FROM (${o}) a) AS n1, (SELECT count(*) FROM (${r}) b)
AS n2`);
34 const n1 = Number(counts.rows[0].n1);
35 const n2 = Number(counts.rows[0].n2);
36 if (n1 !== n2) return { equal: false, rowsOriginal: n1, rowsRewrite: n2, detail: `row counts differ: ${n1}
vs ${n2}` };
37
38 let multisetEqual: boolean | undefined;
39 await s.query('SAVEPOINT qopt_equiv');
40 try {
41   const diff = await s.query(
42     `SELECT (SELECT count(*) FROM ((${o}) EXCEPT ALL (${r})) x) AS missing,
43           (SELECT count(*) FROM ((${r}) EXCEPT ALL (${o})) y) AS extra`,
44   );
45   const missing = Number(diff.rows[0].missing);
46   const extra = Number(diff.rows[0].extra);
47   await s.query('RELEASE SAVEPOINT qopt_equiv');
48   if (missing || extra) {
49     return { equal: false, rowsOriginal: n1, rowsRewrite: n2, detail: `${missing} rows missing and ${extra}
extra rows compared with the original` };
50   }
51   multisetEqual = true;
```

21.4 Errors and scores

classify(): turning PostgreSQL errors into categories

packages/core/src/validate/validator.ts · lines 33–53

```
33 function classify(e: unknown): { errorClass: ErrorClass; reason: string } {
34   const code = pgErrorCode(e);
35   const msg = errorMessage(e);
36   // HypoPG re-raises some catalog lookups as XX000, so match the message too.
37   if (/\\b(column|relation|function|operator( class)?|type|access method|schema)\\b.*\\bdoes not
  exist\\b/i.test(msg)) {
38     return { errorClass: 'hallucination', reason: `references something that does not exist: ${msg}` };
39   }
40   switch (code) {
41     case '42703':
42     case '42P01':
43     case '42883':
44     case '42704':
45       return { errorClass: 'hallucination', reason: `references something that does not exist: ${msg}` };
46     case '42601':
47       return { errorClass: 'invalid_sql', reason: `syntax error: ${msg}` };
48     case '57014':
49       return { errorClass: 'timeout', reason: `timed out: ${msg}` };
50     default:
51       return { errorClass: 'db_error', reason: msg };
52   }
53 }
```

score(): gain minus size and write penalties

packages/core/src/validate/validator.ts · lines 58–71

```
58 function score(v: ValidationResult, checked: CheckedCandidate, tables: TableContext[]): number {
59   const gain = -(v.timeMs?.change ?? v.cost?.change ?? 0);
60   let s = gain * 100;
61   // Penalise index size relative to the table, and write-heavy tables (every index slows writes).
62   for (const idx of checked.indexes) {
63     const t = tables.find((x) => x.name === idx.shape.table);
64     if (!t) continue;
65     if (v.indexBytes && t.tableBytes) s -= Math.min(20, (v.indexBytes / t.tableBytes) * 10);
66     const writes = t.activity.inserts + t.activity.updates + t.activity.deletes;
67     const reads = t.activity.seqScans + t.activity.idxScans;
68     if (writes > 10 * Math.max(1, reads)) s -= 5;
69   }
70   return Math.round(s * 10) / 10;
71 }
```

The LLM layer

Five small files: an interface, a Gemini implementation, a disk cache, the answer schema, and the prompt builder plus the conversation that uses it.

22.1 A two-method interface

LlmProvider: all the pipeline knows about an LLM

packages/core/src/llm/provider.ts · lines 27–31

```
27 export interface LlmProvider {
28   readonly name: string;
29   readonly model: string;
30   generateJson<T>(req: LlmJsonRequest, parse: (value: unknown) => T): Promise<LlmJsonResponse<T>>;
31 }
```

The pipeline never imports Gemini directly. Anything with a `model` name and a `generateJson()` method can be plugged in: the integration tests use a scripted fake, and the evaluation uses a Gemini client pinned to one model. `generateJson` takes a `parse` function, so the caller decides how the answer is validated (here, with zod).

22.2 The Gemini client: cache, fallback, parse

GeminiLlm.generateJson()

packages/core/src/llm/gemini.ts · lines 71–126

```
71 async generateJson<T>(req: LlmJsonRequest, parse: (value: unknown) => T): Promise<LlmJsonResponse<T>> {
72   const temperature = req.temperature ?? 0.2;
73   const keyFor = (model: string) => this.cache.key({ model, system: req.system, messages: req.messages,
74     schema: req.schema, temperature });
75   for (const model of this.models) {
76     const hit = await this.cache.get<CachedGeneration>(keyFor(model));
77     if (hit) return { data: parse(JSON.parse(hit.raw)), ...hit, model, cached: true };
78   }
79   let lastError: unknown;
80   const available = this.models.filter((m) => !this.exhausted.has(m));
81   for (const [i, model] of available.entries()) {
82     const last = i === available.length - 1;
83     let res;
84     try {
85       res = await withRetry(
86         () =>
87         this.ai.models.generateContent({
88           model,
89           contents: req.messages.map((m) => ({ role: m.role, parts: [{ text: m.text }] })),
90           config: {
91             systemInstruction: req.system,
92             temperature,
93             responseMimeType: 'application/json',
94             responseJsonSchema: req.schema,
95           },
96         }),
97         last ? 6 : 2,
98       );
99     } catch (e) {
100       lastError = e;
101       if (isDailyQuota(e)) {
102         this.exhausted.add(model);
103         continue;
104       }
105     }
106   }
```

```

105     if (!last && isOverloaded(e)) continue;
106     throw e;
107 }
108 const raw = res.text ?? '';
109 if (!raw) throw new Error(`${model} returned no text (finish reason: ${res.candidates?.
[0]?.finishReason ?? 'unknown'})`);
110 // Parse before caching so a malformed answer is never replayed from the cache.
111 const data = parse(JSON.parse(raw));
112 const entry: CachedGeneration = {
113     raw,
114     inputTokens: res.usageMetadata?.promptTokenCount ?? 0,
115     outputTokens: res.usageMetadata?.candidatesTokenCount ?? 0,
116 };
117 await this.cache.set(keyFor(model), entry);
118 return { data, ...entry, model, cached: false };
119 }
120 if (this.exhausted.size === this.models.length) {
121     throw new Error(
122         `Gemini free-tier daily quota used up for ${this.models.join(', ')}. Quotas reset at midnight Pacific
time; enabling billing on the key removes the cap.`
123     );
124 }
125 throw lastError;
126 }

```

- **Cache first** (lines 74–77): the key covers the model, instructions, conversation, schema and temperature. Every model in the list is checked, so an answer produced by a fallback model is reused too.
- **Models in order** (lines 80–107): each model gets a short retry budget (2 attempts; 6 for the last). A daily-quota error marks the model exhausted for the rest of the process; an overload moves on to the next model.
- **Structured output** (lines 90–95): `responseMimeType: 'application/json'` plus `responseJsonSchema` make Gemini answer in the required JSON shape.
- **Parse, then cache** (lines 108–118): the reply is parsed and validated before it is written to the cache, so a bad answer is never replayed.

`withRetry()`: backoff that respects the server's requested delay

packages/core/src/llm/gemini.ts · lines 20–34

```

20 async function withRetry<T>(fn: () => Promise<T>, attempts = 5, onWait?: (ms: number) => void): Promise<T>
{
21     let delay = 2000;
22     for (let i = 1; ; i++) {
23         try {
24             return await fn();
25         } catch (e) {
26             const status = e instanceof ApiError ? e.status : undefined;
27             if (i >= attempts || status === undefined || !RETRYABLE.has(status) || isDailyQuota(e)) throw e;
28             const wait = Math.min(120_000, serverRetryDelayMs(e) ?? Math.min(60_000, delay) + Math.random() *
500);
29             onWait?.(wait);
30             await new Promise((r) => setTimeout(r, wait));
31             delay *= 2;
32         }
33     }
34 }

```

22.3 One schema, used twice

The answer contract, written with zod

packages/core/src/llm/schema.ts · lines 7–24

```
7 export const LlmCandidate = z.object({
8   kind: z.enum(['index', 'rewrite', 'statistics', 'config']).describe('Type of change'),
9   title: z.string().describe('Short label, e.g. "B-tree on orders (customer_id, created_at)"'),
10  statements: z
11    .array(z.string())
12    .describe(
13      'index: CREATE INDEX CONCURRENTLY statements. statistics: CREATE STATISTICS statement. config: SET
14      statement. rewrite: optional supporting CREATE INDEX statements (may be empty).',
15    ),
16  rewritten_sql: z.string().describe('For kind=rewrite: the complete rewritten query. Empty string for
17  other kinds. '),
18  rationale: z.string().describe('Why this should help, referring to the plan and statistics'),
19  expected_plan_change: z.string().describe('How the plan should change, e.g. "Seq Scan on orders -> Index
20  Scan using qopt_..."'),
21  citations: z.array(z.string()).describe('Ids of documentation excerpts relied on, e.g. ["D1"]'),
22 });
23
24 export const LlmSuggestions = z.object({
25   diagnosis: z.string().describe('Root cause of the slowness in one paragraph'),
26   candidates: z.array(LlmCandidate).min(1).max(4),
27 });
```

`z.toJSONSchema(LlmSuggestions)` produces the JSON Schema sent to Gemini (with the `$schema` key removed, which the API does not accept). `LlmSuggestions.parse()` then checks the reply against the same definition. The `.describe()` texts end up in the schema as descriptions, so they double as instructions to the model.

22.4 The conversation

Suggester: one conversation per analysis

packages/core/src/llm/suggest.ts · lines 44–63

```
44 private async call(round: number): Promise<{ diagnosis: string; candidates: Candidate[] }> {
45   const res = await this.llm.generateJson({ system: this.system, messages: this.messages, schema:
46     this.schema }, parseSuggestions);
47   this.usage.calls++;
48   this.usage.model = res.model;
49   if (res.cached) this.usage.cachedCalls++;
50   this.usage.inputTokens += res.inputTokens;
51   this.usage.outputTokens += res.outputTokens;
52   this.messages.push({ role: 'model', text: res.raw });
53   return { diagnosis: res.data.diagnosis, candidates: this.toCandidates(res.data, round) };
54 }
55
56 async initial(): Promise<{ diagnosis: string; candidates: Candidate[] }> {
57   this.messages.push({ role: 'user', text: userPrompt(this.input) });
58   return this.call(0);
59 }
60
61 async revise(results: ValidatedCandidate[], round: number): Promise<{ diagnosis: string; candidates:
62   Candidate[] }> {
63   this.messages.push({ role: 'user', text: feedbackPrompt(results) });
64   return this.call(round);
65 }
```

The `Suggester` keeps the message history. `initial()` sends the evidence; `revise()` appends the verdicts as a new user message, so Gemini sees its own earlier answer and why it failed. Each candidate is tagged with its `round`, which the UI shows as “Gemini · revision 1”. Index statements are normalized to `CREATE INDEX CONCURRENTLY`, the form that is safe to run in production.

The RAG code

23.1 Chunking a manual page

chunkDocPage(): walk the page's blocks in order

packages/core/src/rag/chunk.ts · lines 54-86

```

54 root.find(BLOCKS).each((_, el) => {
55   const $el = $(el);
56   // Leaf blocks only: skip anything nested inside another block we already take whole.
57   if ($el.parents(BLOCKS).length) return;
58   const tag = el.tagName.toLowerCase();
59   if (/^h[1-4]$/.test(tag)) {
60     flush();
61     heading = clean($el.text());
62     anchor = $el.closest('[id]').attr('id') ?? anchor;
63     return;
64   }
65   let text: string;
66   if (tag === 'pre') text = $el.text().trim();
67   else if (tag === 'table') {
68     // Long reference tables (operator class listings, catalog columns) drown out prose in retrieval.
69     if ($el.find('tr').length > 15) return;
70     text = $el
71       .find('tr')
72       .map((_, tr) =>
73         $(tr)
74           .find('th,td')
75           .map((_, c) => clean($(c).text()))
76           .get()
77           .join(' | '),
78       )
79       .get()
80       .join('\n');
81   } else if (tag === 'dt') text = `• ${clean($el.text())}`;
82   else text = clean($el.text());
83   if (!text) return;
84   if (buf.join('\n').length + text.length > TARGET && buf.length) flush();
85   buf.push(text);
86 });

```

The page is reduced to its main content (`#docContent`), minus navigation. The code then visits “leaf” blocks in document order: paragraphs, code, tables, list items, definition terms. A heading closes the current chunk and starts a new section, whose anchor comes from the nearest element with an `id`. Blocks accumulate until about 1,400 characters, and oversized text is split at sentence boundaries.

23.2 Embedding and storing

ingestDocs(): embed all chunks, then replace this version's rows

packages/core/src/rag/ingest.ts · lines 75-95

```
75 let vectors: number[][] | undefined;
76 if (opts.embed && embedder) {
77   log(`embedding ${rows.length} chunks with ${embedder.model} (${embedder.dims} dims)...`);
78   embedder.onWait = (msg) => log(` ${msg}`);
79   vectors = await embedder.embed(
80     rows.map((r) => `${r.title} > ${r.heading}\n${r.content}`),
81     'document',
82   );
83 }
84
85 await meta.tx(async (s) => {
86   await s.query('DELETE FROM doc_chunks WHERE pg_version = $1', [opts.version]);
87   for (let i = 0; i < rows.length; i++) {
88     const r = rows[i];
89     await s.query(
90       `INSERT INTO doc_chunks (pg_version, slug, url, title, heading, content, embedding)
91       VALUES ($1, $2, $3, $4, $5, $6, $7::vector)`,
92       [opts.version, r.slug, r.url, r.title, r.heading, r.content, vectors ? JSON.stringify(vectors[i]) :
93       null],
94     );
95   });
96 }
```

All chunks for a PostgreSQL version are replaced in one transaction, so re-running `docs ingest` never leaves a half-updated corpus. Vectors are passed as JSON arrays and cast with `$7::vector`, the text form `pgvector` accepts.

23.3 Hybrid search with RRF

Keyword search: OR the words of the phrase together

packages/core/src/rag/retrieve.ts · lines 39-51

```
39 private async keyword(query: string, version: string, limit: number): Promise<Row[]> {
40   // OR the query's lexemes together: long queries would otherwise match nothing.
41   const res = await this.meta.query(
42     `WITH q AS (
43       SELECT to_tsquery('english', array_to_string(tsvector_to_array(to_tsvector('english', $1)), ' | '))
44     AS q
45     )
46     SELECT id, url, title, heading, content FROM doc_chunks, q
47     WHERE pg_version = $2 AND tsv @@ q.q
48     ORDER BY ts_rank_cd(tsv, q.q, 1) DESC LIMIT $3`,
49     [query, version, limit],
50   );
51   return res.rows;
52 }
```

```
65 async search(queries: string[], version: string, k: number): Promise<DocChunk[]> {
66   const fused = new Map<number, { row: Row; score: number }>();
67   const addList = (rows: Row[], weight = 1) => {
68     rows.forEach((row, rank) => {
69       const cur = fused.get(row.id) ?? { row, score: 0 };
70       cur.score += weight / (RRF_K + rank + 1);
71       fused.set(row.id, cur);
72     });
73   };
74   for (const q of queries) {
75     addList(await this.keyword(q, version, 20));
76     try {
77       addList(await this.vector(q, version, 20));
78     } catch {
79       // Embedding API unavailable: keyword results still stand.
80     }
81   }
82   // At most two chunks per section keeps the context diverse.
83   const perSection = new Map<string, number>();
84   return [...fused.values()]
85     .sort((a, b) => b.score - a.score)
86     .filter(({ row }) => {
87       const n = (perSection.get(row.url) ?? 0) + 1;
88       perSection.set(row.url, n);
89       return n <= 2;
90     })
91     .slice(0, k)
92     .map(({ row, score }, i) => ({ id: row.id, ref: `D${i + 1}`, url: row.url, title: row.title, heading:
row.heading, content: row.content, score: Math.round(score * 10000) / 10000 }));
93 }
```

If the embedding call fails (no key, or a quota error), the vector list is simply missing and keyword results still stand. That is why RAG works, at reduced quality, without a Gemini key.

The pipeline orchestrator

`pipeline.ts` contains `analyze()`, which runs the nine stages of [Chapter 8](#). It is long but linear. Three pieces are worth studying.

24.1 Timed, observable stages

`stage()`: progress, timing and failure recording for every step

`packages/core/src/pipeline.ts` · lines 70–85

```
70 async function stage<T>(name: string, fn: () => Promise<T>): Promise<T> {
71   const log = { name, status: 'running' as const, ms: undefined as number | undefined };
72   run.stages.push(log);
73   emit();
74   const t = performance.now();
75   try {
76     const out = await fn();
77     Object.assign(log, { status: 'done', ms: Math.round(performance.now() - t) });
78     return out;
79   } catch (e) {
80     Object.assign(log, { status: 'failed', ms: Math.round(performance.now() - t), note: errorMessage(e) });
81     throw e;
82   } finally {
83     emit();
84   }
85 }
```

Every step is wrapped in `stage()`, which appends a progress entry, times the step, records failures, and calls `emit()` so the API can publish a live snapshot. That is where the progress chips in the web UI come from.

24.2 Validating candidates one by one

`validateAll()`: de-duplicate by fingerprint, validate sequentially

`packages/core/src/pipeline.ts` · lines 195–215

```
195 const validateAll = async (cands: Candidate[]) => {
196   const fresh: ValidatedCandidate[] = [];
197   for (const c of cands) {
198     let key: string;
199     try {
200       key = statementKey(c);
201     } catch {
202       key = `${c.id}`;
203     }
204     if (seen.has(key)) continue;
205     seen.add(key);
206     const vc: ValidatedCandidate = { ...c };
207     run.candidates.push(vc);
208     fresh.push(vc);
209     if (options.validate) {
210       vc.validation = await validateCandidate(c, vctx);
211       emit();
212     }
213   }
214   return fresh;
215 };
```

The dedupe key is the libpg_query **fingerprint** of each statement, which ignores constants, formatting and the `CONCURRENTLY` keyword. So when the heuristic and Gemini propose the same index with different names or spacing, it is validated once. Validation is deliberately sequential: experiments on the shadow must not overlap.

24.3 The generate → validate → revise loop

The LLM loop inside analyze()

packages/core/src/pipeline.ts · lines 239–251

```
239 run.llm = suggester.usage;
240 try {
241   const first = await stage('llm: suggest', () => suggester.initial());
242   run.diagnosis = first.diagnosis;
243   let latest = await stage('llm: validate ${first.candidates.length}', () =>
    validateAll(first.candidates));
244   for (let round = 1; round <= options.retries && options.validate; round++) {
245     if (latest.some((c) => c.validation?.verdict === 'accepted') || !latest.length) break;
246     const next = await stage('llm: revise (round ${round})', () => suggester.revise(latest, round));
247     latest = await stage('llm: validate ${next.candidates.length}', () => validateAll(next.candidates));
248   }
249 } catch (e) {
250   run.warnings.push(`LLM step failed: ${errorMessage(e)}`);
251 }
```

Only accepted candidates are recommended, best score first

packages/core/src/pipeline.ts · lines 255–258

```
255 // 8. Recommendations: validated winners, best first. Without validation, everything is "unverified".
256 run.recommendations = options.validate
257   ? run.candidates.filter((c) => c.validation?.verdict === 'accepted').sort((a, b) => (b.validation?.score
    ?? 0) - (a.validation?.score ?? 0))
258   : [...run.candidates];
```

If Gemini fails entirely (network, quota), the failure becomes a warning, and the heuristic candidates and everything else in the run are kept. A run only fails as a whole when the input cannot be analysed, for example when the SQL does not parse.

Demo workload and evaluation harness

25.1 Deterministic data

Seeding starts by fixing PostgreSQL's random generator

packages/core/src/workload/seed.ts · lines 69–72

```

69 'SELECT setseed(0.4242)',
70 'DROP TABLE IF EXISTS reviews, events, order_items, orders, products, customers CASCADE',
71 `CREATE OR REPLACE FUNCTION pg_temp.pick(cum float8[], r float8) RETURNS int LANGUAGE sql IMMUTABLE AS
72   $$ SELECT min(i) FROM generate_subscripts(cum, 1) i WHERE r < cum[i] $$`,

```

Orders: generated in time order, with realistic statuses

packages/core/src/workload/seed.ts · lines 133–148

```

133 // Orders are appended in time order (created_at correlates with physical order), like a real table.
134 `INSERT INTO orders
135   SELECT g,
136     1 + floor(random() * ${n.customers}::bigint),
137     CASE WHEN ts > timestamptz '${ANCHOR}' - interval '3 days'
138         THEN (${sqlArray(['pending', 'pending', 'shipped', 'shipped', 'delivered'])}[1 + floor(r *
139   5)::int])
140         WHEN r < 0.001 THEN 'pending'
141         WHEN r < 0.93 THEN 'delivered'
142         WHEN r < 0.98 THEN 'cancelled'
143         ELSE 'returned' END,
144     ts,
145     round((10 + random() * 490)::numeric, 2),
146     (${sqlArray([...new Set(GEO.map((x) => x[0]))]}[1 + floor(random() * 8)::int])
147   FROM (SELECT g, random() AS r,
148     timestamptz '${ANCHOR}' - ((1 - g::float8 / ${n.orders}) * interval '730 days') + (random() *
149   interval '10 minutes') AS ts
150   FROM generate_series(1, ${n.orders}) g) s`,

```

All data is generated inside PostgreSQL with `generate_series` and `random()`. Because `setseed()` fixes the random sequence and the statements run in one session, target and shadow receive identical data; `qopt db setup` compares a checksum of both. The distributions are chosen to create specific problems: orders appended in time order (so BRIN works), a small share of “pending” orders concentrated in the last three days (for the partial index), 0.4% of products tagged “limited” (GIN), and Auckland always in NZ (the correlated-columns trap).

25.2 Queries with known answers

A workload query: SQL generator plus its ground truth

packages/core/src/workload/queries.ts · lines 145–154

```

145 id: 'W13',
146 title: 'Top products by distinct orders (sort spill)',
147 expected: [
148   { kind: 'index', table: 'order_items', column: 'product_id' },
149   { kind: 'config', setting: 'work_mem' },
150   { kind: 'rewrite' },
151 ],
152 expectedLabel: 'btree on order_items (product_id, order_id) for a sort-free index-only scan',
153 sql: () => `SELECT product_id, count(DISTINCT order_id) AS orders FROM order_items GROUP BY product_id
154   ORDER BY orders DESC LIMIT 10`,

```

Each of the 15 queries carries one or more `expected` fixes. Any of them counts as a hit. Parameters come from a seeded random generator (`rng(seed)`, the small mulberry32 algorithm), so every evaluation run uses identical values.

25.3 Scoring a configuration

`topPicks()`: what a user would actually apply

`packages/core/src/eval/runner.ts` · lines 53–57

```
53 export function topPicks(ranked: ValidatedCandidate[]): ValidatedCandidate[] {
54   const byKind = new Map<string, ValidatedCandidate>();
55   for (const c of ranked) if (!byKind.has(c.kind)) byKind.set(c.kind, c);
56   return [...byKind.values()];
57 }
```

A hit must match the known fix and be correct

`packages/core/src/eval/runner.ts` · lines 216–219

```
216 // A change that errors or returns different rows fixes nothing, whatever its shape.
217 hit: recommended.some(
218   (c) => c.validation?.verdict !== 'error' && c.validation?.equivalence?.equal !== false &&
     w.expected.some((e) => matchesExpectation(c, e)),
219 ),
```

The harness analyses each query under each configuration, then applies every configuration's top picks together on the shadow in **one** transaction: it times all 15 queries before, applies the DDL, times them again, and rolls back. Measuring everything together is what revealed interactions, such as W13's index also speeding up W12. The "LLM, no validation" configuration shares its analysis runs with "LLM + validation" (same options, same Gemini answers) and simply recommends every first-round candidate, so the comparison isolates the value of validation.

The applications: CLI, API, web UI

26.1 The CLI

gopt analyze: turn options into one or more analyze() calls

apps/cli/src/index.ts · lines 153–187

```

153 .command('analyze')
154 .description('diagnose a query and propose validated fixes')
155 .argument('[sql]', 'SQL to analyse')
156 .option('-f, --file <path>', 'read SQL from a file')
157 .option('--queryid <id>', 'analyse a captured pg_stat_statements entry')
158 .option('--workload <id>', 'analyse a demo workload query, e.g. W09')
159 .option('--top <n>', 'analyse the N slowest captured statements', (v) => parseInt(v, 10))
160 .addOption(engineOption())
161 .option('--no-rag', 'skip documentation retrieval')
162 .option('--no-validate', 'skip validation (candidates are reported as unverified)')
163 .option('--retries <n>', 'LLM revise rounds after rejections', (v) => parseInt(v, 10),
    DEFAULT_OPTIONS.retries)
164 .option('--plan', 'print the baseline plan')
165 .option('--json', 'print the full result as JSON')
166 .action(async (sqlArg: string | undefined, o) => {
167     const services = getServices();
168     const options = { engine: o.engine as Engine, rag: o.rag, validate: o.validate, retries: o.retries };
169     const inputs: { sql?: string; queryid?: string; label?: string }[] = [];
170     if (o.top) {
171         const { queries } = await captureSlowQueries(services.dbs.target, o.top);
172         inputs.push(...queries.map((q) => ({ queryid: q.queryid })));
173     } else if (o.queryid) inputs.push({ queryid: o.queryid });
174     else if (o.workload) {
175         const w = workloadQuery(o.workload);
176         if (!w) throw new Error(`unknown workload query ${o.workload}`);
177         inputs.push({ sql: w.sql(rng(1), 1), label: w.id });
178     } else if (o.file) inputs.push({ sql: await readFile(o.file, 'utf8') });
179     else if (sqlArg) inputs.push({ sql: sqlArg });
180     else throw new Error('give SQL, --file, --queryid, --workload or --top');
181
182     for (const input of inputs) {
183         const run = await analyze({ ...input, options }, services);
184         if (o.json) console.log(JSON.stringify(run, null, 2));
185         else printRun(run, !!o.plan);
186     }
187 });

```

Every command follows the same pattern: parse options with commander, call one core function, and print a human-friendly summary (or JSON with `--json`). Colours are used only when writing to a terminal. The program always closes the database pools on exit, even after an error.

26.2 The API: one experiment at a time

exclusive(): a one-line queue

apps/api/src/server.ts · lines 33–40

```

33 // Validation times queries on the shadow, so experiments run one at a time: concurrent runs would
34 // measure each other.
35 let queue: Promise<unknown> = Promise.resolve();
36 function exclusive<T>(fn: () => Promise<T>): Promise<T> {
37     const next = queue.then(fn, fn);
38     queue = next.catch(() => {});
39     return next;
40 }

```

Every analysis, workload run and evaluation goes through `exclusive()`, a queue built from one promise chain: each job starts only when the previous one has finished. Two analyses running at once would slow each other down on the shadow and corrupt each other's timings.

```

POST /api/analyze: respond immediately, work in the background
apps/api/src/server.ts · lines 100-131

100 app.post<{ Body: { sql?: string; queryid?: string; label?: string; options?: Partial<AnalyzeOptions> } }>
    ('/api/analyze', async (req, reply) => {
101   const { sql, queryid, label } = req.body ?? {};
102   if (!sql?.trim() && !queryid) return reply.code(400).send({ error: 'sql or queryid is required' });
103   const options: AnalyzeOptions = { ...DEFAULT_OPTIONS, ...req.body.options };
104   const id = randomUUID();
105   const placeholder: AnalysisResult = {
106     id,
107     createdAt: new Date().toISOString(),
108     status: 'running',
109     sql: sql ?? '',
110     source: { type: queryid ? 'captured' : 'adhoc', queryid, label },
111     options,
112     stages: [{ name: 'queued', status: 'running' }],
113     findings: [],
114     context: [],
115     docs: [],
116     candidates: [],
117     recommendations: [],
118     warnings: [],
119   };
120   live.set(id, placeholder);
121   void exclusive(() =>
122     analyze({ sql, queryid, label, options, runId: id, onUpdate: (run) => live.set(id, run) },
    services).then((run) => {
123     live.set(id, run);
124     // Keep finished runs briefly in memory; the store has them afterwards.
125     setTimeout(() => live.delete(id, 10 * 60_000).unref());
126   }),
127   ).catch((e) => {
128     live.set(id, { ...placeholder, status: 'failed', error: errorMessage(e) });
129   });
130   return reply.code(202).send({ id });
131 });

```

The request returns `202 Accepted` with a run id at once. The work continues in the background, and the pipeline's `onUpdate` callback keeps the latest snapshot in a map that `GET /api/runs/:id` serves. Finished runs are also saved in the meta database, so they outlive the process.

26.3 The web UI

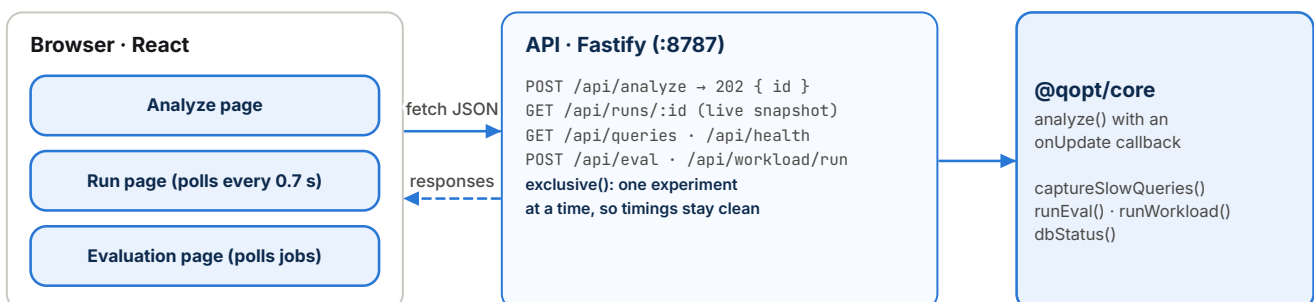


Figure 25. The browser polls the API while a run is in progress; the API serves live snapshots from memory and finished runs from the meta database.

```
22 useEffect(() => {
23   let stop = false;
24   let timer: ReturnType<typeof setTimeout>;
25   const poll = async () => {
26     try {
27       const r = await api.run(id);
28       if (stop) return;
29       setRun(r);
30       if (r.status === 'running') timer = setTimeout(poll, 700);
31     } catch (e) {
32       if (!stop) setError(e instanceof Error ? e.message : String(e));
33     }
34   };
35   void poll();
36   return () => {
37     stop = true;
38     clearTimeout(timer);
39   };
40 }, [id]);
```

The UI is three pages under hash routes (`#/` , `#/runs/:id` , `#/eval`) with no router library. It imports its types from `@qopt/core/types` , which erases to nothing at build time, so the browser bundle contains no database code. The colours follow a small set of design tokens with light and dark themes, and verdicts always pair a colour with an icon and a word.

Tests

43 tests in two groups: 34 unit tests that need no database, and 9 integration tests that run against the local target and shadow.

| File | Tests | What they prove |
|-----------------------|-------|---|
| sql.test.ts | 13 | query shape extraction (aliases, joins, OR, DISTINCT aggregates, anti-patterns); every safety rule |
| plan.test.ts | 7 | plan normalization on real plans; misestimate origin and LIMIT rules; spill and scan findings |
| heuristic.test.ts | 7 | column order, partial/GIN/trigram/BitmapOr rules, existing-index skipping, statistics and work_mem |
| misc.test.ts | 7 | auto_explain parsing (exact 64-bit id), LLM schema, chunking, ground-truth matching |
| validator.int.test.ts | 7 | HypoPG + shadow acceptance with no leftovers; ignored index; hallucinated column; wrong and right rewrites; blocked writes; extended statistics |
| llm-loop.int.test.ts | 2 | the full feedback loop with a scripted model |

27.1 Testing the LLM loop without an LLM

ScriptedLlm: a fake model that returns prepared answers

packages/core/test/llm-loop.int.test.ts · Lines 11-22

```
11 class ScriptedLlm implements LlmProvider {
12   readonly name = 'scripted';
13   readonly model = 'scripted-test-model';
14   readonly requests: LlmJsonRequest[] = [];
15   constructor(private readonly turns: LlmSuggestionsT[]) {}
16   async generateJson<T>(req: LlmJsonRequest, parse: (v: unknown) => T): Promise<LlmJsonResponse<T>> {
17     this.requests.push(structuredClone(req));
18     const turn = this.turns[this.requests.length - 1];
19     if (!turn) throw new Error('script exhausted');
20     return { data: parse(turn), raw: JSON.stringify(turn), inputTokens: 100, outputTokens: 50, model:
this.model, cached: false };
21   }
22 }
```

The first scripted answer contains three bad candidates: an index on a column that does not exist, an index the planner will not use, and a `DROP INDEX` disguised as an index. The test asserts that each gets the right verdict, that the second prompt contains the reasons, and that only the revised candidate is recommended. This tests the real validator on real databases, with fully predictable “model” output.

27.2 Safety rules as tests

Each dangerous pattern has a test

packages/core/test/sql.test.ts · lines 87-96

```
87 it('rejects UNIQUE indexes, data-modifying rewrites and dangerous functions', () => {
88   expect(checkCandidate({ kind: 'index', statements: ['CREATE UNIQUE INDEX i ON t (a)'] })).toMatchObject({
89     ok: false, errorClass: 'unsafe' });
89   expect(checkCandidate({ kind: 'rewrite', statements: [], rewrittenSql: 'WITH d AS (DELETE FROM t
RETURNING *) SELECT * FROM d' })).toMatchObject({
90     ok: false,
91     errorClass: 'unsafe',
92   });
93   expect(checkCandidate({ kind: 'rewrite', statements: [], rewrittenSql: 'SELECT pg_sleep(10)'
94 })).toMatchObject({ ok: false });
94   expect(checkCandidate({ kind: 'rewrite', statements: [], rewrittenSql: 'SELECT * INTO x FROM t'
95 })).toMatchObject({ ok: false });
95   expect(checkCandidate({ kind: 'rewrite', statements: [], rewrittenSql: 'SELECT * FROM t FOR UPDATE'
96 })).toMatchObject({ ok: false });
96 });
```

An experiment must leave the shadow exactly as it was

packages/core/test/validator.int.test.ts · lines 56-65

```
56 it('accepts a useful index via HypoPG + shadow timing and leaves no trace', async ({ skip }) => {
57   if (!available) skip();
58   const before = await indexCount();
59   const ctx = await context('SELECT id FROM orders WHERE customer_id = 4242');
60   const v = await validateCandidate(cand({ statements: ['CREATE INDEX CONCURRENTLY qopt_t_idx ON orders
(customer_id)'] }), ctx);
61   expect(v).toMatchObject({ verdict: 'accepted', method: 'hypopg+shadow', usesCandidate: true });
62   expect(v.timeMs!.change).toBeLessThan(-0.5);
63   expect(v.indexBytes).toBeGreaterThan(0);
64   expect(await indexCount()).toBe(before);
65 });
```

PART V

Using it, and what it achieved

How to install and run qopt, three worked examples with real output, the evaluation results with an honest reading of them, and the project's limitations.

- Chapter 28 · Setup and daily use
- Chapter 29 · Worked examples
- Chapter 30 · Evaluation results
- Chapter 31 · Limitations and future work

Setup and daily use

28.1 Requirements

- Node.js 22 or newer, and npm.
- PostgreSQL 17 with HypoPG and pgvector: either Homebrew on macOS (the setup used to build this project) or Docker.
- Optionally, a Gemini API key from Google AI Studio. Without one, the rule-based advisor, validation and keyword doc search all still work.

28.2 Option A: Homebrew (no Docker)

Install PostgreSQL 17 and build the two extensions against it

```
brew install postgresql@17
PGC=$(brew --prefix postgresql@17)/bin/pg_config
git clone --depth 1 --branch 1.4.3 https://github.com/HypoPG/hypopg.git
(cd hypopg && make PG_CONFIG=$PGC install)
git clone --depth 1 --branch v0.8.6 https://github.com/pgvector/pgvector.git
(cd pgvector && make PG_CONFIG=$PGC install)
```

Create and start the three local servers (data lives in `./pgdata`)

```
npm install
npm run db:local -- init      # initdb x3, config, start, create the qopt databases
npm run db:local -- status    # later: start | stop | status | destroy
```

`scripts/local-db.sh` creates three clusters on ports 5433–5435, appends qopt’s settings to each `postgresql.conf` (for the target: `pg_stat_statements`, `auto_explain` with a 5 ms threshold and JSON output, and the JSON log), and starts them without installing a background service.

28.3 Option B: Docker

The same three servers as containers

```
npm install
npm run db:up      # builds postgres:17 + hypopg + pgvector and starts target, shadow, meta
```

28.4 First run

From an empty setup to the web UI

```
cp .env.example .env      # then add GEMINI_API_KEY=... (optional)
npm run qopt -- db setup   # extensions, demo data (~15 s per database), meta tables
npm run qopt -- workload   # play the workload so there is something to capture
npm run qopt -- docs ingest # fetch, chunk and embed the manual (~ 3 min on the free tier)
npm run dev                # API on :8787 + web UI on http://localhost:5173
```

```
npm run qopt -- db status
```

```
target up postgres://postgres:***@localhost:5433/qopt 17.11 (Homebrew)
        extensions: hypopg, pg_stat_statements, pg_trgm, plpgsql · 6 tables
shadow up postgres://postgres:***@localhost:5434/qopt 17.11 (Homebrew)
        extensions: hypopg, pg_trgm, plpgsql · 6 tables
meta   up postgres://postgres:***@localhost:5435/qopt 17.11 (Homebrew)
        extensions: plpgsql, vector · 2 tables
gemini configured (gemini-3.8-flash)
```

28.5 Everyday commands

The commands you will use most

```
npm run qopt -- capture                # the slowest statements
npm run qopt -- analyze --top 3        # analyse the 3 worst
npm run qopt -- analyze --queryid 669985620531413121 # one captured statement
npm run qopt -- analyze "SELECT ... " --plan # ad-hoc SQL, print the plan
npm run qopt -- analyze --workload W13 --engine llm # a demo query, Gemini only
npm run qopt -- eval --model gemini-3.5-flash-lite # the evaluation, one pinned model
```

28.6 Troubleshooting

| Symptom | Cause | Fix |
|--|---|--|
| "postmaster became multithreaded during startup" | macOS needs a valid locale for PostgreSQL | the script sets <code>LC_ALL=en_US.UTF-8</code> ; do the same if you run <code>pg_ctl</code> by hand |
| web UI shows 500 errors, API logs <code>ECONNREFUSED ...:5435</code> | the databases are stopped | <code>npm run db:local -- start</code> |
| Gemini <code>503 ... high demand</code> | the model is overloaded | automatic fallback to the next model; nothing to do |
| Gemini <code>429 ... PerDay</code> | free-tier daily quota spent for that model | fallbacks take over; wait for the reset (midnight Pacific time) or enable billing |
| captured query says "no auto_explain sample" | no single run crossed the threshold | still analysed with a generic plan; lower <code>auto_explain.log_min_duration</code> if you need timings |
| <code>db status</code> warns "pg_stat_statements not preloaded" | missing <code>shared_preload_libraries</code> | add it to the target's configuration and restart |
| docs ingest pauses ("rate limited; waiting 52s") | free tier allows 100 embeddings per minute | expected; finished batches are cached |

28.7 Using it on your own database

1. Set `TARGET_URL` to your database, and `SHADOW_URL` to a restored copy (for a small database, `qopt shadow sync` copies it with `pg_dump | psql`).
2. On the target: `shared_preload_libraries = 'pg_stat_statements,auto_explain'`, `compute_query_id = on`, `auto_explain.log_format = json`, `log_destination = jsonlog`, a realistic `auto_explain.log_min_duration` (100 ms or more), and `CREATE EXTENSION hypopg`.
3. Reading logs through SQL requires superuser, or the `pg_read_server_files` and `pg_monitor` roles.
4. Set `QOPT_REDACT_VALUES=true` so real data values are not sent to Gemini.

Worked examples

29.1 In the terminal: from capture to recommendation

`capture` lists the statements with the most total time, and shows whether a real plan sample exists for each:

```
npm run qopt -- capture --limit 8
```

| queryid | calls | total ms | mean ms | sample | query |
|----------------------|-------|----------|---------|--------|--|
| -1133799339758276427 | 10 | 7028 | 702.76 | yes | SELECT product_id, count(DISTINCT order_id) AS orders FROM ord... |
| 8039014334451942946 | 10 | 948 | 94.77 | yes | SELECT id, product_id, rating FROM reviews WHERE body ILIKE \$1 |
| 3692209685478437079 | 10 | 563 | 56.25 | yes | SELECT p.category, sum(oi.quantity * oi.unit_price) AS revenue... |
| -417400099287612900 | 10 | 503 | 50.27 | yes | SELECT id, created_at, total FROM orders ORDER BY created_at D... |
| -1914024804467255093 | 10 | 344 | 34.44 | yes | SELECT type, count(*) FROM events WHERE created_at >= \$1 AND c... |
| -5588324482353006980 | 10 | 324 | 32.44 | yes | SELECT product_id, quantity, unit_price FROM order_items WHERE... |
| -156599700804929192 | 10 | 277 | 27.71 | yes | SELECT c.id, count(o.id) AS orders FROM customers c JOIN order... |
| 2343525725688456254 | 10 | 201 | 20.08 | yes | SELECT count(*), sum(total) FROM orders WHERE date(created_at)... |

Analysing W04 (recent pending orders) with the rule-based advisor shows the whole pipeline in one screen: the measured baseline, the findings, the real plan (note the 127× misestimate on the scan), three candidates each validated by HypoPG and then on the shadow, and the ranking.

```
npm run qopt -- analyze --workload W04 --engine heuristic --plan
```

```
Query SELECT id, customer_id, total FROM orders WHERE status = 'pending' AND created_at >= '2026-06-28' ORDER BY cr...
Baseline 16.40 ms (median of 5) · EXPLAIN ANALYZE on shadow
Findings
  HIGH Selective filter on orders evaluated by a full scan (node 3)
  HIGH Row estimate off by 127x at Seq Scan on orders (node 3)
  LOW 100% of time in Seq Scan on orders (node 3)
Plan
  [1] Sort cost=10,934 est_rows=4 actual_rows=758 loops=1 self=0.18ms total=16.36ms misestimate=190x sort_key=created_at
  sort=quicksort (Memory 60kB)
  [2] Gather cost=10,934 est_rows=4 actual_rows=758 loops=1 self=0ms total=16.19ms misestimate=190x
  [3] Seq Scan on orders cost=9,934 est_rows=2 actual_rows=253 loops=3 self=33.86ms total=33.86ms misestimate=127x
  filter=((created_at >= '2026-06-28 00:00:00+05:30'::timestamp with time zone) AND (status = 'pending'::text))
  removed_by_filter=599,241
  planning=0.05ms execution=16.40ms
Candidates (3)
  ACCEPTED [heuristic] index B-tree on orders (status, created_at) -99.4%
    CREATE INDEX CONCURRENTLY qopt_orders_status_created_at_idx ON public.orders (status, created_at)
    hypopg+shadow: HypoPG cost -99.8%; shadow: measured 16.40 → 0.10 ms (-99.4%, median of 5)
  ACCEPTED [heuristic] index B-tree on orders (created_at) WHERE status = 'pending' -99.5%
    CREATE INDEX CONCURRENTLY qopt_orders_created_at_partial_idx ON public.orders (created_at) WHERE status = 'pending'
    hypopg+shadow: HypoPG cost -99.9%; shadow: measured 16.40 → 0.08 ms (-99.5%, median of 5)
  ACCEPTED [heuristic] index BRIN on orders (created_at) -97.8%
    CREATE INDEX CONCURRENTLY qopt_orders_created_at_brin_idx ON public.orders USING brin (created_at)
    hypopg+shadow: HypoPG cost -41.5%; shadow: measured 16.40 → 0.35 ms (-97.8%, median of 5)
Recommendations (3)
  1. B-tree on orders (created_at) WHERE status = 'pending' -99.5%
  2. BRIN on orders (created_at) -97.8%
  3. B-tree on orders (status, created_at) -99.4%
stages: connect 16ms · baseline 220ms · context 17ms · heuristic: validate 3 543ms
```

29.2 In the browser: the Analyze page

qopt query advisor

Analyze

Evaluation

targetshadowmetagemini-3.8-flash459 chunks

Slow queries

Demo workload

7,028 ms total 10 calls 703 ms mean ● sample

SELECT product_id, count(DISTINCT order_id) AS

orders FROM order_items GROUP BY product_id...

948 ms total 10 calls 94.8 ms mean ● sample

SELECT id, product_id, rating FROM reviews

WHERE body ILIKE \$1

563 ms total 10 calls 56.3 ms mean ● sample

SELECT p.category, sum(oi.quantity *

oi.unit_price) AS revenue, count(*) AS items...

503 ms total 10 calls 50.3 ms mean ● sample

SELECT id, created_at, total FROM orders ORDER

BY created_at DESC OFFSET \$1 LIMIT \$2

344 ms total 10 calls 34.4 ms mean ● sample

SELECT type, count(*) FROM events WHERE

created_at >= \$1 AND created_at < timestamptz...

324 ms total 10 calls 32.4 ms mean ● sample

SELECT product_id, quantity, unit_price FROM

order_items WHERE order_id = \$1

277 ms total 10 calls 27.7 ms mean ● sample

SELECT c.id, count(o.id) AS orders FROM

customers c JOIN orders o ON o.customer_id =...

201 ms total 10 calls 20.1 ms mean ● sample

SELECT count(*), sum(total) FROM orders WHERE

date(created_at) = \$1

186 ms total 10 calls 18.6 ms mean ● sample

SELECT id, email FROM customers WHERE id NOT IN

(SELECT customer_id FROM orders WHERE...

175 ms total 10 calls 17.5 ms mean ● sample

SELECT id, full_name, tier FROM customers WHERE

lower(email) = lower(\$1)

116 ms total 10 calls 11.6 ms mean ● sample

SELECT id, customer_id, total FROM orders WHERE

Analyze a query

SELECT ... – or pick a captured slow query

HeuristicGeminiBoth☒ Docs (RAG)☒ Validate Revisions 1

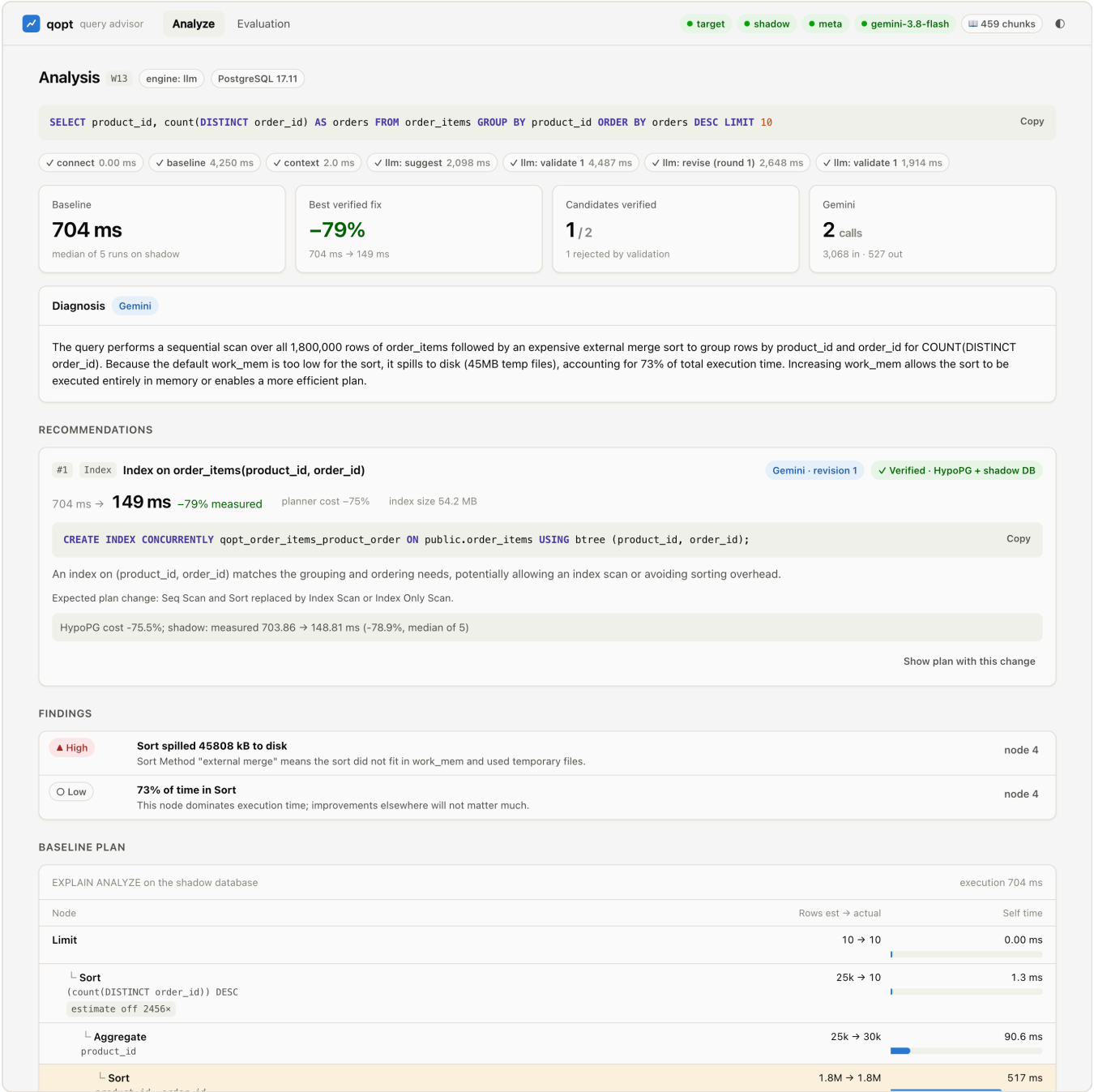
Analyze

Recent analyses

| Query | Engine | Verified fixes | Best gain | When |
|--|-----------|----------------|-----------|---------|
| SELECT id, status, total, created_at FROM orders WHERE customer_id = 62708 ORDER BY create | heuristic | 2 | –100% | 2 h ago |
| SELECT product_id, quantity, unit_price FROM order_items WHERE order_id = 599887 | llm | 2 | –100.0% | 2 h ago |
| SELECT id, status, total, created_at FROM orders WHERE customer_id = 60111 ORDER BY create | llm | 1 | –100% | 2 h ago |
| SELECT id, full_name FROM customers WHERE email = 'User15504@Example.com' OR phone = '+1-5 | heuristic | 1 | –100% | 2 h ago |
| SELECT id, created_at, total FROM orders ORDER BY created_at DESC OFFSET 49000 LIMIT 50 | heuristic | 1 | –91% | 2 h ago |
| SELECT product_id, count(DISTINCT order_id) AS orders FROM order_items GROUP BY product_id | heuristic | 1 | –79% | 2 h ago |
| SELECT p.category, sum(oi.quantity * oi.unit_price) AS revenue, count(*) AS items FROM ord | heuristic | 1 | –94% | 2 h ago |
| SELECT c.id, count(o.id) AS orders FROM customers c JOIN orders o ON o.customer_id = c.id | heuristic | 2 | –52% | 2 h ago |
| SELECT count(*), sum(total) FROM orders WHERE date(created_at) = '2026-03-30' | heuristic | 0 | — | 2 h ago |

Figure 28. The Analyze page. Left: statements captured from pg_stat_statements (“sample” means auto_explain captured a real plan), or the 15 demo queries. Right: the SQL box, engine choice (Heuristic, Gemini, or both), RAG and validation toggles, and recent analyses. The header shows the health of the three databases, the Gemini model and the number of documentation chunks.

29.3 The feedback loop at work (W13)



29.4 When validation rejects everything (W10, lite model)

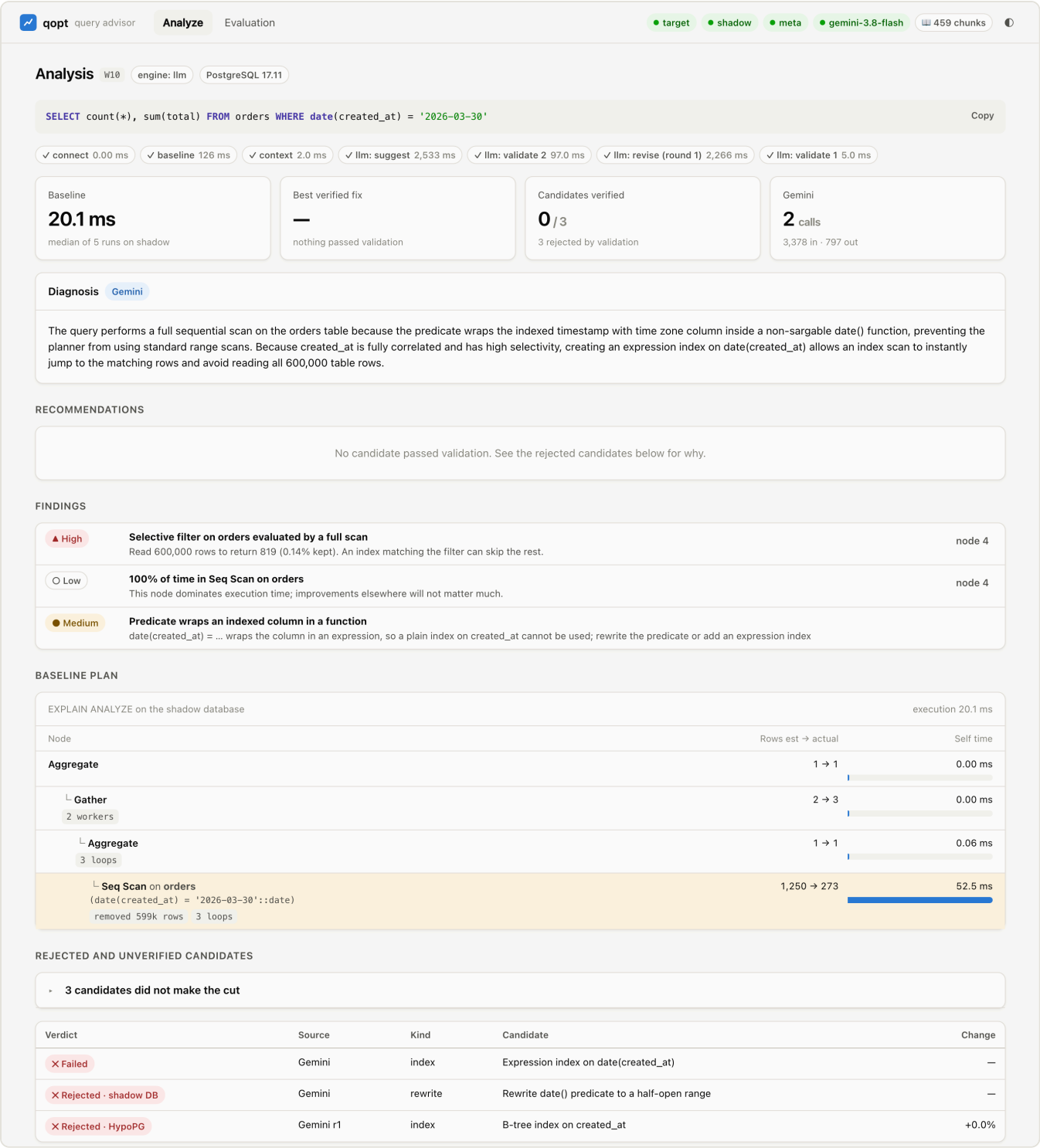


Figure 30. The lite model's W10 run: zero of three candidates survived. The expression index failed (not IMMUTABLE), the range rewrite returned different rows (time zones), and the revised plain index was ignored by the planner. The recommendation is honestly empty, instead of wrong.

Evaluation results

How good are qopt's recommendations, and what does validation add? The evaluation harness answers with measurements against known correct fixes.

30.1 Method

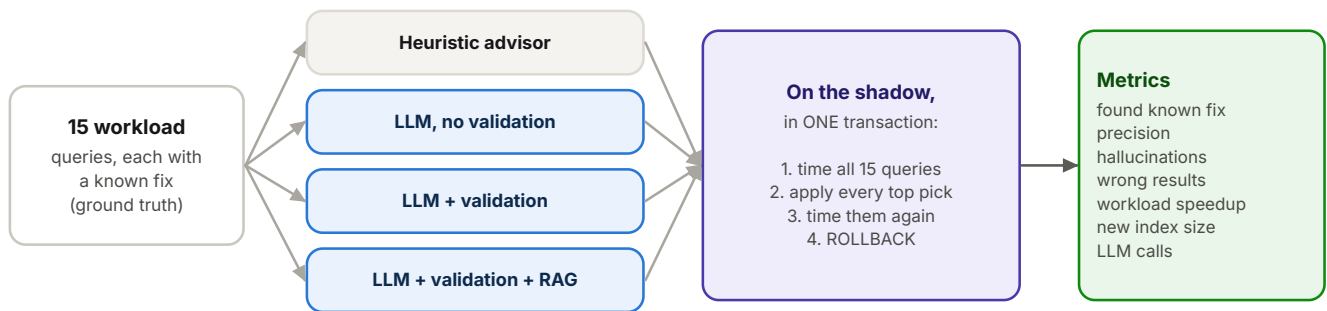


Figure 26. Four configurations analyse the same 15 queries. Each configuration's top picks are applied together on the shadow in one rolled-back transaction, and the whole workload is re-timed.

| Metric | Definition |
|------------------------|---|
| Found known fix | share of queries whose top recommendation matches the ground truth, and does not error or return different rows |
| Precision | share of all proposed candidates that passed validation |
| Hallucinated | share of candidates referencing a missing column or table, or not parsing |
| Wrong results | recommended rewrites that return different rows |
| Workload time, speedup | sum of the 15 median execution times before and after applying the recommendations together |
| New indexes | total size of the indexes the configuration would add |

Setup: PostgreSQL 17.11 on the demo data (600,000 orders), five timed runs per measurement, and `gemini-3.5-flash-lite` pinned for every LLM configuration (the full Flash models had run out of free-tier quota).

30.2 Results

| Configuration | Found known fix | Precision | Hallucinated | Wrong results | Workload time | Speedup | New indexes | LLM calls |
|---|-----------------|-----------|--------------|---------------|----------------|---------|-------------|-----------|
| Heuristic advisor (no LLM) | 87% | 77% | 0% | 0 | 1,097 → 214 ms | 5.1× | 140 MB | — |
| LLM, no validation
gemini-3.5-flash-lite | 80% | 68% | 0% | 0 | 1,088 → 844 ms | 1.3× | 172 MB | 15 |
| LLM + validation
gemini-3.5-flash-lite | 87% | 64% | 0% | 0 | 1,088 → 208 ms | 5.2× | 232 MB | 19 |
| LLM + validation + RAG
gemini-3.5-flash-lite | 87% | 70% | 0% | 0 | 1,086 → 190 ms | 5.7× | 256 MB | 18 |

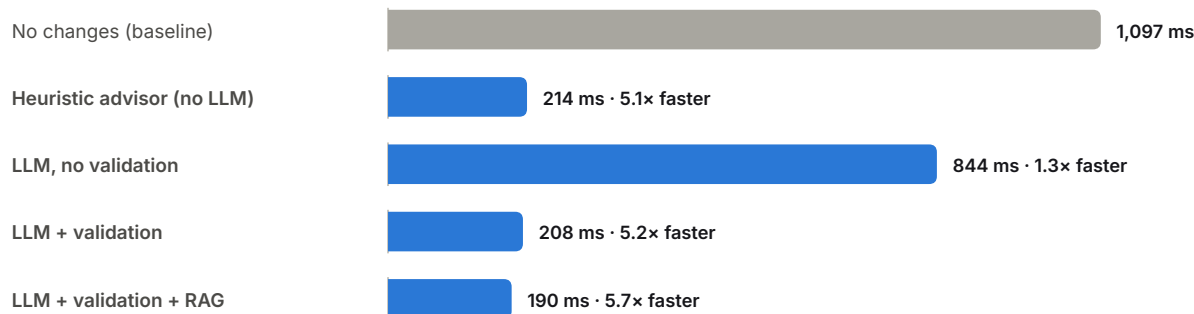


Figure 27. Total workload time after applying each configuration's recommendations. Lower is better. The grey bar is the unchanged workload.

| Query | Baseline | Heuristic | LLM, no valid. | LLM + valid. | LLM + valid. + RAG |
|--|----------|-----------|----------------|--------------|--------------------|
| W01 Customer order history | 11.0 | ✓ 0.01 | ✓ 0.01 | ✓ 0.01 | ✓ 0.01 |
| W02 Order line items | 26.6 | ✓ 0.00 | ✓ 0.00 | ✓ 0.00 | ✓ 0.00 |
| W03 Login by email (case-insensitive) | 17.4 | ✓ 0.01 | ✓ 0.01 | ✓ 0.01 | ✓ 0.01 |
| W04 Recent pending orders | 12.1 | ✓ 0.05 | ✓ 0.06 | ✓ 0.06 | ✓ 0.04 |
| W05 Limited-edition products (array containment) | 3.2 | ✓ 0.04 | ✓ 0.03 | ✓ 0.03 | ✓ 0.03 |
| W06 Products by JSON attribute | 3.0 | ✓ 0.10 | ✓ 0.22 | ✓ 0.22 | ✓ 0.21 |
| W07 Review text search (leading wildcard) | 95.2 | ✓ 0.90 | ✓ 0.89 | ✓ 0.89 | ✓ 0.90 |
| W08 Event counts in a time window | 28.1 | ✓ 0.64 | ✓ 0.20 | ✓ 0.20 | ✓ 0.17 |
| W09 Customers without recent orders (NOT IN) | 25.7 | ✗ 3.5 | ✓ 0.04 | ✓ 0.04 | ✗ 3.1 |
| W10 Daily revenue via date() on a timestamptz | 20.2 | ✗ 20.2 | ✗ 20.2 | ✗ 20.1 | ✓ 0.10 |
| W11 Orders from Auckland customers (correlated columns) | 28.0 | ✓ 28.2 | ✗ 15.0 | ✓ 26.8 | ✓ 26.9 |
| W12 Revenue for premium electronics | 56.5 | ✓ 4.6 | ✗ 54.0 | ✗ 4.0 | ✗ 4.0 |
| W13 Top products by distinct orders (sort spill) | 711 | ✓ 151 | ✓ 749 | ✓ 151 | ✓ 150 |
| W14 Deep OFFSET pagination | 52.8 | ✓ 4.6 | ✓ 4.6 | ✓ 4.7 | ✓ 4.6 |
| W15 Lookup by email OR phone | 6.2 | ✓ 0.02 | ✓ 0.02 | ✓ 0.02 | ✓ 0.02 |

30.3 What the numbers say

- **Validation is the difference between 1.3x and 5.2x.** “LLM, no validation” and “LLM + validation” are scored on the *same* first answers. Applied blindly, those answers included `SET work_mem = '64MB'` for W13, the heaviest query, which measured *slower* (707 → 749 ms). Validation rejected it, and the reasons fed back produced the index that cut W13 to 150 ms.
- **The LLM's value shows on queries that need a rewrite.** Without RAG, Gemini produced the `NOT EXISTS` rewrite for W09 (25.8 → 0.04 ms). The RAG run produced a correct time-zone-aware range rewrite for W10 (20.1 → 0.10 ms). The rule-based advisor can solve neither. But each LLM configuration solved only one of the two: model output varies between runs.
- **On the standard index cases, the small model only matches the rules,** and it proposes larger indexes (140 MB for the heuristic, versus 232–256 MB).
- **RAG's effect is within noise.** The best precision and speedup came with RAG, but the excerpts retrieved for W10 were not about time zones, so that win cannot be credited to retrieval. Telling them apart would need repeated runs.

- **No hallucinated columns** appeared on this schema. The detection is tested, but this workload did not trigger it.
- **Changes interact.** W12 got fast in the LLM rows although no W12 fix was accepted: the W13 index (`product_id, order_id`) also serves W12's join. In W11, better statistics change which join plan is chosen. This is why everything is measured together at the end.
- **The cost gate can reject real wins.** An index on `customers (country, city)` for W11 was rejected because HypoPG estimated only -14.5% cost (the gate is 20%); applied blindly, it measured 47% faster (28.1 → 15.0 ms). The HypoPG pre-filter trades some recall for speed.

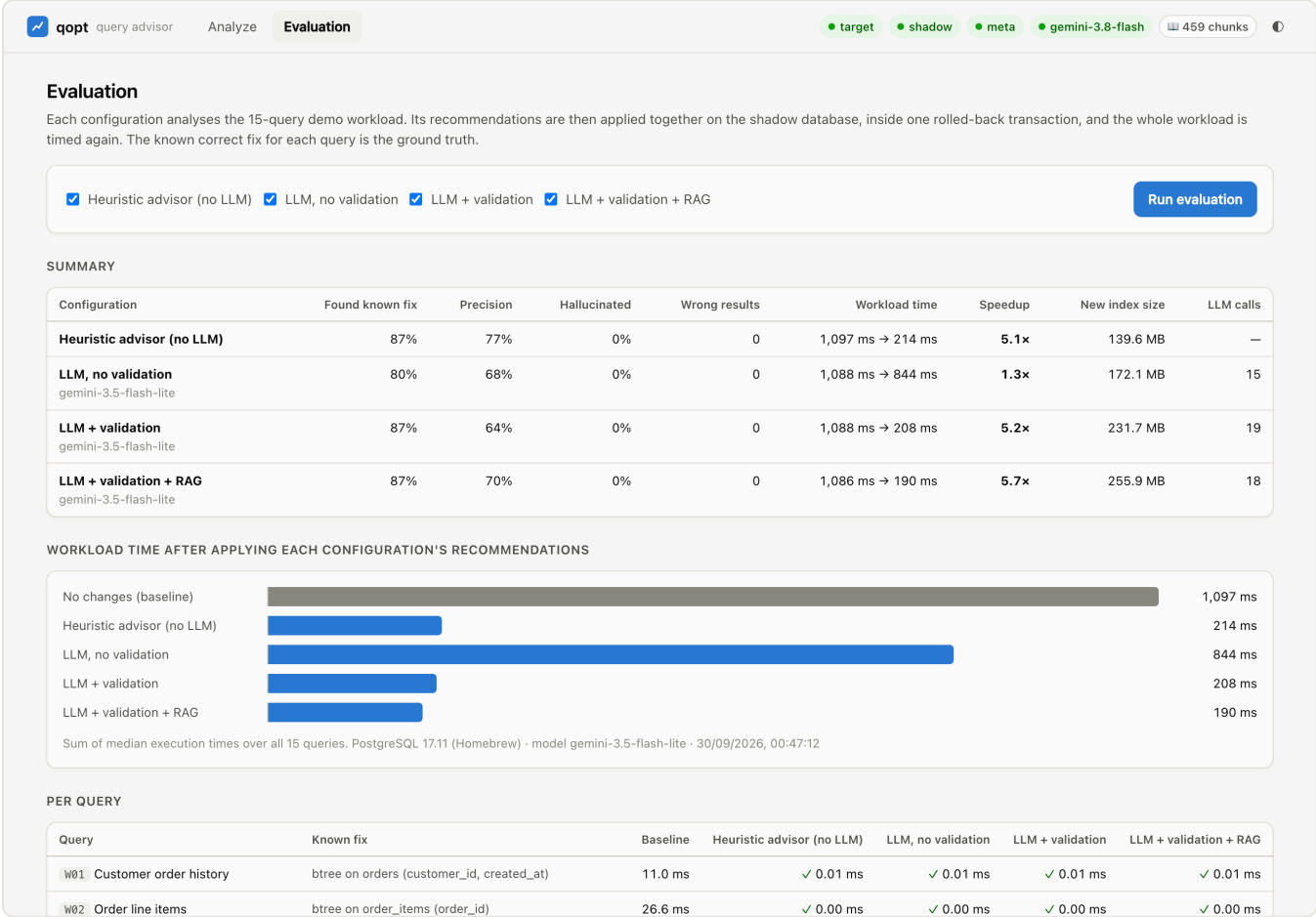


Figure 31. The Evaluation page shows the same report: summary, workload chart and per-query results. Runs started from the UI merge into the saved report.

30.4 How much to trust these numbers

- 15 queries and one run per configuration is a small sample; differences of a few percentage points are noise.
- The data is synthetic and designed to contain known problems; real workloads are messier.
- All three servers ran on one laptop, so timings include some noise; medians and thresholds absorb most of it.
- The LLM rows used a lite model because of free-tier quotas; a stronger model may do better, especially on rewrites.

Limitations and future work

31.1 Known limitations

- **Equivalence is tested, not proven.** Two queries can agree on today's data and differ tomorrow. The quality of the shadow's data bounds the strength of the check.
- **Changes are validated one at a time.** The evaluation measures their combined effect, but an analysis does not, and changes can interact (W11, W12).
- **HypoPG estimates only.** Its cost gate filters quickly but can reject real wins (the W11 composite index). Lowering `QOPT_MIN_COST_GAIN` sends more candidates to real timing, at the cost of more experiments.
- **Timing noise.** A busy machine can flip borderline verdicts. The median of five and the 15% threshold absorb most of it.
- **Queries without a captured sample** get estimates only, because there are no parameter values to time with.
- **Write costs are estimated, not measured.** The score penalizes indexes on write-heavy tables, but qopt does not replay writes to measure the slowdown.
- **The shadow must resemble production.** Different data or statistics mean different plans, and timings that do not transfer.

31.2 Ideas for future work

- **Repeated evaluations** over several seeds and models, with confidence intervals, to settle questions like "does RAG help?".
- **Workload-level index selection:** choose the smallest set of indexes that serves many queries (W13's index also fixed W12), and weigh it against measured write overhead.
- **Validate change sets together** before recommending them, not only in the evaluation.
- **Better shadows:** copy planner statistics into a schema-only shadow (PostgreSQL 18 can export and import statistics), or use copy-on-write database branches for realistic data at low cost.
- **Formal equivalence** for simple rewrites, using an SQL equivalence checker alongside the data-based test.
- **Delivery:** open a pull request with a migration file for an accepted recommendation, or run qopt in CI against a staging database.

Glossary

| | |
|----------------------------------|---|
| Anti-join | A join that keeps rows with <i>no</i> match on the other side; how PostgreSQL executes <code>NOT EXISTS</code> efficiently. |
| auto_explain | PostgreSQL module that logs the actual execution plan of statements slower than a threshold. |
| B-tree | The default index type: a balanced, sorted tree supporting equality, ranges and ordering. |
| Baseline | The query's performance before any change: the median of five timed runs on the shadow. |
| BitmapOr | A plan step that combines the results of several index scans, letting <code>a = 1 OR b = 2</code> use two indexes. |
| BRIN | Block Range Index: stores min/max per range of pages; tiny, and useful for naturally ordered columns. |
| Candidate | A proposed fix (index, rewrite, statistics or setting) waiting for validation. |
| Correlation (pg_stats) | How closely a column's physical order matches its sorted order, from -1 to 1. |
| Cost | The planner's abstract estimate of work, in units of one sequential page read. |
| Covering index | An index containing every column a query needs (often via <code>INCLUDE</code>), enabling index-only scans. |
| DDL / DML | Statements that change structure (<code>CREATE INDEX</code>) / statements that change rows (<code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>). |
| Embedding | A vector of numbers representing a text's meaning; similar texts get nearby vectors. |
| EXCEPT ALL | SQL set difference that respects duplicates; used to compare two queries' results. |
| EXPLAIN / EXPLAIN ANALYZE | Show the planned execution / execute it and report actual rows and times. |
| Extended statistics | Statistics over several columns (<code>CREATE STATISTICS</code>) that capture correlations such as city → country. |
| Finding | A deterministic observation about a plan or query, such as <code>SORT_SPILL</code> ; qopt has fifteen kinds. |
| Fingerprint | A hash of a statement's structure that ignores constants and formatting; used to de-duplicate candidates. |
| Generic plan | A plan for a statement with <code>\$1</code> placeholders, built without concrete values (<code>EXPLAIN (GENERIC_PLAN)</code>). |
| GIN | Generalized Inverted Index: maps elements (array items, JSON keys, trigrams) to the rows containing them. |
| Hallucination | A model output that references something that does not exist, such as an invented column. |
| Heap | PostgreSQL's storage for a table's rows, in no particular order. |

| | |
|------------------------------|---|
| HNSW | A graph-based index for fast approximate nearest-neighbour search on vectors (pgvector). |
| HypoPG | Extension for hypothetical indexes: visible to the planner in one session, never built. |
| IMMUTABLE | A function whose result depends only on its arguments; required for index expressions. |
| Index-only scan | Answering a query from the index alone, without visiting the table. |
| LLM | Large language model, such as Gemini: a neural network that generates text from a prompt. |
| Median | The middle value of a sorted list; robust to outliers, used for all timings. |
| Meta database | qopt's own storage: documentation chunks, embeddings and analysis runs. |
| Misestimate / q-error | When estimated and actual row counts differ; the q-error is the larger ratio of the two. |
| Partial index | An index over only the rows matching a <code>WHERE</code> condition. |
| pg_stat_statements | Extension recording cumulative statistics per normalized statement. |
| pg_trgm | Extension providing trigram matching, so <code>LIKE '%x%'</code> can use a GIN index. |
| pgvector | Extension adding a vector type, distance operators and vector indexes. |
| Plan / planner | The tree of steps chosen to execute a query / the part of PostgreSQL that chooses it. |
| Query id | A 64-bit hash identifying a normalized statement, shared by <code>pg_stat_statements</code> and the logs. |
| RAG | Retrieval-augmented generation: retrieving documents and adding them to the model's prompt. |
| RRF | Reciprocal rank fusion: merges ranked lists by summing <code>1/(60 + rank)</code> . |
| Rollback | Undoing everything a transaction did, including DDL in PostgreSQL. |
| Sargable | Able to use an index ("search argument-able"); <code>date(col) = ...</code> is not, but a range on <code>col</code> is. |
| Savepoint | A marker inside a transaction that can be rolled back to, undoing only the work after it. |
| Self time | Time spent in a plan node itself, excluding its children. |
| Sequential scan | Reading every row of a table. |
| Shadow database | A copy of the target where qopt runs experiments inside rolled-back transactions. |
| Spill | A sort or hash exceeding <code>work_mem</code> and writing temporary files to disk. |
| Target database | The database being tuned; qopt only reads from it. |
| Token | A piece of text (roughly a word fragment) as processed by an LLM. |
| Transaction | A group of statements that succeed or fail together (<code>BEGIN ... COMMIT / ROLLBACK</code>). |
| Validation | qopt's checks that a candidate is safe, correct and measurably faster before it is recommended. |
| work_mem | Memory each sort or hash operation may use before spilling to disk (default 4 MB). |

CLI reference

Run every command as `npm run qopt -- <command> [options]`.

| Command | Options | What it does |
|--|---|---|
| <code>db setup</code> | <code>--scale <n></code> (1 = 600k orders), <code>--no-seed</code> | extensions, demo data on target and shadow, checksum, meta tables, reset statistics |
| <code>db status</code> | — | connectivity, versions, extensions, warnings, Gemini |
| <code>workload</code> | <code>--iterations <n></code> (20), <code>--scale</code> , <code>--seed</code> , <code>--queries W01,W07</code> , <code>--no-reset</code> | play the demo workload against the target |
| <code>capture</code> | <code>--limit <n></code> (15), <code>--json</code> | slowest statements, with sample availability |
| <code>analyze [sql]</code> | <code>-f/--file</code> , <code>--queryid</code> , <code>--workload</code> , <code>--top <n></code> , <code>--engine heuristic llm both</code> , <code>--no-rag</code> , <code>--no-validate</code> , <code>--retries <n></code> , <code>--plan</code> , <code>--json</code> | diagnose and propose validated fixes |
| <code>docs ingest</code> | <code>--version <major></code> , <code>--no-embed</code> | fetch, chunk, embed and store the manual pages |
| <code>docs search <query></code> | <code>--k <n></code> (5) | try the retriever |
| <code>docs status</code> | — | chunk counts per version |
| <code>eval</code> | <code>--configs</code> , <code>--queries</code> , <code>--scale</code> , <code>--seed</code> (42), <code>--model <name></code> , <code>--merge</code> | run the ablation study |
| <code>shadow sync</code> | <code>--schema-only</code> | refresh the shadow from the target with <code>pg_dump</code> <code>psql</code> |

HTTP API reference

Served by `apps/api` on `127.0.0.1:8787`. All bodies are JSON.

| Endpoint | Purpose |
|--|---|
| <code>GET /api/health</code> | database status, Gemini configuration, documentation counts, validation thresholds |
| <code>GET /api/queries?limit=20</code> | captured slow statements, with samples and warnings |
| <code>GET /api/workload</code> | the 15 demo queries, with their known fixes |
| <code>POST /api/workload/run</code> | <code>{ iterations }</code> : play the workload in the background (409 if one is already running) |
| <code>POST /api/analyze</code> | <code>{ sql queryid, label?, options? }</code> → <code>202 { id }</code> ; the analysis runs in the queue |
| <code>GET /api/runs/:id</code> | a live snapshot while running; the stored result afterwards |
| <code>GET /api/runs?limit=30</code> | recent runs, summarized |
| <code>GET /api/eval</code> | the latest evaluation report, the job status and the available configurations |
| <code>POST /api/eval</code> | <code>{ configs?, queries? }</code> : run an evaluation, merged into the saved report |
| <code>GET /api/jobs</code> | workload and evaluation job status with a log tail |

The 15 demo queries

The SQL as run by the evaluation (parameters from seed 42), and the known correct fix that serves as ground truth.

W01 · Customer order history

```
SELECT id, status, total, created_at
FROM orders
WHERE customer_id = 60111
ORDER BY created_at DESC LIMIT 20
```

Known fix: btree on orders (customer_id, created_at)

W02 · Order line items

```
SELECT product_id, quantity, unit_price
FROM order_items
WHERE order_id = 599887
```

Known fix: btree on order_items (order_id)

W03 · Login by email (case-insensitive)

```
SELECT id, full_name, tier
FROM customers
WHERE lower(email) = lower('user83619@example.com')
```

Known fix: expression index on customers (lower(email))

W04 · Recent pending orders

```
SELECT id, customer_id, total
FROM orders
WHERE status = 'pending' AND created_at >= '2026-06-29'
ORDER BY created_at
```

Known fix: btree on orders (status, created_at) or partial index WHERE status = pending

W05 · Limited-edition products (array containment)

```
SELECT id, name, price
FROM products
WHERE tags @> ARRAY['limited']
```

Known fix: GIN on products (tags)

W06 · Products by JSON attribute

```
SELECT id, name
FROM products
WHERE attributes @> '{"color": "teal"}'
```

Known fix: GIN on products (attributes jsonb_path_ops)

W07 · Review text search (leading wildcard)

```
SELECT id, product_id, rating
FROM reviews
WHERE body ILIKE '%overheat%'
```

Known fix: GIN on reviews (body gin_trgm_ops)

W08 · Event counts in a time window

```
SELECT type, count(*)
FROM events
WHERE created_at >= '2026-05-31 12:00+00' AND created_at < timestampz '2026-05-31 12:00+00' + interval '2 hours'
GROUP BY type
```

Known fix: BRIN or btree on events (created_at)

W09 · Customers without recent orders (NOT IN)

```
SELECT id, email
FROM customers
WHERE id NOT IN (SELECT customer_id
FROM orders
WHERE created_at >= '2026-05-30')
ORDER BY id LIMIT 100
```

Known fix: NOT EXISTS rewrite (+ index on orders.customer_id)

W10 · Daily revenue via date() on a timestampz

```
SELECT count(*), sum(total)
FROM orders
WHERE date(created_at) = '2026-03-30'
```

Known fix: sargable range rewrite + btree on orders (created_at)

W11 · Orders from Auckland customers (correlated columns)

```
SELECT c.id, count(o.id) AS orders
FROM customers c
JOIN orders o ON o.customer_id = c.id
WHERE c.country = 'NZ' AND c.city = 'Auckland'
GROUP BY c.id
```

Known fix: CREATE STATISTICS on customers (country, city) + btree on orders (customer_id)

W12 · Revenue for premium electronics

```
SELECT p.category, sum(oi.quantity * oi.unit_price) AS revenue, count(*) AS items
FROM order_items oi
JOIN products p ON p.id = oi.product_id
WHERE p.category = 'electronics' AND p.price > 1384
GROUP BY p.category
```

Known fix: btree on order_items (product_id)

W13 · Top products by distinct orders (sort spill)

```
SELECT product_id, count(DISTINCT order_id) AS orders
FROM order_items
GROUP BY product_id
ORDER BY orders DESC LIMIT 10
```

Known fix: btree on order_items (product_id, order_id) for a sort-free index-only scan

W14 · Deep OFFSET pagination

```
SELECT id, created_at, total
FROM orders
ORDER BY created_at DESC OFFSET 49000 LIMIT 50
```

Known fix: btree on orders (created_at); keyset pagination in the app

W15 · Lookup by email OR phone

```
SELECT id, full_name
FROM customers
WHERE email = 'User15504@Example.com' OR phone = '+1-555-7592061'
```

Known fix: btree on customers (email) and (phone) for a BitmapOr, or UNION rewrite